Passenger Assignment Model in a Public Transportation Network

Michel Bierlaire Evangelos Paschalidis

August 14, 2026

Part I
Problem Definition and Identifiability

1 Introduction

This document describes the modeling assumptions, mathematical formulation, and inference components implemented in the software. It is intended as technical documentation for users and developers of the package.

The objective of this software is to infer an aggregate representation of public transportation demand from operational data. The demand is characterized by a flow for each origin–destination (OD) pair and each desired departure time interval.

The software follows four main steps:

1. Develop a detailed network representation of the public transportation system.
2. Formulate an assignment model that maps an OD matrix to passenger flows on the network.
3. Define measurement equations linking model outputs to observed data, yielding either a likelihood or an explicitly weighted linear objective.
4. Detect topology-driven structural-zero OD cells before estimation.
5. Estimate demand using ML, MAP, fixed-routing linear and block-coordinate methods, or Bayesian variational inference.

2 Modeling Elements

The following sections document the mathematical objects and modeling assumptions used internally by the software.

2.1 Spatial Representation

Let $\mathcal{S}$ denote the set of stops in the area of interest. Each stop can act as an origin, a destination, or a transfer location.

Definition 1 (OD pair)**.** *An origin–destination pair is a tuple*

$$\mathfrak{q} = (\mathfrak{s}_\mathfrak{o}, \mathfrak{s}_\mathfrak{d}) \in \mathcal{Q} = \mathcal{S} \times \mathcal{S}.$$

2.2 Temporal Representation

The analysis period is partitioned into a set $\mathcal{T}$ of departure time intervals. Each interval represents a desired departure time window.

Definition 2 (Departure time interval)**.** *Each time bin* $t \in \mathcal{T}$ *corresponds to a half-open interval*

$$[a_t, b_t) \subset \mathbb{R}, \qquad a_t < b_t,$$

where a_t *and* b_t *denote the start and end time (in minutes) of the desired departure interval. The end of the final bin may be included by an explicit analysis-period boundary convention. Half-open membership prevents a departure on a shared boundary from belonging to two adjacent bins.*

For each departure interval $t \in \mathcal{T}$, the desired departure window is the interval $[a_t, b_t)$. The admissible access window introduced below is a separate, closed eligibility interval and does not change this unique bin membership.

Definition 3 (OD demand)**.** *For each OD pair* $q \in \mathcal{Q}$ *and departure time interval* $t \in \mathcal{T}$*, let*

$$f_q^t \geqslant 0$$

denote the number of passengers wishing to depart during interval t *from origin* s_o *to destination* s_d*.*

The full demand vector is denoted by

$$f = \{f_q^t : q \in \mathcal{Q},\ t \in \mathcal{T}\}.$$

2.3 Passenger Journeys, Vehicle Legs, and Events

OD demand represents passenger journeys from a first boarding at the origin to a final alighting at the destination. A journey with L vehicle legs contains $L - 1$ transfers and generates one boarding and one alighting on every leg. It therefore contributes one initial boarding, one final alighting, $L - 1$ transfer boardings, and $L - 1$ transfer alightings. Raw boarding and alighting totals are not journey-origin and journey-destination marginals when transfers occur.

For a conditional destination model, Q_{ot} denotes journeys whose first boarding occurs at origin o in departure period t. It must come from an external journey-entry source or a declared low-dimensional production model; it cannot silently be equated to all boardings at (o, t). Every model must state which productions, attractiveness values, routing shares, detection factors, and dispersion parameters are fixed, normalized, estimated, or regularized.

2.4 Physical Stops and Timetable Identity

Scenario-stop identifiers define the external OD and measurement geography. An explicit mapping may aggregate platforms into physical stop places for transfer construction; without one, the mapping is the identity. Such a mapping must cover every scenario stop exactly once and is part of the model assumptions. A route pattern is identified by line, direction, and its complete ordered physical-stop sequence. Scheduled trips retain their exact event times, so opposite directions, express patterns, platform variants, and different service periods remain distinct unless an explicit mapping states otherwise.

2.5 Observation Resolution

The atomic observation is a boarding or alighting count attached to an unambiguous scheduled event. Stop-, route-, period-, or vehicle-level totals are derived aggregates with an explicit mapping. An absent record is unobserved, whereas a recorded zero is an observed zero. This distinction is preserved in the likelihood.

2.6 Free, Frozen, and Structurally Zero Demand

Let $\mathcal{U}$ denote the free OD-time cells and $\mathcal{F}$ the frozen cells. Only $\mathbf{f}_{\mathcal{U}}$ belongs to the estimation parameterization; fixed values $\mathbf{f}_{\mathcal{F}}$ are constants. Frozen-zero cells contribute neither a parameter nor a forward-response column. Positive frozen cells contribute a fixed observation offset. The complete demand vector is reconstructed only when a complete reported OD table is required.

Structural zeros are frozen zeros justified before estimation by declared timetable rules. A cell is excluded when no admissible path exists or when every path exceeds the accepted transfer bound; optional rules may additionally limit initial wait, journey time, or service frequency. Classification is performed on OD-time candidates using the same access, transfer, and physical-stop assumptions as routing. A nonzero pre-existing fixed value that conflicts with a detected structural zero is an error rather than an implicit overwrite.

Connectivity is therefore defined at the spatial resolution of the scenario stops used by the OD table. These identifiers may be individual platforms. A pair that is disconnected under the identity mapping may become connected when an authoritative or reviewed platform-to-physical-stop mapping supplies the interchange. Even at physical-stop level, connectivity remains time dependent: directionality, service hours, access windows, minimum transfer time, and the transfer bound may make a particular OD-time cell infeasible. "No path" and "a path exists but requires too many transfers" are recorded as distinct structural-zero reasons.

3 Intrinsic Underdetermination of OD Estimation

OD estimation from passenger counts is an inverse problem. Let $\mathbf{f} \in \mathbb{R}^{\mathrm{n}}_{+}$ collect the unknown OD-time demands and let $\mathbf{y} \in \mathbb{N}^{\mathrm{m}}$ collect the observed boarding and alighting counts. For fixed network and route-choice assumptions, the expected observations can be written schematically as

$$\mathbb{E}[\mathbf{y} \mid \mathbf{f}] = \boldsymbol{\mu}(\mathbf{f}).$$

In the fixed-routing case this relationship is affine,

$$\boldsymbol{\mu}(\mathbf{f}) = \rho\left(\mathsf{B}\mathbf{f} + \mathbf{b}^{\mathrm{fix}}\right), \tag{1}$$

where a column of B is the expected pattern of observed events generated by one unit of demand in an OD-time cell. This compact expression exposes the central difficulty: passenger counts observe events along journeys, not the journeys' origins and final destinations directly.

The problem is underdetermined whenever distinct feasible demands produce the same expected counts. In the linear case, if a nonzero direction $\boldsymbol{\nu}$ satisfies $\mathsf{B}\boldsymbol{\nu} = \mathbf{0}$, then $\mathbf{f}$ and $\mathbf{f}+\boldsymbol{\nu}$ are observationally equivalent whenever both are feasible. More generally, nearly collinear columns of B create weakly identified directions: the data can distinguish them only through small differences that may be overwhelmed by measurement noise. Having more count records than unknowns is therefore neither necessary nor sufficient for identification; the rank and conditioning of the response, the observation coverage, and the imposed model restrictions matter.

Several common features aggravate this ambiguity. OD pairs may share most of their vehicle legs, transfer boardings are indistinguishable from initial boardings in raw stop totals, and aggregation over stops or time can erase the few events that distinguish two journeys. Missing observations create no constraint at all. Even counts at every observed boarding and alighting event need not uniquely identify every OD-time cell, because those events are linked through unobserved passenger trajectories.

Consequently, an estimator cannot manufacture information absent from the observations. Maximum likelihood selects among demands using only the likelihood and may be unstable or nonunique in weak directions. MAP estimation adds explicit prior or regularization assumptions and selects a stable point, but stability must not be confused with empirical identification. A

reduced demand model, such as gravity, constrains many OD cells through a small number of shared parameters; this can make the parameter vector identifiable while remaining conditional on stronger behavioral and production assumptions. Bayesian inference expresses the remaining uncertainty, but its posterior is also shaped by these assumptions where the likelihood is weak.

The report therefore distinguishes three questions throughout:

1. *structural feasibility*: can the timetable connect the OD-time pair under the accepted transfer and waiting rules?
2. *statistical identification*: do the available observations distinguish the retained parameters?
3. *model adequacy*: does the fitted model reproduce relevant count patterns without systematic residual structure?

A structural zero answers only the first question. Regularization or dimension reduction addresses the second through assumptions. Neither guarantees the third, which must be assessed separately.

Part II
Mathematical Forward Models

Purpose and Organization of the Forward Models

The estimation methods developed later require a map from OD-time demand to the counts that would be observed under that demand. This part constructs that map in stages. Once routing is fixed, the computational objective is to obtain and reuse the measurement-response matrix B, or an operator that applies it without reconstructing routes. The detailed assignment model is the behavioral reference: it represents scheduled events, access, transfers, generalized costs, route choice, and passenger-flow propagation on the complete time-dependent network. The reduced journey-response model expresses the same estimation interface in terms of feasible scheduled journeys and their expected measurement events. The RAPTOR example illustrates how those feasible journeys are found; it is an example of the reduced model, not a separate forward model. RAPTOR is the current practical construction mechanism, but it is not essential to the later estimator: a time-expanded preprocessing that constructed the same B once would provide the same estimation-stage advantage.

The remaining sections describe two complementary layers. The additive operator decomposition exploits linearity once routing is fixed, allowing the same response to be evaluated in complete, compressed, or sharded form without changing its mathematical meaning. The observation model then maps the predicted events to count means and supplies the statistical likelihood. Thus the detailed and reduced assignments are alternative representations of the routing response, while operator decomposition and the likelihood are downstream components that can be combined with either representation when their assumptions match. They are not competing demand models; those are introduced separately in Part III.

4 Detailed Assignment Model

This model provides the most explicit representation of passenger movements used in the report. Its purpose is to define the behavioral reference against which reduced or fixed-routing responses are interpreted and validated. The section first states its scope, then constructs the time-expanded network and costs, and finally derives route probabilities and flow propagation.

4.1 Scope and Assumptions

The detailed forward model maps passenger-journey demand to expected movements and observed events on a scheduled public-transport network. Its mathematical interpretation relies on the following assumptions.

1. The timetable, stop geography, admissible access window, transfer rules, and generalized link costs are exogenous during one assignment.

2. Demand is indexed by origin, final destination, and desired-departure interval. It represents complete passenger journeys, not individual vehicle legs.

3. Route choice follows the stated Dial–logit construction on the admissible time-expanded graph. The dispersion parameter and all cost coefficients are common inputs unless a model explicitly defines segmentation.

4. Passengers do not alter travel times, vehicle capacity, or route-choice probabilities. There is no denied boarding or crowding feedback in the current model.

5. Conditional on demand and routing probabilities, expected flows are additive across OD-time cells. This is an expectation model; it does not preserve individual simulated passenger identities.

These assumptions define the domain in which fixed-routing loading is exact. Changing a routing-sensitive input requires a new routing calculation, while changing demand alone does not.

Fixed routing does not impose one route or one vector of route shares on a physical OD pair for the entire day. Each departure-time cell accesses its own scheduled trips and connections and can therefore have different alternatives and probabilities. "Fixed" means only that these cell-specific probabilities are held constant while demand parameters change. Because crowding, capacity, and denied boarding do not currently feed back into costs or availability, this is consistent with the present assignment. A capacity-dependent model would instead require routing and demand estimation to be coupled.

4.2 Departure-Time and Access Assumptions

Desired departure interval and admissible boarding window. For each OD pair $\mathsf{q} \in \mathcal{Q}$ and departure time interval $\mathsf{t} \in \mathcal{T}$, we associate the desired departure *interval* $[\mathfrak{a}_\mathsf{t}, \mathfrak{b}_\mathsf{t}]$ and an admissible schedule-deviation window of half-width $\Delta^{\text{access}}_{\max} > 0$ (user-defined, e.g. 15 minutes).

Admissible access links. An access link is included in the graph *only if* the departure time τ is not too far from the desired interval, namely

$$\tau \in \left[\mathfrak{a}_\mathsf{t} - \Delta^{\text{access}}_{\max},\ \mathfrak{b}_\mathsf{t} + \Delta^{\text{access}}_{\max}\right],$$

where τ denote the actual departure time (in minutes) of a boarding at the origin stop. As described later, the boarding is modeled by a link, that is included if the distance from τ to the interval $[\mathfrak{a}_\mathsf{t}, \mathfrak{b}_\mathsf{t}]$ does not exceed $\Delta^{\text{access}}_{\max}$.

Schedule-deviation penalty. When the access link is included, its generalized cost is defined by a (possibly asymmetric) piecewise-linear penalty with a *flat* (zero-penalty) region inside the interval:

$$\mathfrak{c}_{\text{access}}(\tau; \mathsf{t}) = \beta_{\text{early}}\ \max\big(0,\ \mathfrak{a}_\mathsf{t} - \tau\big) + \beta_{\text{late}}\ \max\big(0,\ \tau - \mathfrak{b}_\mathsf{t}\big),$$

where $\beta_{\text{early}} \geqslant 0$ and $\beta_{\text{late}} \geqslant 0$ are user-defined coefficients. Therefore,

$$\mathfrak{c}_{\text{access}}(\tau; \mathsf{t}) = 0 \quad \text{whenever} \quad \tau \in [\mathfrak{a}_\mathsf{t}, \mathfrak{b}_\mathsf{t}],$$

and penalties apply only to departures strictly earlier than $\mathfrak{a}_\mathsf{t}$ or strictly later than $\mathfrak{b}_\mathsf{t}$.

Note that the thresholds $\Delta^{\text{access}}_{\max}$ and $\Delta^{\text{transfer}}_{\max}$ are fixed configuration parameters; the assignment is differentiable w.r.t. costs and OD flows given a fixed graph.

4.3 Network Representation

The public transportation system is represented as a directed time-expanded graph. All timetable times (arrival/departure event times) are expressed in minutes on the same time axis as the departure-time intervals.

A key modeling choice is (i) the separation of each stop centroid into two distinct nodes (an origin *centroid-in* and a destination *centroid-out*), and (ii) the split of each timetable event into an *arrival* and a *departure* node. This second split prevents ill-defined instantaneous moves (e.g., transferring or boarding at the exact same time as arrival) and ensures that all links respect a strict chronological (or lexicographic) order, so that the resulting time-expanded network is acyclic.

Definition 4 (Nodes)**.** *The node set consists of three families of nodes:*

- ***Centroid-in nodes (non time-tagged, indexed by departure interval):*** *for each stop* $\mathtt{s} \in \mathcal{S}$ *and for each departure interval* $\mathtt{t} \in \mathcal{T}$*, a node*

$$(\mathtt{s}, \mathtt{t})^{\text{in}}$$

  used as a demand-injection handle for passengers departing from stop $\mathtt{s}$ *with desired departure interval* $\mathtt{t}$*. These nodes do not correspond to a physical time instant; they are placed* before all event nodes *in the topological order (conceptually at* $-\infty$*). The interval index* $\mathtt{t}$ *is used only to (i) select admissible boarding events through the access-window rule, and (ii) compute schedule-deviation penalties.*

- ***Centroid-out nodes (non time-tagged):*** *for each stop* $\mathtt{s} \in \mathcal{S}$*, a node* $\mathtt{s}^{\text{out}}$ *representing the point where passengers arriving at stop* $\mathtt{s}$ *exit the network. These nodes are not associated with a physical time; they are placed* after all event nodes *in the topological order (conceptually at* $+\infty$*) so that the global graph remains acyclic.*

- ***Timetable event nodes (trip-specific, split into arrival and departure):*** *for each stop* $\mathtt{s}$*, each scheduled arrival time* τ^{arr} *and departure time* τ^{dep} *associated with a given vehicle run (trip), we create two nodes*

$$(\mathtt{s}, \tau^{\text{arr}})^{\text{arr}} \qquad \textit{and} \qquad (\mathtt{s}, \tau^{\text{dep}})^{\text{dep}}.$$

  These nodes are trip-specific*: if two trips serve the same stop at the same clock time, they still correspond to different event nodes.*

Definition 5 (Lines and event-to-line mapping)**.** *Let* $\mathcal{L}$ *denote the set of public transport lines. Each timetable event node (arrival or departure) is associated with exactly one operated line, captured by the mapping*

$$\lambda : \mathcal{V}^{\text{event}} \to \mathcal{L}, \qquad v \mapsto \lambda(v),$$

where $\mathcal{V}^{\text{event}}$ *is the set of timetable event nodes. In practice,* $\lambda(v)$ *is derived from the timetable: event nodes created from a given trip inherit the line identifier of that trip. Therefore, for a given trip at stop* $\mathtt{s}$*, the corresponding arrival and departure nodes share the same line label.*

Centroid-in nodes are not time-tagged: they are placed before all timetable event nodes (conceptually at $-\infty$) so that access links can connect to departure events that occur either before or after the desired departure window without violating acyclicity. Centroid-out nodes are not time-tagged and are placed after all event nodes (conceptually at $+\infty$), preserving a global topological order.

Remark. The separation of centroid-in and centroid-out nodes is essential for the validity of the forward dynamic-programming loading algorithm. Without this separation, egress links could break the acyclic structure and violate the topological order implied by time.

Definition 6 (Links)**.** *The link set* $\mathcal{A}$ *contains the following link types:*

1. ***Access links (time-windowed):*** *for an OD pair* $\mathtt{q} = (\mathtt{s_o}, \mathtt{s_d})$ *and departure interval* $\mathtt{t}$*, access links connect the origin centroid-in node* $(\mathtt{s_o}, \mathtt{t})^{\text{in}}$ *to* selected departure *event nodes at the origin stop,*

$$(\mathtt{s_o}, \mathtt{t})^{\text{in}} \to (\mathtt{s_o}, \tau)^{\text{dep}},$$

   only for departure times τ *satisfying*

$$\tau \in \left[\mathtt{a_t} - \Delta^{\text{access}}_{\max},\ \mathtt{b_t} + \Delta^{\text{access}}_{\max}\right].$$

2. ***Ride links:*** *for each trip, ride links connect a departure event at the current stop to an arrival event at the next stop,*

$$(\mathtt{s}_\mathtt{i}, \tau_\mathtt{i}^{\mathrm{dep}})^{\mathrm{dep}} \to (\mathtt{s}_{\mathtt{i}+1}, \tau_{\mathtt{i}+1}^{\mathrm{arr}})^{\mathrm{arr}}.$$

3. ***Dwell/continue links (within-trip at a stop):*** *for each trip and each stop event, a dwell link connects the arrival and departure nodes at the same stop,*

$$(\mathtt{s}, \tau^{\mathrm{arr}})^{\mathrm{arr}} \to (\mathtt{s}, \tau^{\mathrm{dep}})^{\mathrm{dep}}.$$

In the implementation, strictly positive dwell time is enforced by regularizing timetable records where arrival equals departure (using a small minimum dwell time).

4. ***Transfer links (inter-line, time-windowed):*** *transfer links connect an* arrival *event node to a later* departure *event node at the same stop and represent waiting and switching* between different lines*. A transfer link*

$$(\mathtt{s}, \tau_1)^{\mathrm{arr}} \to (\mathtt{s}, \tau_2)^{\mathrm{dep}}, \qquad \textit{with} \qquad \tau_2 > \tau_1$$

is included only if

   (a) the two event nodes belong to different lines*, and*

   (b) the waiting time satisfies $\tau_2 - \tau_1 \leqslant \Delta_{\max}^{\mathrm{transfer}}$*.*

No transfer links are created between events of the same line. In particular, transfer links are never created for $\tau_2 = \tau_1$*.*

5. ***Egress links (global graph, destination-gated by masking):*** *the global graph contains egress links from every arrival event node at stop* $\mathtt{s}$ *to the centroid-out node* $\mathtt{s}^{\mathrm{out}}$*:*

$$(\mathtt{s}, \tau)^{\mathrm{arr}} \to \mathtt{s}^{\mathrm{out}}.$$

For a given OD pair $\mathtt{q} = (\mathtt{s}_\mathtt{o}, \mathtt{s}_\mathtt{d})$*, only the destination centroid-out node* $\mathtt{s}_\mathtt{d}^{\mathrm{out}}$ *is enabled as a terminal node; all other egress options are disabled by a destination-dependent mask.*

Each link $\ell \in \mathcal{A}$ is associated with:

- a generalized cost $\mathtt{c}_\ell$,
- a capacity $\mathtt{u}_\ell$ for ride links (stored in the data structure, not yet used in costs).

Assumption 1 (Acyclic time-expanded network)**.** *All links in the time-expanded network are consistent with a strict topological order. All* event-to-event *links strictly increase clock time. In addition, centroid-in nodes* $(\mathtt{s}, \mathtt{t})^{\mathrm{in}}$ *are placed* before *all event nodes (conceptually at* $-\infty$*), and centroid-out nodes* $\mathtt{s}^{\mathrm{out}}$ *are placed* after *all event nodes (conceptually at* $+\infty$*). With this convention, access and egress links always respect the global ordering.*

With the event split, feasible movements follow the pattern

$$(\mathtt{s}, \mathtt{t})^{\mathrm{in}} \to (\mathtt{s}, \tau)^{\mathrm{dep}} \to (\mathtt{s}', \tau')^{\mathrm{arr}} \to (\mathtt{s}', \tau'')^{\mathrm{dep}} \to \cdots \to (\mathtt{s}_\mathtt{d}, \bar{\tau})^{\mathrm{arr}} \to \mathtt{s}_\mathtt{d}^{\mathrm{out}}.$$

Transfer links satisfy $\tau_2 > \tau_1$ *and dwell links are enforced to be strictly forward in time through a minimum dwell regularization. Therefore the directed graph is acyclic and admits a topological ordering, which allows value functions and flows to be computed using dynamic programming.*

4.4 Generalized Cost

Each passenger seeks to minimize a generalized cost that includes:

- in-vehicle travel time (ride links),
- waiting and transfer time (transfer links) and dwell time at stops (dwell/continue links),
- transfer disutility through coefficient β_{transfer},
- early/late departure penalty relative to the desired departure *interval* (access links; zero penalty within $[a_t, b_t]$).

For a path p, the generalized cost is

$$C(p) = \sum_{\ell \in p} c_\ell.$$

4.4.1 Link-type-specific cost formulation

The generalized cost of each link depends on its type. All coefficients appearing below are fixed user-defined parameters.

Ride links. For a ride link ℓ representing the movement of a vehicle between two stops, connecting a departure event to a subsequent arrival event,

$$(s_i, \tau_i^{\text{dep}})^{\text{dep}} \to (s_{i+1}, \tau_{i+1}^{\text{arr}})^{\text{arr}},$$

the generalized cost is equal to the in-vehicle travel time:

$$c_\ell = \tau_{i+1}^{\text{arr}} - \tau_i^{\text{dep}}.$$

Dwell/continue links. For a dwell/continue link at a stop,

$$(s, \tau^{\text{arr}})^{\text{arr}} \to (s, \tau^{\text{dep}})^{\text{dep}},$$

the generalized cost is the dwell time at the stop:

$$c_\ell = \tau^{\text{dep}} - \tau^{\text{arr}}.$$

Transfer links (inter-line, bounded waiting). Consider a transfer link $\ell : (s, \tau_1)^{\text{arr}} \to (s, \tau_2)^{\text{dep}}$ with $\tau_2 > \tau_1$. Such a link is included only if (i) the two events correspond to *different lines* and (ii) the waiting time $\tau_2 - \tau_1$ does not exceed the user-defined threshold $\Delta_{\max}^{\text{transfer}}$. When included, its generalized cost is

$$c_\ell = \beta_{\text{transfer}} \, (\tau_2 - \tau_1),$$

where $\beta_{\text{transfer}} > 0$ is a user-defined coefficient.

Access links (bounded schedule deviation with a flat region). Consider an access link from the centroid-in node $(s_o, t)^{\text{in}}$ to an origin *departure* event node $(s_o, \tau)^{\text{dep}}$. The link is included only if

$$\tau \in \left[a_t - \Delta_{\max}^{\text{access}}, \; b_t + \Delta_{\max}^{\text{access}} \right].$$

When included, its generalized cost is

$$c_\ell = \beta_{\text{early}} \max(0, a_t - \tau) + \beta_{\text{late}} \max(0, \tau - b_t),$$

with user-defined coefficients $\beta_{\text{early}} \geqslant 0$ and $\beta_{\text{late}} \geqslant 0$.

Egress links to centroids. For an egress link $\ell : (\mathsf{s}, \tau)^{\text{arr}} \to \mathsf{s}^{\text{out}}$, we set $\mathsf{c}_\ell = 0$.

Interpretation of cost coefficients. All time quantities in the model are measured in minutes. Travelers may perceive these minutes differently depending on context. The coefficients β_{transfer}, β_{early} and β_{late} act as perception weights converting physical time into generalized cost. These coefficients are fixed and specified by the user; they are not estimated within the inference procedure. Only the OD demand and optionally the dispersion parameter θ are estimated.

5 Assignment Model

5.1 Input

The input is the OD vector $\mathbf{f}$ (scenario order), which is injected through centroid-in nodes.

5.2 Assignment Procedure

The classical assignment procedure based on deterministic shortest paths is not differentiable with respect to link costs and is therefore not well suited for gradient-based calibration methods. We adopt a differentiable assignment model inspired by the logit-based approach of Dial (1971), adapted to the acyclic time-expanded public transportation network.

5.3 Differentiable Dial-style assignment

Let $\mathbf{f}$ denote the OD demand. For each OD pair q and departure interval t, passengers choose routes according to a logit model defined on the time-expanded network.

5.3.1 Capacity constraints (current status)

Although ride links are associated with nominal capacities u_ℓ in the data structure, the current implementation does not incorporate capacity-dependent penalties into generalized costs. All assignment computations are therefore performed without congestion or overload effects.

Introducing capacity-aware penalties while preserving differentiability (e.g., using smooth overload functions such as softplus penalties) is a natural future extension, but it is not part of the current version of the software.

5.3.2 Meaning of fixed routing

The software does not store one canonical physical route per OD pair. It builds one time-expanded graph for the complete timetable, with event nodes tied to scheduled trips and one origin centroid for every stop and departure-time bin. Feasible alternatives are implicit link sequences in this graph rather than an enumerated path table. Access links from a time-bin centroid reach only departures in its configured time window and carry the corresponding schedule- deviation costs. Thus, the same physical OD in two time bins can face different lines and scheduled trips; even if its physical stop or line sequence is unchanged, its trip sequence can differ.

For each destination, the Dial recursion produces one conditional probability per enabled time-expanded link. A demand time bin is an aggregation interval: all demand in one OD-time cell is injected at the same time-bin centroid and shares its distribution over accessible departure events. The current model contains no finer within-bin departure-cohort index.

In the ordinary assignment call these destination value functions and link probabilities are recomputed before demand is loaded. In *fixed routing*, the effective link masks and probabilities are computed once for a fixed graph, timetable, generalized-cost vector, destination masks,

and dispersion parameter θ, then reused for every demand evaluation. Complete, sharded, concurrent, matrix-free, and gravity-model operators preserve this same routing contract. The gravity model changes how OD-time demand is generated; it does not define different routing probabilities.

Consequently, "the route set is constant throughout the day" and "route-choice probabilities are constant throughout the day" are false in general. "Routing probabilities are held fixed during fixed-routing estimation" is true. Furthermore, routing is currently independent of demand and crowding: nominal ride-link capacities are stored as metadata but do not cap boarding, generate denied boarding, change passenger propagation, or enter utility. Changing only OD demand therefore leaves probabilities unchanged, and fixed loading agrees with a freshly recomputed assignment when all routing-sensitive inputs are unchanged.

If crowding or capacity becomes endogenous, fixed routing would no longer be exact. A possible outer fixed-point scheme would alternate demand estimation, capacity-aware dynamic assignment, routing-cache reconstruction, and re-estimation until demand and assigned flows stabilize.

5.3.3 Logit routing model

Passengers are assumed to choose routes according to a logit model with dispersion parameter $\theta > 0$. For any path p,

$$\Pr(p) \propto \exp\left(-\frac{C(p)}{\theta}\right), \qquad C(p) = \sum_{\ell \in p} c_\ell.$$

When θ is small, flows concentrate on lowest-cost paths. When θ is large, flows spread across multiple attractive alternatives.

Computational and behavioral motivation. The logit formulation admits an efficient dynamic-programming implementation on the acyclic time-expanded network. It is differentiable with respect to generalized costs and, consequently, with respect to OD demand. This property is essential when the assignment model is embedded in gradient-based calibration and variational inference.

5.3.4 Flow propagation

Fix an OD pair $q = (s_o, s_d)$ and a departure time interval $t \in \mathcal{T}$. The corresponding demand f_q^t is injected at the origin centroid-in node $(s_o, t)^{\text{in}}$.

Let y_i denote the flow reaching node i during the forward pass. Initialization is

$$y_{(s_o,t)^{\text{in}}} = f_q^t, \qquad y_i = 0 \quad \text{for all } i \neq (s_o, t)^{\text{in}}.$$

Given transition probabilities $P_\ell(i)$ on outgoing links $\ell = (i \to j)$, link flows are

$$x_\ell = y_i \, P_\ell(i), \qquad \ell = (i \to j),$$

and node flows are propagated in topological order by

$$y_j \leftarrow y_j + x_\ell, \qquad \ell = (i \to j).$$

Destination gating. The global graph contains egress links for all stops, but for a given OD pair $q = (s_o, s_d)$ the model enables only egress links leading to s_d^{out} via a destination-dependent mask.

Validity and fallback. The direct operator is valid only while dispersion, routing probabilities, link costs, graph structure, OD ordering, and the measurement mapping remain fixed. It is invalid when dispersion is estimated, costs depend on demand, capacity or congestion feeds back into routing, or graph, layout, or mapping indices change. The normal fixed-routing loader remains the reference and fallback implementation and is always required when link-level outputs are requested. The estimated-dispersion path never constructs or uses the direct operator.

6 Reduced Journey-Response Model

Repeated evaluation of the complete event graph is unnecessary when routing conditions are held fixed during demand estimation. The reduced model therefore summarizes, for every retained OD-time cell, its feasible journeys, fixed conditional shares, and expected contributions to observations. This section defines that representation mathematically before the following subsection illustrates its timetable search on a small network.

6.1 Purpose and Assumptions

The reduced model replaces repeated detailed assignment by a response for each retained OD-time alternative. It assumes that timetable feasibility and the conditional distribution over scheduled journeys are computed before demand estimation and remain fixed during that estimation. Each response records the expected contribution of one passenger journey to every modeled measurement. Responses are additive, and frozen positive demand contributes a fixed offset. Structural-zero cells have no response column and no estimated parameter.

This reduction is exact relative to its constructed journey choices when their shares and the observation mapping remain fixed. It is not a behavioral claim that passengers use the same route throughout the day: every OD-time cell may have a different feasible set and different shares. Nor does it represent endogenous crowding. Approximation can enter through bounded transfer depth, choice-set pruning, temporal aggregation, or a deliberately simplified demand model; these assumptions must be distinguished from the algebra of the response operator itself.

Journey construction and the initial shares use only the timetable, stop and transfer topology, departure-time definition, generalized journey attributes, and declared choice rules. Observed passenger counts are not used to select, prune, or weight journeys. They enter only after the fixed response B has been constructed, through the likelihood used to estimate demand or gravity parameters. This separation prevents the same counts from silently determining both the response design and its fitted demand.

6.2 Desired-Departure-Time Averaging

A broad demand period must not be represented by whichever scheduled vehicle happens to be convenient for preprocessing. Desired passenger departure time is defined independently of supply. For OD-period cell $\mathsf{c} = (\mathsf{o}, \mathsf{d}, \mathsf{t})$, the initial model assumes

$$\mathsf{T}^*_{\mathsf{c}} \sim \mathrm{Uniform}(\mathsf{a}_{\mathsf{t}}, \mathsf{b}_{\mathsf{t}}).$$

With S_{t} equal-width midpoint samples,

$$\tau_{\mathsf{ts}} = \mathsf{a}_{\mathsf{t}} + \left(\mathsf{s} + \frac{1}{2}\right)\frac{\mathsf{b}_{\mathsf{t}} - \mathsf{a}_{\mathsf{t}}}{\mathsf{S}_{\mathsf{t}}}, \qquad w_{\mathsf{ts}} = \frac{1}{\mathsf{S}_{\mathsf{t}}}, \qquad \mathsf{s} = 0, \ldots, \mathsf{S}_{\mathsf{t}} - 1.$$

The timetable is queried only after τ_{ts} and w_{ts} have been fixed. Thus increased service frequency cannot itself attract more desired-time weight. Uniform desired departures are an explicit assumption, not an empirical conclusion.

Equal-count midpoints are retained for compatibility. A fixed-step reference instead partitions $[a_t, b_t]$ into intervals of prescribed maximum duration and gives every midpoint its exact elapsed-duration weight, including a shorter final interval. Adaptive service-aware quadrature begins with coarse subintervals, compares cached sparse journey or measurement responses, and bisects only where feasibility, selected services, sparse support, or response values change. Timetable information selects numerical evaluation locations; it never changes the uniform elapsed-time mass. Sparse differences use a normalized symmetric $\ell\ell_1$ measure over the union of supports.

Let $A_{c,s}$ be the measurement response generated from journeys feasible for sample s. The averaged response is

$$\bar{A}_c = \sum_s w_{ts} A_{c,s}.$$

For the default completed-journey interpretation, infeasible samples retain their original mass in diagnostics, while the served response conditions on the feasible set F_c:

$$p_c^{\text{feasible}} = \sum_{s \in F_c} w_{ts}, \qquad \bar{A}_c^{\text{served}} = \frac{\sum_{s \in F_c} w_{ts} A_{c,s}}{p_c^{\text{feasible}}}.$$

This conditioning estimates completed public-transport journeys; it does not estimate latent unserved demand and does not move an infeasible desired time to an earlier, later, next-day, or supply-weighted departure.

For latent desired-departure demand, the alternative "preserve mass" policy uses the unconditioned response

$$\bar{A}_c^{\text{latent}} = \sum_{s \in F_c} w_{ts} A_{c,s},$$

so infeasible subintervals contribute an explicit zero response. Conditional route shares still sum to one, while a separate served-time fraction scales the response atoms. Feasible samples are never renormalized to replace the missing mass. The selected interpretation is part of artifact identity.

Adaptive diagnostics report evaluations, cache hits, accepted and refined intervals, depth, feasible and infeasible time shares, support changes, a heuristic relative response error, and unresolved probability mass. The last quantity is the weight stopped at a sample cap, depth limit, or minimum resolution while still unstable; it is not advertised as a rigorous error bound. Quadrature changes preprocessing and cache identity but not the cost of the subsequently compiled estimation objective.

The compatible pointwise implementation retains whole-period coverage and a deterministic priority queue. The recommended integral-response method instead compares the probability-weighted parent midpoint integral with the sum of two child-midpoint integrals. Point supports need not agree. Global refinement selects the interval with the largest absolute contribution and stops when

$$\sum_I e_I \leqslant \epsilon_a + \epsilon_r \|Q\|_1.$$

The integrated journey stage resolves active observations once and accumulates the sparse expected measurement response, separated by destination. Exact trip-service identity remains a stricter diagnostic mode. Requested and effective comparison modes are reported separately.

A bounded number of timetable departure and maximum-wait-window edges may seed the initial numerical partition so that a narrow feasible-service window is not silently missed. Every interval retains elapsed-duration probability. The timetable therefore informs quadrature locations but never supplies behavioral desired-departure weights. Numerical integration error, unresolved interval mass, and infeasible desired-time mass are distinct diagnostics.

Probability mass is canonicalized only at the aggregation boundary. Finite values within the configured tolerance of zero or one are projected to the exact boundary and the raw value, canonical value, and correction are retained in diagnostics. Material excursions remain errors, and the strict $0 < p_c^{\text{feasible}} \leqslant 1$ invariant for retained choice sets is not weakened.

The merger preserves the two probability layers $P(s, r) = w_{ts} P(r \mid s)$. Identical scheduled paths or response atoms receive the sum of their joint weights rather than new uniform shares. The desired-time period identifies the OD cell, whereas actual boarding and alighting events retain their timetable periods for observation matching. Consequently multiple sample-specific searches still yield one response column per retained OD-period cell.

The canonical model contract classifies each cell independently as free, fixed-zero, or fixed-positive. Timetable feasibility never promotes a fixed cell into the free parameterization. A feasible fixed-zero cell is retained only in diagnostics and contributes neither a response column nor an offset. A fixed-positive cell uses the same sampled averaging, remains outside the free columns, and contributes its response multiplied by its fixed flow to the measurement offset; absence of feasible sampled weight is a blocking error. Only retained free cells define the origin-period groups requiring production inputs. Searches use the exact declared sparse origin-period support rather than an implicit Cartesian product of all origins and periods.

Zero feasible weight freezes a cell at zero. By default, positive feasible weight below 0.5 excludes the cell from the initial model, weight from 0.5 to below 0.9 retains it with a strong warning, and weight of at least 0.9 retains it normally. These configurable thresholds are operational safeguards, not universal behavioral constants. Sampling levels 1, 3, 6, 12, and higher allow pre-estimation convergence analysis; one sample is a low-fidelity diagnostic, not the recommended representation for long periods with vehicle-specific counts.

6.3 Timetable Feasibility and Journey Events

For origin o, desired departure time t, and destination d, a bounded round-based timetable search returns feasible journey alternatives $j \in \mathcal{J}_{odt}$. A journey contains an ordered sequence of scheduled vehicle legs. If it uses L_j legs, it has $L_j - 1$ transfers and contributes a boarding and alighting event for every leg. Feasibility therefore depends on the timetable, access window, minimum transfer time, directed footpaths, and maximum transfer count.

Let C_j be the generalized journey cost and let $\theta > 0$ be the choice dispersion. Conditional journey shares are

$$p_{j|odt} = \frac{\exp(-C_j/\theta)}{\sum_{k \in \mathcal{J}_{odt}} \exp(-C_k/\theta)}.$$

Other declared pruning or dominance rules act before this normalization. A different OD-time cell may have a different alternative set and different shares even when its physical origin and destination are unchanged.

Here t is the latent desired-departure period, not the period of the first boarding. Every sampled query carries t explicitly. Alternatives that first board in different timetable periods are ranked, pruned, and normalized jointly within the same (o, d, t) choice set. Each boarding and alighting event retains its realized period, so demand in period t_0 may affect measurement rows in t_1. Period-semantics version 2 and sampling-integration schema 6 invalidate older sampled choices and downstream response artifacts.

For measurement m, let h_{jm} be the number or weighted contribution of events generated by journey j. The response of one passenger in cell $c = (o, d, t)$ is

$$B_{mc} = \sum_{j \in \mathcal{J}_c} p_{j|c} h_{jm}.$$

Hence B_{mc} is an expected event contribution, not an observed route or an individual simulated trajectory. Columns with identical event responses may be represented by one equivalence class without changing forward or adjoint products.

6.4 A Loop-Oriented Description of a RAPTOR Query

A RAPTOR query can be understood as an outer loop over the number of vehicle boardings, containing a timetable scan and a walking-relaxation loop. Its state consists of labels attached to physical stops rather than paths through a global time-expanded graph. A label records the earliest retained arrival at a stop in a given boarding round, together with the journey legs and the additive attributes needed later, such as waiting, walking, and in-vehicle time. The query proceeds as follows.

1. *Initialize the access round.* At the requested departure time, the origin receives its initial label and every other stop is initially unreachable. A loop follows the admissible footpaths out of each newly reached stop. Whenever walking produces an earlier label, that stop is placed back in the walking queue so that multi-link walking connections are closed. This is round zero because no vehicle has yet been boarded.

2. *Loop over boarding rounds.* If at most K transfers are permitted, the algorithm performs $\mathsf{K}+1$ transit rounds. Round $\mathtt{r}$ starts only from labels retained in round $\mathtt{r}-1$ and adds exactly one vehicle boarding. This makes the transfer bound part of the search state: a journey requiring another vehicle cannot leak into a round in which it is not allowed.

3. *Loop over scheduled trips and their stops.* For each trip, its stop times are scanned once in travel order. Before boarding, the scan compares the trip departure time at the current stop with the labels from the preceding round. If the passenger can be present in time, the trip becomes boardable. The scan then carries that boarded state forward along the same trip. At every subsequent stop it constructs a candidate arrival label, records the boarding and alighting events, and retains the better candidate according to the deterministic arrival and feature ordering. If a later stop offers a preferable feasible boarding state for the same trip, the carried state is replaced before the scan continues.

4. *Loop over walking connections again.* Once all trips have been scanned for the round, the walking-closure loop starts from the stops improved by transit. Walking changes arrival and walking time but consumes no additional boarding, so these labels remain in the same round. They become the possible boarding points for the next round.

5. *Collect and filter the results.* After the last permitted round, the algorithm loops over each destination and compares its labels across rounds. A label is discarded if another arrives no later with no more boardings. Waiting-time and total-journey-time limits are then applied. The surviving labels form the bounded timetable-feasible alternatives, and each retained state supplies its vehicle legs and boarding/alighting events. If no label survives, a separate topology-only check distinguishes a disconnected pair, a pair exceeding the transfer bound, and a pair for which no scheduled journey is feasible at the queried time.

Textbook RAPTOR commonly uses a further *marking* optimization: in the next round it scans only route patterns serving stops whose labels improved in the preceding round. The current implementation preserves the same bounded round semantics but uses the simpler deterministic variant of scanning every scheduled trip in every round. This distinction affects preprocessing time, not the meaning of the retained journey labels. In either variant, observed counts play no role in these loops. Counts enter only after the retained journeys have been converted into the fixed measurement-response matrix.

6.5 A RAPTOR Example

RAPTOR is a round-based timetable-routing method. A round counts vehicle boardings, not transfers: round zero contains access and walking labels, round one contains journeys using

one vehicle, and round two permits two vehicles and hence one transfer. The following small timetable illustrates the logic.

Consider four physical stops $\mathsf{A}, \mathsf{B}, \mathsf{C}, \mathsf{D}$ and three scheduled trips:

Trip	Line	Stop sequence with times	Role
r_1	Red	A 08:00, B 08:10, C 08:20	first leg
b_1	Blue	B 08:13, D 08:28	connection
g_1	Green	A 08:07, D 08:35	direct trip

Suppose the passenger is ready at A at 07:58 and that the three-minute connection at B satisfies the minimum-transfer rule. Initially, round zero labels A at 07:58. In round one, scanning trips boardable from A produces labels $\mathsf{B} = 08{:}10$ and $\mathsf{C} = 08{:}20$ through the Red line and $\mathsf{D} = 08{:}35$ through the direct Green line. In round two, the label at B makes the Blue trip boardable at 08:13; it improves the destination label to $\mathsf{D} = 08{:}28$. The resulting nondominated alternatives include

$$\mathsf{A} \xrightarrow{\mathsf{g}_1} \mathsf{D} \quad (0 \text{ transfers, } 08{:}35), \qquad \mathsf{A} \xrightarrow{\mathsf{r}_1} \mathsf{B} \xrightarrow{\mathsf{b}_1} \mathsf{D} \quad (1 \text{ transfer, } 08{:}28).$$

Neither alternative dominates the other in both arrival time and number of transfers. A zero-transfer limit retains only the direct trip; a one-transfer limit also admits the faster connection.

The example also shows why routing is time dependent. A passenger becoming ready at 08:05 misses r_1 but can still board g_1, so the connecting alternative disappears even though the physical OD pair is unchanged. A later query could encounter an entirely different set of trips. RAPTOR therefore returns timetable-feasible journey summaries for a specified origin and departure time; it does not estimate OD demand and it does not impose one all-day route share.

When this search is used to construct a reduced response, each retained alternative contributes boarding and alighting events for all of its vehicle legs. The connecting journey above contributes boardings at A and B and alightings at B and D, whereas the direct journey contributes only a boarding at A and an alighting at D. This distinction is what allows event-level counts, when sufficiently informative, to discriminate between journey alternatives.

6.6 Why the Reduced Approach Changes the Computational Scale

The large difference between detailed assignment and the reduced approach does not arise from replacing one shortest-path routine by a universally faster one. The two calculations represent the network differently, are invoked at different stages, and expose optimization problems of very different dimensions.

The central computational object is the assignment response, not RAPTOR itself. Let A map OD-time demand to flows on the links or events of the detailed assignment, and let M aggregate those flows into the modeled count rows. With fixed routing,

$$\boldsymbol{x} = A\mathbf{f}, \qquad \boldsymbol{\mu} = M\boldsymbol{x} = MA\mathbf{f} = B\mathbf{f}.$$

Thus the measurement-response matrix is $\mathsf{B} = \mathsf{MA}$. It can in principle be built from either the time-expanded graph or RAPTOR-derived journeys. Once B is available, estimation no longer needs to know how its columns were obtained.

This also means that a redesigned time-expanded workflow could obtain the same computational benefit. If it enumerated or summarized a bounded journey set, projected those journeys directly onto measurements, stored the resulting compact B, and then removed graph traversal from the optimizer, its estimation stage would be conceptually the same as the reduced workflow. The present difference is therefore a difference in response construction and representation, not a mathematical capability exclusive to RAPTOR.

The detailed model constructs a global time-expanded graph. Every scheduled arrival and departure is represented by an event node, and waiting, riding, transfer, access, and egress

possibilities are represented by links. A forward assignment propagates passenger flow over this graph. Estimation additionally requires derivatives of measurements with respect to demand or behavioral parameters. A direct differentiated construction may therefore retain state whose scale is governed not only by the number of network events, but also by the number of destination groups or measurement responses. Repeating these graph traversals and derivative calculations at many optimization iterations is what creates the hours-long runtime and the prohibitive memory requirement.

The earlier sharded algorithm already exploited the fact that fixed routing is linear, but it represented B only *implicitly*. For shard s, an objective evaluation computed

$$B_s f_s = M A_s f_s$$

by loading the corresponding routing state and propagating flows through the complete time-expanded graph. Its adjoint similarly applied $A_s^\top M^\top$ by a reverse graph traversal. Routing probabilities could be reused, but the measurement-response columns were not constructed and retained. Sharding limited the amount of live state and made the calculation possible in memory; it did not eliminate the expensive graph work from each operator product. This is why reducing a run to fewer shards reduced memory much more clearly than elapsed time.

For small cases, $B = MA$ can be constructed from the detailed graph. At full- network scale, however, a dense B has one row per measurement and one column per free OD-time cell and is far too large. Constructing it column by column would require an enormous number of unit-demand assignments, while a reverse construction can require event-level derivative state for a very large collection of measurements. Fixed routing guarantees linearity, but it does not by itself make explicit matrix construction affordable.

RAPTOR uses a different state representation. It scans timetable routes in a bounded sequence of rounds, with one additional vehicle boarding per round, and retains compact labels at stops rather than constructing a global graph of all timetable events. For one origin and departure time, its working state is principally determined by the stops, timetable records scanned, and permitted boarding rounds. It does not carry a measurement derivative for every event while searching for feasible journeys.

More importantly, the RAPTOR journey representation constructs the relevant entries of B directly. If journey j for OD-time cell c has fixed share $p_{j|c}$ and contributes h_{jm} events to measurement m, then

$$B_{mc} = \sum_{j \in \mathcal{J}_c} p_{j|c} h_{jm}.$$

Unobserved time-expanded links never appear in this construction. Each retained journey touches only a small number of boarding and alighting rows, so the response can be stored sparsely and identical columns can be compressed. This is the practical difference: the graph-based shards repeatedly *apply* an implicit B, whereas the reduced approach builds a compact measurement-level B once and reuses it.

The two constructions produce the same matrix only when they use equivalent OD-time definitions, physical-stop mappings, access and transfer rules, feasible journey sets, generalized costs, route-choice probabilities, and measurement aggregation. Bounded transfer depth, journey pruning, or different share rules may instead give

$$B_{\text{RAPTOR}} \neq B_{\text{graph}}.$$

The reduced matrix is exact relative to its retained journeys and fixed shares, but its agreement with the detailed response is an empirical validation question. Counts are used to estimate demand after either matrix is fixed; they do not choose the RAPTOR journeys.

The decisive saving comes from using this search as preprocessing rather than inside every objective evaluation. Feasible journeys and their attributes are computed once, converted into fixed expected measurement responses, and then reused. Estimation operates on those

responses. With the conditional gravity model, it also optimizes a small collection of shared behavioral and production coefficients instead of one variable per feasible OD-time cell. The resulting workflow is therefore

$$\begin{array}{c}\text{timetable search once} \longrightarrow \text{compact fixed response} \\ \Downarrow \\ \text{many inexpensive objective evaluations}\end{array}$$

whereas repeated detailed estimation places the expensive network calculation inside the optimization loop.

Three changes are consequently responsible for laptop-scale estimation:

1. explicit construction and reuse of a compact measurement-response matrix, rather than repeated application of an implicit graph-based operator;
2. timetable-native, bounded-round routing that makes this compact response practical to construct; and
3. a low-dimensional gravity parameterization instead of an unrestricted OD-time parameter vector.

These effects must not be attributed to RAPTOR alone. Searching every origin, departure instant, and destination can still require substantial preprocessing, and storing uncompressed journey or measurement responses can still be large. The approach remains practical because routing is bounded, preprocessing is reused, responses are sparse or compressed, and detailed assignment is reserved for post-fit validation.

7 Additive Operator Decomposition

The detailed and reduced models both become additive in demand once their routing responses are fixed. This section isolates that common mathematical structure. It explains why OD columns can be partitioned or compressed without cutting journeys through the network, and why the forward and adjoint products remain exact even when the full response matrix is never written explicitly. In particular, sharding an implicit response controls construction and working memory, but does not provide the same runtime benefit as constructing and reusing compact measurement-response columns.

7.1 Mathematical Sharding Identity

Let the free OD-time cells be partitioned into disjoint index sets $\mathcal{C}_1, \ldots, \mathcal{C}_S$. Writing P_s for the injection of the coordinates in shard s into the complete free vector gives

$$\mathbf{f} = \sum_{s=1}^{S} P_s \mathbf{f}_s, \qquad B\mathbf{f} = \sum_{s=1}^{S} B_s \mathbf{f}_s, \qquad B_s = BP_s.$$

Thus the complete expected measurement vector is

$$\boldsymbol{\mu} = \rho\left(\sum_{s=1}^{S} B_s \mathbf{f}_s + \mathbf{b}^{\text{fix}}\right). \tag{2}$$

The partition is a decomposition of demand columns, not a cut of the transit network. Every B_s is defined using complete feasible journeys and may therefore contain paths through any stop, line, or geographical region. The identity is exact provided that the shards are disjoint,

cover every free cell exactly once, and use the same routing and measurement assumptions. Frozen demand appears once in $\mathbf{b}^{\text{fix}}$ and must not be duplicated among shards.

The adjoint decomposes compatibly. For any measurement-space vector $\boldsymbol{\nu}$,

$$B^{\mathsf{T}}\boldsymbol{\nu} = \begin{bmatrix} B_1^{\mathsf{T}}\boldsymbol{\nu} \\ \vdots \\ B_S^{\mathsf{T}}\boldsymbol{\nu} \end{bmatrix},$$

up to the declared ordering of the free coordinates. Forward contributions may therefore be accumulated independently, and the derivative with respect to each shard uses its corresponding transpose product.

This additive structure does *not* generally make the statistical objective a sum of independent shard objectives. For Poisson or negative- binomial observations, the likelihood is evaluated after the shard predictions have been summed in Equation (2). Shards are mathematically independent only at the linear-response stage; their parameters remain coupled through shared measurement means and through any nonseparable prior or demand model.

7.2 Exact Reduced and Fixed-Routing Operators

For free demand $\mathbf{f}_{\mathcal{U}}$ and positive frozen demand $\mathbf{f}_{\mathcal{F}}$, the fixed-routing prediction is

$$\boldsymbol{\mu} = \rho\left(A_{\mathcal{U}}\mathbf{f}_{\mathcal{U}} + A_{\mathcal{F}}\mathbf{f}_{\mathcal{F}}\right) = \rho\left(A_{\mathcal{U}}\mathbf{f}_{\mathcal{U}} + \mathbf{b}^{\text{fix}}\right).$$

Frozen-zero cells have neither columns nor offset contributions. The matrix $A_{\mathcal{U}}$ need not be materialized: an operator is mathematically defined by the products $\mathbf{f} \mapsto A_{\mathcal{U}}\mathbf{f}$ and $\boldsymbol{\nu} \mapsto A_{\mathcal{U}}^{\mathsf{T}}\boldsymbol{\nu}$ satisfying

$$\boldsymbol{\nu}^{\mathsf{T}}(A_{\mathcal{U}}\mathbf{f}) = \mathbf{f}^{\mathsf{T}}(A_{\mathcal{U}}^{\mathsf{T}}\boldsymbol{\nu}).$$

If response columns fall into exact equivalence classes $e(c)$, with one representative column in $\bar{B}$, then

$$B\mathbf{f} = \bar{B}\bar{\mathbf{f}}, \qquad \bar{f}_e = \sum_{c:e(c)=e} f_c.$$

This compression changes neither the external demand coordinates nor their gradients. For a linear reduced parameterization $\mathbf{f} = \Phi\boldsymbol{z}$, the direct parameter response is $H = B\Phi$, and its adjoint is $H^{\mathsf{T}} = \Phi^{\mathsf{T}}B^{\mathsf{T}}$.

OD blocks without network cuts. For problems too large for a monolithic solve, the package partitions only the free OD columns. The default deterministic order is destination group, time bin, and then free-column index, with deterministic subdivision of oversized groups and optional merging of small compatible groups. Every block uses the complete time-expanded network; partitioning never removes links or prevents a path from traversing another part of the service. User-defined partitions are accepted only if every free column occurs exactly once and no frozen column is present. Block membership and the complete partition are fingerprinted.

Let B be the active block, A_B its independently constructible operator, and $\widehat{\mathbf{y}}$ the current complete prediction. A trial changes only $\mathbf{f}_B$ and is evaluated incrementally as

$$\widehat{\mathbf{y}}_{\text{trial}} = \widehat{\mathbf{y}} + A_B(\mathbf{f}_B^{\text{trial}} - \mathbf{f}_B).$$

The remaining OD variables are held fixed. An update is accepted only if it is finite, respects the bounds, and does not increase the complete objective beyond the numerical acceptance tolerance. Damping and backtracking are applied when needed. Rejected updates leave the complete state unchanged. Separable quadratic priors are evaluated conditionally; unsupported cross-block prior or constraint coupling is rejected rather than ignored. Exact full products are performed at configurable validation boundaries to detect incremental drift, not after every block.

8 Measurement Model and Likelihood

Let Y denote observed data (e.g., passenger counts on selected links, or aggregates derived from them). Operational measurements are available only for a subset of quantities. Let $\mathcal{M}$ denote the index set of available measurements. For each $m \in \mathcal{M}$ we denote by Y_m the observed count.

Assignment produces expected passenger events; estimation requires a probability model for the operational records. This section completes the forward chain by aggregating assigned events into measurement means, accounting for detection when applicable, and defining the count likelihood. Its assumptions concern observation and noise, not route choice or demand generation.

8.1 Statistical Scope and Assumptions

The observation model is conditional on the forward response and makes three distinct assumptions: the mapping from assigned events to measurement rows is known; the detection factor is fixed unless explicitly included in the parameterization; and observations are conditionally distributed according to the selected count likelihood. Absence of a measurement is not an observed zero. Conditional independence in the likelihood is a statistical modeling assumption and does not imply that passenger movements or the corresponding expected counts are physically unrelated.

Predicted measurement from the assignment model. The assignment model produces link flows $\mathbf{x}(\mathbf{f};\theta) = \{x_\ell(\mathbf{f};\theta) : \ell \in \mathcal{A}\}$. A deterministic mapping associates each measurement m with a subset of links $\mathcal{A}_m \subseteq \mathcal{A}$, so that the model-predicted quantity is the sum of the corresponding link flows:

$$\lambda_m(\mathbf{f};\theta) = \sum_{\ell \in \mathcal{A}_m} x_\ell(\mathbf{f};\theta).$$

In the current version of the software, aggregation weights are implicitly equal to one.

Asymmetric counting device (undercounting). The counting device is asymmetric: it is more likely to *miss* passengers than to count passengers that are not present. We model this by introducing a detection rate $\rho \in (0,1]$ such that the expected observed count is

$$\mu_m(\mathbf{f};\theta,\rho) = \rho\,\lambda_m(\mathbf{f};\theta).$$

Negative binomial likelihood. Conditionally on $(\mathbf{f},\theta,\rho,r)$, the observations are assumed independent and

$$Y_m \mid \mathbf{f},\theta,\rho,r \;\sim\; \mathrm{NegBin}\big(r,\; p_m(\mathbf{f};\theta,\rho,r)\big), \qquad m \in \mathcal{M},$$

where $r > 0$ is a dispersion parameter and the success probability p_m is defined so that $\mathbb{E}[Y_m] = \mu_m(\mathbf{f};\theta,\rho)$:

$$p_m(\mathbf{f};\theta,\rho,r) = \frac{r}{r+\mu_m(\mathbf{f};\theta,\rho)} = \frac{r}{r+\rho\,\lambda_m(\mathbf{f};\theta)}.$$

Hence, for $y \in \mathbb{N}_0$,

$$\Pr(Y_m = y \mid \mathbf{f},\theta,\rho,r) = \frac{\Gamma(y+r)}{\Gamma(r)\,y!}\,(1-p_m(\mathbf{f};\theta,\rho,r))^y\,(p_m(\mathbf{f};\theta,\rho,r))^r\,.$$

Fixed measurement parameters. In the current version of the software, the detection rate ρ and the dispersion parameter r are treated as fixed, user-specified hyperparameters and are not estimated within the Bayesian inference procedure. Only the OD demand vector and optionally the dispersion parameter θ are inferred. Joint estimation of (ρ, r) is a possible future extension.

Likelihood and normalization. The likelihood is the product over observed measurements:

$$\mathcal{L}\big(\{Y_m\}_{m\in\mathcal{M}} \mid f, \theta, \rho, r\big) = \prod_{m\in\mathcal{M}} \Pr\big(Y_m = y_m^{\mathrm{obs}} \mid f, \theta, \rho, r\big)\,.$$

The corresponding log-likelihood is

$$\ell(f, \theta, \rho, r) = \sum_{m\in\mathcal{M}} \Big[\log\Gamma\big(y_m^{\mathrm{obs}} + r\big) - \log\Gamma(r) - \log\big(y_m^{\mathrm{obs}}!\big) + r\log p_m(f; \theta, \rho, r) + y_m^{\mathrm{obs}} \log(1 - p_m(f; \theta, \rho, r))\Big].$$

For posterior computations, terms constant with respect to (f, θ) may be dropped.

9 Planned Reusable Temporal Assignment-Map Architecture

This section is a design specification and implementation roadmap. It extracts a stable computational architecture from the forward models developed earlier in this part, especially the scheduled time-expanded formulation, the reduced journey-response formulation, the additive fixed-routing decomposition, and the measurement model. It deliberately distinguishes verified public-library functionality from proposed work. In particular, it does not specify an alternative supply model: the first target backend remains scheduled, time-expanded assignment.

The following labels are used throughout this section.

IMPLEMENTED The public library contains a concrete, tested implementation of the stated behavior for the current canonical direct-scheduled workflow. Broader generalization is stated explicitly where it remains incomplete.

PARTIALLY IMPLEMENTED Relevant tested foundations exist, but the complete contract or end-to-end workflow specified here does not.

EXPERIMENTAL A callable prototype exists, but its interface or operational guarantees are not yet a stable general contract.

UNIMPLEMENTED The component is a proposed design and is not currently supplied by the public library.

When a component spans several existing modules, the label refers to the complete component described here, not to every useful foundation beneath it.

9.1 Layered forward decomposition

Let $\theta \in \mathbb{R}^p$ be a low-dimensional vector of demand-model parameters. A demand generator g produces the complete physical OD-time demand vector $x \in \mathbb{R}^n$ in a prescribed canonical order. A preprocessed assignment map $A \in \mathbb{R}^{m\times n}$ maps that demand to predicted boarding and alighting measurements $\hat{y} \in \mathbb{R}^m$:

$$\theta \overset{g}{\longmapsto} x \overset{A}{\longmapsto} \hat{y}, \qquad \hat{y} = A\, g(\theta). \tag{3}$$

Assignment, demand, observation, and estimation are independent layers. The assignment layer defines how passenger demand creates expected measurement events. The demand layer defines g. The observation layer compares $\hat{y}$ with recorded counts. The estimation layer chooses an algorithm for varying θ. This separation makes the expensive supply-dependent calculation reusable across model specifications.

Implementation status. IMPLEMENTED for the current direct-scheduled full-network workflow. The public library composes gravity demand generation, measurement operators, likelihoods, and optimizers in this form. The content-addressed full-network temporal assignment artifact has been constructed, persisted, loaded in a fresh process, and benchmarked through the same forward/adjoint contract. Generalization to every possible demand, observation, and assignment backend remains a separate roadmap.

9.2 Strict assignment-map contract

The external numerical responsibility of an assignment component is limited to the two products

$$x \longmapsto Ax, \qquad r \longmapsto A^{\mathsf{T}} r, \tag{4}$$

where $r \in \mathbb{R}^m$ is a measurement-space vector. Neither product requires a dense realization of A.

An assignment artifact may depend on network topology, the scheduled timetable, temporal discretization, fixed route-choice parameters, fixed departure-time-choice parameters, transfer, waiting, and journey-duration constraints, measurement definitions, and coefficient-pruning tolerances. These inputs determine the physical response represented by A.

It must not depend on a gravity-model specification, gravity parameters, priors, the likelihood, the optimizer, or estimated OD values. Consequently, changing a demand model, prior, likelihood, or optimizer must not trigger assignment preprocessing. Changing the timetable, temporal intervals, route-choice parameters, departure-choice parameters, or feasibility constraints must invalidate the artifact. Measurement-definition changes must likewise invalidate the corresponding measurement map.

Implementation status. IMPLEMENTED for the direct-scheduled artifact path. Tested linear-operator and gravity-measurement protocols expose forward and adjoint products, shapes, fingerprints, and operational capabilities. The persisted artifact identity separates physical indexing, supply, timetable, temporal, choice, feasibility, measurement, coefficient-policy, numerical, and schema dependencies, with explicit mismatch diagnostics. Existing fixed-routing operators validate against this identity through an adapter. The broader dependency boundary is not yet shared by every assignment backend and demand model.

9.3 Assignment-backend abstraction

Downstream code should depend on a stable abstract interface rather than on the internal time-expanded representation. A minimal conceptual contract is:

```
class AssignmentOperator:
    number_of_demand_cells: int
    number_of_measurements: int

    def matvec(self, demand):
        """Calculate predicted measurements A @ demand."""

    def rmatvec(self, residual):
        """Calculate the adjoint product A.T @ residual."""
```

Production implementations additionally need a numerical type, immutable identity metadata, and validated dimensions. Those additions must not weaken the two-operation numerical contract in Equation (4).

Other assignment backends could later implement the same canonical indexing and forward/adjoint contract. Such a substitution must not require changes to demand models, observation models, likelihoods, or optimizers.

Implementation status. IMPLEMENTED. The public library provides the runtime-checkable `AssignmentOperator` protocol with canonical dimensions, index and artifact fingerprints, and the required `matvec` and `rmatvec` products. A validated adapter exposes existing explicit, matrix-free, and sharded fixed-routing operators through this contract without altering their numerical products. The authoritative scheduled time-expanded calculation is also available through the same contract as a deliberately unoptimized reference backend.

9.4 Canonical indexing and compatibility

Three stable canonical tables define the spaces on which A operates:

1. a physical OD–departure-interval cell table, with one immutable row for every demand coordinate;
2. a boarding and alighting measurement table, with one immutable row for every predicted count; and
3. a temporal-interval table giving ordered interval identifiers and exact boundaries.

Every demand model must return values in the assignment artifact's canonical OD-time order. Display labels are not identities: canonical rows must retain stable physical identifiers and interval identifiers. Compatibility checks must cover dimensions, schemas, and fingerprints and must fail explicitly, rather than silently reordering, truncating, or accepting incompatible inputs.

Implementation status. IMPLEMENTED for the direct-scheduled artifact path. Existing reduced-OD and fixed-routing paths use immutable layouts, explicit dimensions, fingerprints, schemas, and fail-closed cache validation. Immutable canonical interval, physical OD-time, and boarding/alighting measurement tables are now populated by the direct-scheduled builder and persisted in the artifact manifest, including separate physical-artifact and free/fixed-binding fingerprints. They are not yet populated and consumed by every assignment backend and every demand-model workflow.

9.5 Non-uniform temporal discretization

The proposed temporal partition consists of ordered, non-overlapping intervals

$$\mathrm{I}_1, \mathrm{I}_2, \ldots, \mathrm{I}_{\mathsf{K}}. \tag{5}$$

Intervals should be finer when supply, demand, route availability, or transfer conditions change rapidly, and broader during stable conditions. Selecting the partition is therefore a model-design problem, not merely a global choice of interval length. A candidate partition should balance

$$\text{temporal approximation loss} + \lambda\, \text{assignment complexity}, \tag{6}$$

where $\lambda \geqslant 0$ expresses the desired precision–resource compromise.

Candidate diagnostics include service frequencies and headways, active lines and route patterns, running times, transfer opportunities, boarding and alighting count profiles, approximate assignment support, and projected nonzeros, memory, and execution time. Rather than impose one arbitrary partition, preprocessing should eventually report a precision–complexity Pareto frontier from which a documented choice can be made.

Implementation status. UNIMPLEMENTED. The library accepts temporal layouts in existing pipelines, but it does not generally select a data-driven non-uniform partition or construct the Pareto frontier described by Equation (6).

9.6 Scheduled time-expanded preprocessing

The scheduled time-expanded graph described earlier in this part remains the authoritative routing representation during preprocessing. For each physical OD pair and departure interval s, preprocessing should:

1. construct feasible joint departure-time and route alternatives;
2. apply fixed exogenous departure-time-choice and route-choice parameters;
3. calculate the alternative probabilities;
4. project boarding and alighting contributions onto measurement interval t; and
5. consolidate those contributions into a sparse temporal block $A_{t,s}$.

Departure-time and route choice are thus combined inside scheduled time-expanded preprocessing. If x_s denotes the demand coordinates in departure interval s and y_t the predicted measurements in interval t, the resulting operator satisfies

$$y_t = \sum_s A_{t,s} x_s. \tag{7}$$

The time-expanded graph is not traversed during estimation; repeated objective and gradient evaluations use only the persisted assignment response.

Implementation status: scheduled routing foundation. IMPLEMENTED. The public library contains scheduled time-expanded network construction and assignment foundations, including journey and measurement-response preprocessing. A canonical scheduled reference operator now evaluates the complete time-expanded forward calculation and obtains its exact adjoint by reverse-mode differentiation. It validates canonical and supply-dependent identities and is intended for small equivalence tests rather than repeated production estimation.

Implementation status: fixed-routing linear operations. IMPLEMENTED. Dense, sparse, matrix-free, sharded, and selected-block fixed-routing paths provide tested forward products; the relevant estimation operators also provide adjoint products and bounded operational controls.

Implementation status: complete temporal-block conversion. IMPLEMENTED for the current canonical direct-scheduled path. The library can derive an exact, independently addressable sparse family $\{A_{t,s}\}$ column by column from the authoritative scheduled reference operator, with progress, ETA, deadline handling, duplicate consolidation, and coefficient-mass diagnostics. A faster constructor batches unit responses in one JIT-compiled fixed-shape kernel, pads the final chunk, and aggregates measurements on device before transferring bounded response chunks. It compiles once and never materializes the complete dense map. The production constructor now goes further: it prepares fixed scheduled routing probabilities once, propagates only measurement-supported edges, emits sparse measurement contributions into deterministic shards, checkpoints them atomically, resumes after interruption, selectively rebuilds corrupt shards, and then finalizes canonical temporal blocks. A general explicit joint departure-time-choice layer beyond the current canonical departure intervals remains unfinished. The resulting block operator is exposed through the established gravity measurement-operator protocol, including forward, adjoint, and batched products, so gravity objectives require no representation-specific changes.

9.7 Sparse temporal block structure

The intended block map is sparse for three distinct reasons. First, *causality* prevents demand from affecting earlier measurements. Second, *temporal banding* follows from a maximum feasible journey duration, which limits the active range of $\mathsf{t} - \mathsf{s}$. Third, *spatial sparsity* arises because one OD journey visits only a small subset of all measured locations and event types.

Construction of each block should merge duplicate row–column contributions, remove exact zeros, and optionally prune negligible probabilities using an explicit, auditable tolerance. Pruning must retain diagnostics for removed probability mass and its distribution. The representation should use compact integer indices, persist temporal blocks independently, and support efficient forward and adjoint access. Compressed sparse row (CSR) storage is natural for Ax; compressed sparse column (CSC), or equivalently a persisted transposed CSR representation, avoids repeated conversion for $\mathsf{A}^\mathsf{T}\mathsf{r}$.

Implementation status. IMPLEMENTED for the persisted temporal-block path. Exact in-memory temporal blocks now merge duplicates, remove zeros, retain coefficient-mass diagnostics, and provide JAX-compatible forward and adjoint products. A versioned manifest persists blocks independently and validates their content hashes before reuse. A CPU backend constructs and retains CSR for forward products and CSC for adjoint products; its custom JAX reverse rule returns the CSC adjoint without materializing a dense matrix. Direct alternative-level construction and the complete probability-pruning workflow remain unfinished.

9.8 Block-support complexity profiling

Before expensive construction, a bounded profiler should estimate active block support, projected nonzeros, persistent storage, peak construction memory, and forward/adjoint execution cost. Its estimates should be reported by temporal block and in aggregate so that an infeasible partition can be rejected before full preprocessing. The profiler is also the computational-complexity input to the Pareto analysis in Equation (6).

Implementation status. PARTIALLY IMPLEMENTED. Fixed-routing block-coordinate workflows provide bounded, resumable support preflight, resource limits, retained-state limits, progress, and checkpoint validation. A canonical temporal profiler now uses causality and a maximum journey duration to report feasible blocks, candidate entries, excluded entries, and conservative storage. It does not yet estimate spatial support, peak construction memory, or execution time from the scheduled graph, nor compare candidate temporal partitions.

9.9 Independent demand-model interface

The demand layer should depend only on its parameters, explanatory data, and the canonical demand-cell table. A conceptual differentiable interface is:

```
class DemandModel:
    parameter_names: tuple[str, ...]

    def demand(self, parameters):
        """Generate demand in canonical OD-time order."""

    def transpose_jacobian_product(
        self, parameters, demand_gradient
    ):
        """Calculate J_g(parameters).T @ demand_gradient."""
```

Possible implementations include minimal global gravity, time-specific gravity, origin-group production, destination-group attraction, low-rank OD interactions, gravity plus residual corrections, directly parameterized OD demand, and externally supplied demand. Switching between these models must reuse the same compatible assignment artifact.

Implementation status. PARTIALLY IMPLEMENTED. The generic reduced-demand code supplies composable production, attraction, impedance, and low-rank interaction specifications; canonical demand generation is differentiable and several progressive gravity specifications are reusable with the same response operator. Existing fixed, directly supplied, and reduced parameterizations provide additional foundations. There is not yet one stable public `DemandModel` protocol covering every listed family, explicit canonical compatibility, and a named transpose-Jacobian product.

9.10 Independent observation model and differentiation

The observation layer receives observed counts and Ax, not routing structures. It should select independently among Poisson, negative-binomial, possible zero-inflated formulations, and future count likelihoods. With

$$x = g(\theta), \qquad \hat{y} = Ax, \tag{8}$$

the chain rule gives

$$\nabla_\theta L = J_g(\theta)^\top A^\top \nabla_{\hat{y}} L. \tag{9}$$

Thus neither the dense assignment matrix nor the full gravity Jacobian needs to be materialized. The observation component controls the distributional assumption, dispersion, detection terms when modeled, zero-inflation design, and calibration mask, while remaining independent of assignment internals.

Implementation status. PARTIALLY IMPLEMENTED. The generic reduced-demand path implements Poisson, negative-binomial, zero-inflated Poisson, and zero-inflated negative-binomial contributions independently of routing, and automatic differentiation composes them with operator products. A stable general observation-model protocol shared by every estimation path is not yet defined.

9.11 Reusable assignment artifact

The conceptual persisted layout is:

```
assignment_map/
|-- manifest.json
|-- fixed_measurement_offset.npy
‘-- blocks/
    ‘-- block-*.npz
```

The current artifact embeds the canonical OD-time, measurement, and temporal interval tables in the manifest under `canonical_index`. The manifest also records a schema version, canonical-index fingerprints, network and timetable fingerprints, temporal discretization, choice-parameter fingerprints, feasibility constraints, coefficient precision and threshold, dimensions, total nonzeros, block dimensions, content hashes, and construction diagnostics. It does not contain a gravity specification, prior, likelihood, or optimizer configuration. Run-level progress, checkpoint, and benchmark records are maintained alongside the artifact rather than as required payload subdirectories. Payloads are published atomically and validated before reuse; incompatible or incomplete artifacts fail closed.

Implementation status. IMPLEMENTED for the current direct-scheduled artifact. The library has a content-addressed, versioned temporal-block artifact with an atomic publication step, canonical tables, complete identity metadata, per-block content hashes, fixed-offset validation, and fail-closed fresh-process loading. Its direct construction workspace contains deterministic validated shards and a checkpointed manifest; completed shards survive interruption and invalid shards are rebuilt selectively before atomic final publication. The full-network artifact was loaded in a fresh process and used for a measured forward product; diagnostic, progress, and benchmark records remain run-level products rather than required contents of the immutable assignment artifact.

Production activation is policy controlled. Explicit `off` leaves the existing loader unchanged, while explicit `direct` constructs or resumes the block artifact. In `auto` mode, a valid persistent artifact is reused before preparing routing; otherwise construction occurs only when the expected evaluation savings strictly exceed its estimated construction cost. Invalid final artifacts are rejected, moved to a uniquely named quarantine directory for diagnosis, and rebuilt through the atomic publication path.

9.12 Validation requirements

The scheduled temporal assignment map must pass the following validation sequence before it can replace the authoritative preprocessing calculation in estimation:

1. compare scheduled sparse blocks with the authoritative time-expanded assignment;
2. test forward equivalence on several deterministic random vectors;
3. verify the adjoint identity
$$\langle Ax, r\rangle = \langle x, A^{\mathsf{T}} r\rangle; \tag{10}$$
4. measure nonzeros, storage, peak memory, and forward/adjoint execution time;
5. report approximation errors by time interval, transfer count, and journey class;
6. track affected observed count mass, not only matrix norms;
7. verify artifact invalidation and canonical-index compatibility; and
8. verify that changing only the demand model reuses the same assignment artifact.

Tolerance choices, random seeds, coefficient thresholds, and reference fingerprints belong in the validation record. Exact equivalence tests should be separated from explicitly approximate pruning tests.

Implementation status. PARTIALLY IMPLEMENTED. Existing operator tests cover forward equivalence, adjoint products, cache incompatibility, fingerprints, numerical agreement, frozen-demand offsets, construction progress, interruption and resume, finalized-artifact reuse, selective shard repair, and selected performance diagnostics. Production-policy tests additionally cover explicit disablement, economically unjustified automatic decline without routing work, fresh-process reuse, gravity-protocol matrix products, and safe quarantine and rebuilding of an incompatible final artifact. Stratified approximation and affected-count-mass reporting remain to be implemented for this representation.

The executable public validation `docs/source/examples/direct_scheduled_gravity_validation.py` applies this interface end to end to `simple_example_02`. It constructs the canonical demand and observation intervals, reports shard progress, evaluates the same negative-binomial gravity objective and adjoint gradient through both the established BCOO operator and the temporal-block operator, and supports a mandatory fresh-process cache-reuse check. On the

development laptop, the first validation constructed 1,201 nonzeros in 0.31 seconds (0.48 seconds for complete activation) and stored 15,492 bytes. The objective absolute difference was $3.05\,10^{-5}$ and the largest gradient absolute difference was $1.91\,10^{-6}$. A second Python process loaded the persistent artifact in 0.011 seconds, performed no construction, and reproduced the same numerical differences. The test suite executes this two-process protocol automatically.

Construction now uses a single monotonic deadline contract shared by activation, routing guards, support and planning guards, shard validation and construction, temporal assembly, and atomic persistence. A normal bounded stop returns a structured terminal record rather than a partial operator or an undifferentiated error. The record identifies the phase, elapsed and remaining time, completed work, next resumable unit, checkpoint and artifact locations, predicted next cost, checkpoint reusability, and any overshoot caused by an indivisible operation. Versioned progress events use the same phase vocabulary and remain independent of presentation libraries.

Shard construction stops only after atomically committing and validating the current shard, and a later invocation reuses it. Fixed-shape destination-group routing batches, destination-group origin-support fragments, aggregate materialized support, the deterministic plan, and one temporal fragment per numerical source shard are also content validated and persisted. Thus resume bypasses the routing factory, support discovery, and planning when their checkpoints are compatible, and temporal assembly restarts at its first missing fragment. Final persistence checks the deadline between blocks and removes an unpublished staging directory on termination. Valid completed artifacts retain cache-first priority even under an almost-expired new budget. Routing is compiled once for a bounded padded batch shape; each batch is synchronized, transferred, content validated, and atomically persisted before the next dispatch. Downstream support and numerical construction load at most one routing batch and never concatenate the global destination-by-link arrays. JAX compilation and one batch execution remain indivisible internally: they are guarded before launch and record overshoot. Origin-support discovery checks the deadline between bounded origin chunks and atomically commits every completed destination group, so resume continues at the first missing group. Numerical measurement-shard construction may now use an explicitly bounded thread pool. All workers share the one compiled executable, but each owns its own routing-batch reader. Per-worker workspace preflight and a resident-shard ceiling bound admission. Workers write validated payloads only to isolated staging locations; the parent publishes them and advances the manifest in canonical plan order, even when workers finish out of order. Worker count is excluded from scientific provenance, interrupted parallel and serial runs therefore reuse the same cache, and failures remove unpublished staging files without invalidating previously committed shards. Origin-specific support materialization has a separate positive configurable entry ceiling, whose default is 125 million. This ceiling is an operational memory guard rather than scientific provenance, so increasing it preserves compatible destination-group support checkpoints. Summary-only analysis can avoid materialization, but production numerical construction cannot. Numerical-shard preflight diagnostics likewise retain actual and permitted values for shard count, manifest size, filesystem work, sparse products, construction dispatches, and worker memory. Unsafe plans raise a structured subclass of `MemoryError`. Shard-sizing controls remain provenance because they alter persisted identities; purely operational ceilings are stored in the plan and re-evaluated at reuse time, so an evidence-based ceiling change does not discard compatible support checkpoints. Only the current CPU chunk or destination group may be repeated. Therefore the deadline hardening is sufficient for bounded public full-network probes.

9.13 Implementation roadmap

Table 1 records the audited status of the complete components, using the conservative convention at the start of this section. Priorities order the remaining architectural work; a dash

indicates an existing foundation rather than a new implementation priority.

Table 1: Roadmap for the reusable temporal assignment-map architecture.

Component	**Status**	**Priority**
Scheduled time-expanded assignment foundation	IMPLEMENTED	–
Scheduled time-expanded reference operator	IMPLEMENTED	–
Fixed-routing linear operations	IMPLEMENTED	–
Stable assignment-operator abstraction	IMPLEMENTED	–
Canonical physical OD-time index	IMPLEMENTED	–
Data-driven non-uniform partition selection	UNIMPLEMENTED	1
Block-support complexity profiler	PARTIALLY IMPLEMENTED	1
Scheduled $A_{t,s}$ construction	IMPLEMENTED	–
Sparse temporal-block persistence	IMPLEMENTED	–
Optimized temporal CSR/CSC forward and adjoint	IMPLEMENTED	–
Independent demand-model contract	PARTIALLY IMPLEMENTED	2
Interchangeable gravity specifications	IMPLEMENTED	–
Independent observation models	PARTIALLY IMPLEMENTED	3
Assignment-backend substitution test	UNIMPLEMENTED	4

Implementation status. IMPLEMENTED for the current canonical direct-scheduled workflow. The full-network path now includes the canonical index, fixed scheduled routing, temporal sparse-block construction, content-addressed persistence, atomic resumption, and the tested CSR/CSC forward and adjoint backend. The remaining priorities are generalizations: data-driven partition selection, broader demand and observation protocols, backend substitution, direct alternative-level construction, and a more general joint departure-time-choice layer.

9.14 Architectural invariants

The following requirements are non-negotiable for implementation of this roadmap:

1. *Assignment preprocessing is independent of the gravity specification.*
2. *Demand models produce vectors in canonical OD-time order.*
3. *Assignment operators provide forward and adjoint operations.*
4. *Estimation never traverses the time-expanded graph.*
5. *Changing only the demand model does not rebuild the assignment artifact.*
6. *Approximation and pruning are accompanied by quantitative diagnostics.*
7. *Long preprocessing operations are resumable and report progress.*
8. *Downstream estimation code depends on the assignment contract, not on the scheduled time-expanded implementation.*

The eighth invariant is the sole future-proofing requirement for possible alternative assignment representations; no additional supply methodology is specified here.

For public preprocessing, progress is a machine-readable contract rather than a presentation-layer convention. Physical-stop, route-pattern, service-period and timetable indexing, bounded RAPTOR rounds and destination scans, journey-choice construction, measurement-response aggregation, origin-support discovery, structural-zero classification, and artifact publication report completed and total units, elapsed time, observed throughput, estimated remaining time, confidence, and the current unit. Events occur at phase boundaries, approximately every

one percent, or at the first unit boundary after ten seconds. The ETA is explicitly unavailable before the first measured unit. A library or backend call that cannot expose internal work remains an identified indivisible unit and is not assigned a fictitious completion estimate.

Implementation status. IMPLEMENTED for the current direct-scheduled full-network workflow. The current artifact path collectively enforces the canonical contract, index tables, forward/adjoint products, demand-model separation, progress reporting, resumability, and fail-closed artifact reuse. Approximation-specific affected-count-mass reporting and substitution across independent assignment backends, demand protocols, and observation protocols remain future work.

Part III
Demand Models and Their Assumptions

10 Cell-Level OD Deviations

10.1 Model Card: Cell-Level Parameterization

Formulation and dimension. Every free OD-time cell receives its own log-deviation parameter relative to a strictly positive baseline. The parameter dimension is therefore the number of free cells (plus any separately estimated observation or route-choice parameters). Frozen and structural-zero cells are absent from this dimension.

Required information and identifying assumptions. The model requires a baseline OD matrix with the exact estimation indexing. It assumes that multiplicative deviations around this matrix are a meaningful description of uncertainty and that the selected prior or penalty adequately regularizes directions not distinguished by counts. A zero baseline is an immutable zero under this parameterization unless the analyst supplies a positive seed.

Recoverable quantities and limitations. The model can express independent local changes and is therefore the least restrictive retained demand parameterization. Its flexibility is also its main limitation: count data commonly identify only aggregates or combinations of cells, while computation and memory grow with the number of free cells. Individual estimates may consequently be dominated by the baseline and prior. This model is appropriate for small problems, controlled validation, or local refinement where cell-level freedom is scientifically justified; it is not the preferred primary parameterization for the full network.

10.2 Baseline demand and Bayesian deviation parameterization

The model assumes that the analyst can provide an *a-priori* demand matrix $\mathbf{f}^{(0)} = \{\mathsf{f}_{\mathsf{q}}^{\mathsf{t},(0)}\}$, aligned with the OD/time-bin indexing of the scenario. Rather than placing a prior directly on the demand levels $\mathbf{f}$, we parameterize demand in terms of *log-deviations* from $\mathbf{f}^{(0)}$. This improves numerical stability, facilitates weakly informative priors, and makes the role of prior information explicit.

For each free cell c, an unconstrained raw deviation $\widetilde{z}_{\mathsf{c}}$ is mapped smoothly to a bounded log-deviation and positive demand:

$$z_{\mathsf{c}} = z_{\max} \tanh(\widetilde{z}_{\mathsf{c}}/z_{\max}), \qquad \mathsf{f}_{\mathsf{c}} = \mathsf{f}_{\mathsf{c}}^{(0)} \exp(z_{\mathsf{c}}).$$

Thus $|z_{\mathsf{c}}| \leqslant z_{\max}$ and a positive baseline can vary only within the declared multiplicative range. This is a smooth saturation, not hard clipping.

Zero baseline cells. If $\mathsf{f}_{\mathsf{q}}^{\mathsf{t},(0)} = 0$, then $\mathsf{f}_{\mathsf{q}}^{\mathsf{t}} = 0$ for all values of $\widetilde{\mathsf{z}}_{\mathsf{q}}^{\mathsf{t}}$. Therefore, OD cells that are zero in the baseline matrix remain structurally zero during inference. In order to allow strictly positive deviations from zero, a small but non zero seed value must be provided: $\mathsf{f}_{\mathsf{q}}^{\mathsf{t},(0)} = \varepsilon > 0$.

11 Conditional Gravity Demand

11.1 Model Card: Reduced Conditional Gravity

Formulation and dimension. Within each origin–departure-time production group, the model distributes a declared journey total across feasible destinations through a conditional logit expression based on attractiveness, scheduled travel time, transfers, and any explicitly introduced extensions. Its dimension is the number of shared behavioral coefficients plus the dimension of a declared production basis, rather than the number of OD cells.

Required information and identifying assumptions. The minimal model requires feasible journey alternatives, their fixed routing shares and attributes, positive destination attractiveness, and either externally justified journey productions or a low-dimensional production basis. It assumes a common parametric response to the included attributes and conditional independence from omitted attributes. The normalization within each production group identifies only relative destination utility; an unconstrained common additive utility is not identified.

Recoverable quantities and limitations. The shared parameters can be identified from count patterns even when individual OD cells cannot, and the fitted OD table follows from the estimated parameters and declared productions. However, this apparent precision is conditional on the gravity form, attractiveness values, feasible-set construction, fixed routing, and production assumptions. Omitted spatial or temporal heterogeneity can create systematic residuals. Because its parameter dimension remains small, this is the preferred primary demand model for the full-network MAP estimator, subject to adequacy and sensitivity checks.

11.2 Minimal Conditional Model

For feasible cell c in production group g, let $a_c > 0$ be attractiveness, T_c expected scheduled travel time, and K_c expected transfers. The minimal utility, conditional share, and demand are

$$V_c = \log a_c - \beta_T T_c / T_{\text{scale}} - \beta_K K_c, \qquad p_c = \frac{\exp(V_c)}{\sum_{k \in g} \exp(V_k)}, \qquad x_c = q_g p_c.$$

Externally supplied q_g gives the smallest model. If productions are estimated through a declared basis Z, a low-dimensional alternative is

$$q_g = q_g^{(0)} \exp((Z\gamma)_g).$$

Structural-zero and frozen cells do not receive free gravity-response rows. The count mean is obtained by applying the response operator to x and adding the frozen offset.

11.3 Declarative Gravity Specifications

The implementation no longer assumes that the first three parameters exhaust the model. A versioned `GravityModelSpecification` declares the scope, parameterization, normalization, feature mapping, and regularization of every component. The conservative template is `configs/gravity/gravity_model_spec.yaml`. It can be loaded and validated without editing estimation code; unknown fields and unsupported combinations are errors rather than ignored options.

The supported categorical scopes are global, origin, destination, time period, origin–time, destination–time, origin zone, destination zone, zone pair, and a named custom group. The scopes `none` and `fixed` introduce no estimated parameter. A smooth temporal basis is also supported when its cell-by-basis matrix is present in the feature cache and is named in the

time specification. These scopes can be applied, subject to component-specific validation, to production corrections, destination-attractiveness corrections, journey-time, transfer, and waiting-time coefficients, and departure-time effects. Negative-binomial dispersion is global or fixed. Estimated residual demand is deliberately rejected because no supported observation model yet identifies it separately.

For a general declared model, cell utility and production are

$$V_c = \log a_c + \alpha_c + \tau_c - \beta_{T,c} T_c / T_{\text{scale}} - \beta_{K,c} K_c - \beta_{W,c} W_c,$$
$$q_g = q_g^{(0)} \exp(\delta_g), \qquad x_c = q_{g(c)} p_c.$$

Positive behavioral coefficients use a softplus transformation with a small strictly positive floor. Production corrections are log multipliers; attractiveness and temporal corrections are additive utilities. Categorical deviations use either an explicit reference category or an exact sum-to-zero expansion. A grouped positive coefficient consists of one positive base and normalized log deviations. Origin–time and custom production corrections must carry ridge regularization because their dimension can otherwise approach the number of production groups.

The parameter layout is derived deterministically from the specification. It records one named slice per block, raw-to-physical and physical-to-raw transformations, constraints, regularization, and a stable fingerprint. Legacy names `beta_time`, `beta_transfer`, `dispersion`, and `production_scale` are retained, so the original three-parameter model and the optional global production scale remain backward compatible. Nested specifications are warm-started by matching these stable parameter names; new normalized deviations start at zero and reproduce the parent prediction exactly.

11.4 Feature and Identifiability Contracts

`GravityFeatures` is an immutable sparse cell table, not a dense origin–destination–time tensor. In addition to the canonical origin, destination, departure-time, and origin–time indices, it may contain time period, origin-zone, destination-zone, destination–time, zone-pair, named custom-group, waiting-time, and smooth-time-basis arrays. Required mappings are inferred from the specification and checked before estimation for length, integer type, contiguous labels, declared group count, and constancy within a production group where required. All optional mappings participate in the feature-cache fingerprint.

Invalid specifications are rejected, including a global production scale with an estimated detection-rate scale, an unnormalized categorical effect, and an unregularized high-dimensional production correction. Technically valid but weakly identified choices produce visible warnings. Examples are a global additive attractiveness correction, which cancels in each conditional softmax, and simultaneous grouped production and attraction corrections, which may be strongly correlated even after normalization. Model selection must therefore use grouped holdout performance, gradient and curvature diagnostics, regularization sensitivity, and count-mass audits rather than calibration likelihood alone.

The generalized generator preserves structural zeros exactly and applies only sparse per-cell vectors and grouped segment reductions. Its JAX implementation and independent NumPy reference agree at the established floating-point tolerances. The same demand function is exercised through negative-binomial objectives, calibration masks, fixed-positive measurement offsets, batched forward gradients, and matrix-free adjoint gradients. Unsupported measurement rows must be excluded by an explicit calibration mask.

11.5 Information Content and Measurement Design

The gravity model can diagnose weak information about its own parameters. Sensitivities of predicted counts, likelihood curvature, profile behavior, prior sensitivity, and posterior uncertainty

reveal parameter combinations that existing measurements distinguish poorly. Measurement-level derivatives also show which observed rows contribute to those combinations. This is not the same as residual analysis: a large residual indicates inadequate fit, whereas weak curvature indicates inadequate information, and either can occur without the other.

The low parameter dimension does not imply that every reconstructed OD cell is locally informed by counts. A precise-looking cell estimate may follow mainly from the gravity form, attractiveness, production, and feasible-set assumptions. Choosing new counter locations is therefore a separate experimental-design problem. Candidate measurements must be compared by their expected increase in information rank or conditioning, or by the expected reduction in uncertainty for declared quantities of interest. The gravity response supplies the derivatives needed for that analysis, but it does not automatically select new measurement locations.

12 Progressive Reduced-Dimensional Demand Modelling

The complete OD–time table is both computationally large and statistically underdetermined by aggregate boarding and alighting counts. The software therefore resolves one composable model specification before JAX tracing. A disabled component creates no parameter block, design matrix, objective term, or run-time branch. This is a software design choice; the statistical components themselves are established methods or explicit transit adaptations.

For origin-period group $g = (o, t)$ and cell $c = (o, d, t)$,

$$\log P_g = \log P_g^{(0)} + Z_g^P \theta_P, \qquad V_c = \log A_{dt}^{(0)} + Z_{dt}^A \theta_A - X_c \beta + I_c, \qquad q_c = P_g \frac{\exp V_c}{\sum_{k \in g} \exp V_k}.$$

The fixed-routing mean remains $\mu = Bq + c$. Period coefficients are the first relaxation because systematic temporal residuals can otherwise be aliased into every spatial component. Origin-group effects modify productions, whereas destination-group effects modify relative attractiveness; they are not interchangeable.

Automatic regions are deterministic nested partitions of normalized supply, topology, and measurement-observability signatures. They are model-resolution regions, not socioeconomic zones. The implementation adapts the automated resolution and held-out-validation motivation of Menon et al., 2015 and the homogeneity/contiguity hierarchy of Yuan et al., 2019; applying those ideas to transit supply and observability is a methodological hypothesis requiring empirical validation.

After margin heterogeneity has been assessed, a rank-r interaction uses

$$I_{odt} = U_{G_O(o),:} V^T_{G_D(d),:}.$$

The columns of both factors are centered. Ridge penalties stabilize their scale, but rotational ambiguity means individual factor columns are not given a behavioral interpretation. Low rank follows the dimensional-reduction insight used for time-varying transit OD inference by Chen et al., 2025; the present JAX/MAP implementation is not a reproduction of that paper's MCMC method.

Multiplicative gravity relationships are fitted in count scale rather than by log-linear least squares, following the Poisson pseudo-likelihood argument of Santos Silva and Tenreyro, 2006. The negative-binomial parameterization has mean μ, dispersion r, and variance $\mu + \mu^2/r$ Cameron and Trivedi, 2013. ZIP and ZINB add a separate Bernoulli zero state using stable log-space mixtures Lambert, 1992. This state is not a timetable structural zero: network-infeasible cells have already been removed, whereas zero inflation concerns retained measurement records.

Warm starts copy common named blocks, expand period or hierarchy blocks under their declared sum-to-zero convention, and initialize new interaction columns with small deterministic

Component	Added parameters	Purpose	Cost	Risk	Diagnostic trigger
Period effects	$T - 1$ per block	Temporal heterogeneity	Low	Reference/centering	Residuals by period
Origin groups	$G_O - 1$	Production heterogeneity	Low	Weakly observed origins	Origin-group residuals
Destination groups	$G_D - 1$	Attraction heterogeneity	Low	Attraction confounding	Destination residuals
Rank r	$r(G_O + G_D)$	Residual OD association	Moderate	Scale and rotation	OD pattern after margins
Negative binomial	1	Overdispersion	Low	Large-r boundary	Variance beyond Poisson
ZIP/ZINB	design columns	Separate zero process	Low	Mean/zero confounding	Excess zeros after NB

Table 2: Optional components of the resolved demand family.

Stage	Mean structure	Observation family
M0	Global production and impedance	Poisson
M1	Add period coefficients	Poisson
M2	M1	Negative binomial
M3	Add origin groups	Negative binomial
M4	Add destination groups	Negative binomial
M5	Add rank one	Negative binomial
M6	Ranks 2, 4, then 8	Negative binomial
Sensitivity	Smallest adequate mean	ZIP or ZINB only if justified

Table 3: Recommended progressive ladder; advancement is diagnostic, not automatic.

values. In-sample assignment residuals diagnose fit; held-out measurements diagnose generalization and must never enter fitting. Every block reports sensitivity, associated observations, gradient or curvature evidence, penalty contribution, held-out improvement, and whether it is data-supported, mixed, assumption-dominated, or unidentified. No synthetic "percentage from data" is reported. Baselines, grouping, gravity form, regularization, low rank, structural-zero rules, fixed routing, zero inflation, and identification constraints remain explicit assumptions.

The generic optimizer applies the same operational contract as the minimal reduced-OD fitter. Maximum likelihood and Gaussian-prior MAP are distinct modes; vector or canonical named bounds act in raw parameter coordinates. An absolute time budget requires an atomic checkpoint. A stopped run records the latest accepted iterate and resumes only when the complete model identity (specification, layout, features, grouping, response operator, observations, and software schema) matches. Compilation time and warm value-and-gradient time are reported separately from optimization time. Each accepted iteration additionally reports its duration, a rolling mean over at most ten iterations, the remaining iteration allowance, estimated remaining seconds, and the expected UTC completion time. The estimate excludes JAX compilation and is therefore meaningful only after several iterations. With a configured checkpoint, progress includes its path and age; an interactive interrupt atomically saves the latest accepted iterate and returns an explicit "interrupted" status suitable for later identity-validated resume.

For migration of existing examples, a thin adapter exposes an established fixed-routing gravity operator directly to the generic kernel. If the legacy OD rows require canonical reordering, the adapter permutes the demand vector on device before the operator product; it never materializes a dense response matrix. Bounded end-to-end tests exercise this path on both synthetic public networks and on the public Geneva GTFS snapshot. On the latter (96 free OD cells and 8,967 measurements, CPU, one optimization iteration), the generic M0 kernel compiled in 0.17 seconds and its warm value-and-gradient took 1.1×10^{-4} seconds. These figures validate integration and kernel cost, not statistical convergence; model comparison still requires the full progressive fitting and held-out workflow described above.

13 Auxiliary Demand Baselines

13.1 Route-Level Models

13.1.1 Model Card: Route-Level IPF

Formulation and required information. For one ordered route pattern, IPF finds a non-negative upstream-to-downstream vehicle-leg table matching the observed boarding and alighting marginals that are explicitly marked as available. It requires compatible marginals and a strictly positive seed on the feasible triangular support.

Assumptions and limitations. The seed and entropy-like proportional adjustment determine the allocation not fixed by the marginals. With incomplete marginals the result is explicitly underdetermined; even with complete marginals, many interior tables may share the same margins. Most importantly, a route boarding can be an initial boarding or a transfer, so the fitted cells represent vehicle legs rather than complete passenger journeys. Route-level tables must not be stitched together and interpreted as network OD demand without additional passenger-trajectory information.

Appropriate use. This model is an inexpensive audit baseline, data-consistency check, and possible initializer. It is not a competing full-network journey-OD estimator.

For an ordered route with stops $1, \dots, n$, let $x_{ij} \geqslant 0$ for $i < j$ and $x_{ij} = 0$ otherwise. IPF alternately scales feasible rows and columns toward the available boarding and alighting marginals,

$$\sum_{j>i} x_{ij} = b_i, \qquad \sum_{i<j} x_{ij} = a_j.$$

Unobserved rows or columns are not constrained and remain dependent on the seed. Complete marginals must be compatible with the triangular support; for each prefix through k,

$$\sum_{j=1}^{k} a_j \leqslant \sum_{i=1}^{k-1} b_i.$$

13.2 Entropy Models

13.2.1 Model Card: Entropic Transport

Formulation and required information. Balanced entropic transport allocates flow over feasible OD support while matching supplied journey-origin and journey-destination marginals and trading generalized cost against entropy. The unbalanced variant replaces exact marginal equality with declared divergence penalties. Both require a feasible support, costs, a regularization scale, and genuine passenger-journey marginals—not raw boarding and alighting totals in a network with transfers.

Assumptions and limitations. The resulting table is selected by the chosen cost, entropy scale, support, and marginal penalties; it is not identified by APC counts alone. Balanced transport is infeasible when supported marginals are incompatible. Unbalanced transport can reconcile disagreement, but the reconciliation is a modeling choice and does not change leg-event counts into journey marginals.

Appropriate use. Entropy models are valuable when credible journey marginals exist, for synthetic validation, or as a smooth initializer for another demand model. They are auxiliary baselines rather than the primary estimator in the present full-network setting.

For feasible support C, generalized cost c_{od}, and entropy scale $\varepsilon > 0$, the balanced model solves

$$\min_{x \geqslant 0} \sum_{(o,d) \in C} c_{od} x_{od} + \varepsilon \sum_{(o,d) \in C} x_{od} (\log x_{od} - 1)$$

subject to supplied journey-origin and journey-destination marginals. The unbalanced variant replaces exact marginal constraints with declared positive Kullback–Leibler penalties. It can reconcile inconsistent totals, but cannot turn vehicle-leg event counts into passenger-journey marginals.

14 Comparison of Demand Models

The models above answer different questions and are not interchangeable. The following comparison makes their principal assumptions and failure modes explicit.

Model	Unknown scale	Principal identifying information or assumption	Main limitation and role
Cell deviations	One parameter per free OD-time cell	Detailed count response, baseline matrix, and cell-level prior or penalty	Flexible but intrinsically weakly identified and high dimensional; use for small problems or targeted refinement.
Conditional gravity	A few shared coefficients, optionally plus a small production basis	Feasible alternatives, journey productions, attractiveness, and common responses to travel time and transfers	Misspecification can propagate systematically across many cells; preferred full-network model with MAP and diagnostics.
Route-level IPF	One triangular vehicle-leg table per route pattern	Observed route boarding and alighting marginals plus a positive seed	Does not identify complete journeys or distinguish transfers; audit baseline and initializer only.
Entropic transport	One flow per supported OD cell, implicitly selected by entropy	Genuine journey marginals, generalized costs, support, and regularization or marginal penalties	Solution depends on supplied marginals and entropy choices; auxiliary baseline or initializer.

14.1 Consequences for Interpretation

Parameter count is not the same as information content. Cell-level estimation has the largest parameter space but does not become more informative merely by being flexible. Gravity reduces dimension by asserting that many OD cells share behavioral coefficients and conditional normalization. IPF and entropy obtain determinate numerical tables by imposing marginal and seed or entropy structure; this determinacy must not be reported as identification from stop counts.

Likewise, the models estimate different objects. Route-level IPF estimates vehicle-leg movements. The other models target passenger journeys, but entropy requires externally meaningful journey marginals, while conditional gravity requires journey productions or a declared model for them. Raw APC boardings include transfer boardings and therefore cannot silently substitute for either journey origins or gravity productions.

14.2 Recommended Modeling Hierarchy

For a large network, the primary specification is the smallest conditional gravity model that is scientifically defensible, estimated by MAP. Its low dimension addresses both computation and

the intrinsic underdetermination of cell-level OD recovery. Increasing model complexity should be evidence-led: first examine residual structure, curvature, grouped holdout, and sensitivity; then introduce a narrowly defined temporal, spatial, production, or transfer effect and verify that it improves adequacy without destroying identification.

Cell-level deviations remain a controlled comparison and local refinement tool. Route-level IPF and entropic transport supply audits or initial values, not alternative evidence that overrides the journey model. Estimates from different rows of the table should therefore be compared at common observable quantities—especially predicted boarding and alighting counts—and not only by comparing OD matrices whose meanings and identifying assumptions differ.

Part IV
MAP Estimation and Alternative Inference Methods

15 MAP as the Primary Estimation Method

15.1 Operational Inference Hierarchy

The recommended operational estimator is MAP applied to the smallest adequate conditional gravity model. This combines a low-dimensional behavioral parameterization with explicit regularization of directions that counts do not identify strongly. It returns a reproducible point estimate without introducing one optimization variable per feasible OD-time cell.

MAP is primary here for practical and inferential reasons, not because it is universally superior. Maximum likelihood is the essential no-prior reference: it reveals how strongly the data determine the parameters and whether MAP is sensitive to regularization. Variational Bayesian inference targets a different output, an approximate posterior distribution, and is appropriate when uncertainty propagation is central and the approximation can be validated.

The fixed-routing linear MAP formulation is a separate model class. Its weighted least-squares observation model, physical-flow parameterization, and quadratic regularization need not match the negative-binomial gravity model. Agreement is expected only when the response, loss, parameterization, constraints, and prior penalty have all been made equivalent. Sharing the label "MAP" does not by itself make two estimators comparable.

15.2 Reduced and Linear MAP Formulations

For raw reduced-model parameters $\boldsymbol{\eta}$, Gaussian prior center $\mathbf{m}$, and positive scales $\mathbf{s}$, gravity MAP minimizes

$$Q_{\mathrm{MAP}}(\boldsymbol{\eta}) = -\ell(\boldsymbol{\eta}) + \frac{1}{2}\sum_{j}\left(\frac{\eta_j - m_j}{s_j}\right)^2.$$

The prior is defined on the declared raw parameterization. Infinite prior scales remove the penalty and recover the corresponding ML objective exactly. Finite scales generally change the optimum, especially along weakly identified directions.

Prior scale. The raw unconstrained variables, rather than the bounded effective deviations, are assumed independent and normally distributed:

$$\widetilde{z}_q^t \sim \mathcal{N}(0, \sigma_z^2).$$

The parameter $\sigma_z > 0$ controls the strength of regularization. Small values of σ_z constrain demand to remain close to the baseline, while large values allow the effective deviations to approach their smooth bounds. In the current version of the software, σ_z is a fixed hyperparameter.

The distinction between $\widetilde{z}_q^t$ and z_q^t is important: the Gaussian density is evaluated at the raw variable $\widetilde{z}_q^t$, whereas the assignment and all reported demand samples use the transformed value z_q^t.

15.2.1 Fixed-routing linear MAP estimation at scale

When θ and all generalized costs are fixed, routing probabilities are constant and the complete measurement prediction is affine in the free OD demand. Let $\mathbf{f}_u \in \mathbb{R}^n$ denote the free physical

flows, let $\mathfrak{m}$ be the number of observations, and let $\mathbf{c} \in \mathbb{R}^{\mathfrak{m}}$ be the measurement contribution of strictly positive frozen demand. The fixed-routing model is

$$\widehat{\mathbf{y}} = \mathsf{A}\mathbf{f}_{\mathcal{U}} + \mathbf{c}, \qquad \mathsf{A} \in \mathbb{R}^{\mathfrak{m}\times\mathfrak{n}}.$$

Frozen-zero cells have no column in A, while positive frozen cells contribute only to $\mathbf{c}$. Thus neither kind of frozen cell increases the estimation dimension.

The implemented linear MAP problem uses positive observation weights $W = \operatorname{diag}(\boldsymbol{w})$, componentwise physical bounds $\boldsymbol{\ell} \leqslant \mathbf{f}_{\mathcal{U}} \leqslant \mathbf{u}$, and an explicitly selected ordered collection of linear regularization blocks $(\mathsf{L}_{\mathsf{k}}, \mathbf{r}_{\mathsf{k}}, \lambda_{\mathsf{k}})$:

$$\min_{\boldsymbol{\ell}\leqslant\mathbf{f}_{\mathcal{U}}\leqslant\mathbf{u}} \left\{ \frac{1}{2} \left\| W^{1/2}(\mathsf{A}\mathbf{f}_{\mathcal{U}} + \mathbf{c} - \mathbf{y}) \right\|_2^2 + \frac{1}{2} \sum_{\mathsf{k}} \lambda_{\mathsf{k}} \left\| \mathsf{L}_{\mathsf{k}}\mathbf{f}_{\mathcal{U}} - \mathbf{r}_{\mathsf{k}} \right\|_2^2 \right\}.$$

The supported choices are no regularization, ridge toward the prior demand, and scaled ridge toward the prior demand. No recommendation is applied implicitly. This weighted least-squares mode is distinct from the negative-binomial ML and MAP objectives described below: the two are comparable only when their observation models and priors have deliberately been made equivalent.

16 Maximum Likelihood and Maximum A Posteriori Estimation

The differentiable assignment and measurement models can be used for point estimation without constructing a full posterior approximation. Let

$$\boldsymbol{\eta} = \begin{cases} (\widetilde{\boldsymbol{z}}_{\mathcal{U}}, \widetilde{\mathfrak{u}}), & \text{if } \theta \text{ is estimated,} \\ \widetilde{\boldsymbol{z}}_{\mathcal{U}}, & \text{if } \theta \text{ is fixed,} \end{cases}$$

denote the vector of raw unconstrained parameters. The same smooth transformations defined above map $\boldsymbol{\eta}$ to the effective demand vector $\mathbf{f}$ and, when applicable, to θ. Consequently, the likelihood-based ML, MAP, and Bayesian methods use the same forward model and negative binomial likelihood. The separate fixed-routing linear mode uses the weighted objective defined in Section 15.2.1.

16.1 Maximum a posteriori estimation

Maximum a posteriori (MAP) estimation uses the same likelihood and priors as the Bayesian model, but returns only the posterior mode:

$$\widehat{\boldsymbol{\eta}}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\eta}} \{\ell(\boldsymbol{\eta}) + \log \mathfrak{p}(\boldsymbol{\eta})\}.$$

Here $\mathfrak{p}(\boldsymbol{\eta})$ contains the Gaussian prior on $\widetilde{\boldsymbol{z}}$ and, when θ is estimated, the Gaussian prior on $\widetilde{\mathfrak{u}}$. The priors are evaluated on the raw variables, not on their smoothly bounded transforms.

The software implements ML and MAP through the common penalized objective

$$\mathrm{Q}_{w}(\boldsymbol{\eta}) = -\ell(\boldsymbol{\eta}) - w \log \mathfrak{p}(\boldsymbol{\eta}), \qquad w \geqslant 0.$$

The setting $w = 0$ gives pure maximum likelihood and $w = 1$ gives MAP under the same prior as the Bayesian model. Intermediate or larger values provide regularized point estimates, but do not in general correspond to the posterior of the stated Bayesian model.

17 Maximum Likelihood as a Reference Estimator

ML removes the explicit prior contribution while retaining the same parameterization and likelihood as the corresponding nonlinear MAP problem. It is therefore the appropriate diagnostic for prior sensitivity. A large ML–MAP difference is expected when the likelihood is weak, the prior is informative, or the optimum lies near a bound. Conversely, numerical agreement does not prove identification.

The public interface `compare_reduced_od_likelihoods` fits Poisson and negative-binomial specifications from the same gravity features and response operator. The shared artifact fingerprint establishes that the comparison changes the observation model rather than the underlying network problem, while distinct model fingerprints prevent accidental result reuse. Poisson is primarily a dispersion diagnostic: a material improvement from the negative-binomial model indicates variation beyond the Poisson mean–variance relationship, but does not by itself establish that the gravity specification is adequate.

The companion `run_reduced_od_prior_sensitivity` operation first computes ML and then warm-starts named MAP scenarios from the ML solution. Each scenario records its prior fingerprint and displacement from ML. This makes weak, moderate, and informative prior comparisons reproducible without silently changing the likelihood or prepared response artifacts.

Because the negative-binomial raw vector contains a dispersion coordinate that is absent from Poisson, one aligned bounds vector cannot safely configure both fits. The comparison interface therefore accepts likelihood-specific fit configurations. Bounds may be keyed by canonical raw parameter name and are resolved against each actual layout before compilation. Shared parameter names receive the same declared intervals, while the dispersion interval applies only to negative binomial. Returned entries retain the resolved ordering, initial point, bounds, shared prepared-artifact fingerprint, and distinct model fingerprint. Structured callbacks expose compilation, optimization iterations, checkpoint writes, deadlines, and model completion without adding objective evaluations.

17.1 Maximum likelihood estimation

The maximum likelihood estimator (MLE) is

$$\widehat{\boldsymbol{\eta}}_{\mathrm{ML}} = \arg\max_{\boldsymbol{\eta}} \ell(\boldsymbol{\eta}),$$

where

$$\ell(\boldsymbol{\eta}) = \log \mathcal{L}(Y \mid \mathbf{f}(\widetilde{\boldsymbol{z}}), \theta(\widetilde{u}), \rho, r)\,,$$

with the dependence on $\widetilde{u}$ omitted when θ is fixed. The MLE uses no prior information. The baseline demand $\mathbf{f}^{(0)}$ still appears in the parameterization $\mathbf{f} = \mathbf{f}^{(0)} \exp(\boldsymbol{z})$, but it is not a probabilistic prior in the ML objective.

After optimization, the raw point estimate is transformed using exactly the same smooth mappings as in Bayesian inference:

$$\widehat{z}^{\,t}_q = z_{\max} \tanh(\widehat{\widetilde{z}}^{\,t}_q / z_{\max}), \qquad \widehat{f}^{\,t}_q = f^{t,(0)}_q \exp(\widehat{z}^{\,t}_q),$$

and, when estimated,

$$\widehat{u} = u_{\max} \tanh(\widehat{\widetilde{u}}/u_{\max}), \qquad \widehat{\theta} = \exp(\widehat{u}).$$

17.2 Numerical optimization and local uncertainty

The current point-estimation implementation minimizes Q_w with gradient-based SciPy optimizers such as BFGS or L-BFGS-B. Gradients are supplied by JAX automatic differentiation. Because the objective need not be globally convex, convergence to the global optimum

is not guaranteed; initialization, stopping tolerances, and local curvature should therefore be inspected.

When requested, the software computes the Hessian of Q_w at the optimum and uses its inverse as a local covariance approximation. For ML this is an asymptotic likelihood-based approximation; for MAP it is a Laplace-style local approximation to posterior curvature. These standard errors are not equivalent to a complete posterior distribution and may be unreliable for weakly identified, asymmetric, bounded, or multimodal problems.

17.3 Precision, Bounds, and Convergence Semantics

The reduced-OD estimator uses double precision by default. In the `float64_required` mode, estimation stops before compilation unless JAX 64-bit support is active; it can be enabled by setting `JAX_ENABLE_X64=true` before starting the process or by calling `jax.config.update('jax_enable_x64', True)` before creating JAX arrays. The less restrictive `float64_preferred` and `allow_float32` modes are explicit choices. The result records the requested NumPy type, actual JAX input type, compiled objective and gradient types, and whether 64-bit support was enabled.

A requested function tolerance is checked against floating-point resolution. If Q_0 is the initial objective, the implementation compares

$$\texttt{ftol}\,\max(1, |Q_0|)$$

with a conservative multiple of $\epsilon_{\text{mach}}\max(1, |Q_0|)$. An unresolvable tolerance is rejected by default and may instead be admitted with an explicit warning. This avoids reporting convergence from a stopping threshold finer than the arithmetic can represent.

Optional lower and upper bounds apply to the raw optimization parameters. Their dimensions and the initial point are validated before SciPy is called. Convergence at a bound is assessed using the projected gradient: components that point outside the feasible region at an active bound are removed, while the remaining infinity norm is compared with the declared gradient tolerance. Non-finite objectives or gradients fail immediately.

Three conclusions are deliberately distinct:

Optimizer termination records SciPy's status, success flag, and message without reinterpretation.

Numerical convergence additionally requires finite quantities, a resolvable tolerance, and a sufficiently small projected gradient.

Scientific admissibility is not assigned automatically. It remains a user decision informed by adequacy, identification, production, and sensitivity diagnostics.

Consequently, an optimizer success flag alone is not reported as a successful scientific estimate.

18 Bayesian Inference via Variational Inference

Let $\widetilde{z}_{\mathcal{U}}$ denote the vector of raw OD-deviation variables for free cells and let $\widetilde{u}$ denote the raw variable associated with the dispersion parameter of the route choice model. The effective parameters supplied to the forward model are deterministic transformations of these raw variables. The posterior distribution is proportional to

$$p(\widetilde{z}_{\mathcal{U}}, \widetilde{u} \mid Y) \propto \mathcal{L}(Y \mid f(\widetilde{z}_{\mathcal{U}}), \theta(\widetilde{u}))\; p(\widetilde{z}_{\mathcal{U}})\, p(\widetilde{u}).$$

When θ is fixed, $\widetilde{u}$ and its prior are omitted.

Exact Bayesian inference is computationally intractable for high-dimensional OD vectors. The current implementation therefore uses *stochastic variational inference* (SVI) as implemented in NumPyro.

Variational approximation. The posterior is approximated by a parametric distribution $q_\phi(\widetilde{z}_\mathcal{U}, \widetilde{u})$ in the raw unconstrained parameter space. The parameters ϕ are chosen to maximize the *Evidence Lower Bound* (ELBO):

$$\mathrm{ELBO}(\phi) = \mathbb{E}_{q_\phi}\left[\log p(Y, \widetilde{z}_\mathcal{U}, \widetilde{u}) - \log q_\phi(\widetilde{z}_\mathcal{U}, \widetilde{u})\right].$$

Maximizing the ELBO is equivalent to minimizing $\mathrm{KL}(q_\phi \,\|\, p)$ between the variational approximation and the true posterior.

Guide family. The baseline implementation uses Gaussian variational families in an unconstrained parameter space (e.g., diagonal or low-rank approximations), and optimization is performed using stochastic gradients (Adam). For a requested low-rank value r_q, the implemented effective rank is

$$r_q^{\mathrm{eff}} = \min(r_q, d),$$

where $d = |\mathcal{U}| + \mathbf{1}\{\theta \text{ is estimated}\}$ is the reduced statistical dimension. This prevents a guide configured for the original model from retaining unnecessary covariance factors after many OD cells are frozen. The effective rank, rather than the oversized requested value, is recorded in the inference diagnostics. When $d = 0$, no guide is constructed.

Positive route-choice dispersion. When θ is estimated, an unconstrained variable $\widetilde{u}$ is mapped smoothly by

$$u = u_{\max} \tanh(\widetilde{u}/u_{\max}), \qquad \theta = \exp(u),$$

and a Gaussian prior is placed on $\widetilde{u}$. If a requested prior center μ_u is specified in effective log-dispersion space, its raw center is

$$\mu_{\widetilde{u}} = u_{\max} \operatorname{arctanh}(\mu_u/u_{\max}), \qquad |\mu_u| < u_{\max}.$$

Posterior transformation and reporting. Posterior samples produced by the variational guide are samples of the raw variables $\widetilde{z}$ and, when applicable, $\widetilde{u}$. Before reporting demand or θ, the software applies the same smooth transformations used during likelihood evaluation:

$$z_q^t = z_{\max} \tanh(\widetilde{z}_q^t/z_{\max}), \qquad f_q^t = f_q^{t,(0)} \exp(z_q^t),$$

and

$$u = u_{\max} \tanh(\widetilde{u}/u_{\max}), \qquad \theta = \exp(u).$$

Consequently, reported posterior summaries correspond exactly to the parameter values used by the assignment and likelihood, rather than to untransformed raw samples.

18.1 Interpretation of the Variational Result

Bayesian inference targets a distribution rather than a point. Posterior means, medians, modes, and transformed summaries need not equal ML or MAP. Variational inference adds another approximation by restricting the posterior to a declared guide family. Its uncertainty estimates must therefore be checked for optimization stability and, where possible, against simulation, local curvature, or a more accurate reference on a reduced problem.

19 Comparison of the Estimation Methods

The three estimation approaches share the assignment model, measurement equation, likelihood, raw parameter space, and smooth parameter transformations. They differ in the statistical quantity they target.

ML versus MAP. The MLE maximizes the likelihood alone, whereas MAP balances the likelihood with the specified prior. They therefore need not coincide. MAP is typically pulled toward the prior center and can be more stable in weakly identified problems. As the amount of informative data increases and the likelihood dominates the prior, the ML and MAP estimates will often become close, subject to regularity and identifiability conditions.

MAP versus Bayesian inference. If MAP and Bayesian inference use exactly the same likelihood, priors, and parameterization, the MAP estimate is the mode of the exact Bayesian posterior. It is not generally equal to the posterior mean or median. Equality between the mode and mean occurs only in special cases, such as an exactly symmetric unimodal posterior. The difference may be substantial for skewed, weakly identified, bounded, or multimodal posteriors.

Variational approximation. The implemented Bayesian procedure reports means and other summaries computed from the variational approximation q_ϕ, not from the exact posterior. Differences between MAP and VI results can therefore arise from three distinct sources:

1. the intrinsic difference between a posterior mode and a posterior mean;
2. approximation error introduced by the selected variational guide; and
3. optimization and finite posterior-sampling error in the VI procedure.

The posterior draws used to compute VI summaries introduce Monte Carlo variability, but this is not the only, and often not the dominant, reason for differences between the estimates.

When estimates should be close. Under standard regularity conditions, with a large informative sample and a well-identified model, the posterior is often approximately Gaussian and concentrated. In that regime the MLE, MAP estimate, posterior mode, and posterior mean may be close. This is an asymptotic property rather than an equality guaranteed by the software.

Recommended comparisons. For validating the implementation, pure ML should be compared across optimizers using $\mathsf{w} = 0$, and MAP should be compared with the mode of the same posterior using $\mathsf{w} = 1$. Comparing an ML or MAP point estimate directly with the reported VI posterior mean is useful diagnostically, but disagreement should not automatically be interpreted as a software error. In every comparison, the effective transformed values $(\mathbf{f}, \theta)$ should be compared rather than the raw variables $(\tilde{\mathbf{z}}, \tilde{\mathbf{u}})$.

19.1 Decision Rule for the Present Application

The full-network default is conditional-gravity MAP with fixed routing. The ML fit should be retained as a sensitivity reference whenever it is numerically stable. Local Hessian uncertainty is a diagnostic, not a replacement for a posterior. Variational inference is reserved for questions that require posterior uncertainty and for problem sizes where its approximation and runtime are acceptable. Cell-level linear MAP remains useful for smaller validation problems and targeted refinements, but is not the default for unrestricted full-network OD recovery.

Part V
Validation and Model Assessment

20 Adequacy and Identification

20.1 Full-Data Diagnostics

Full-data adequacy asks whether fitted means reproduce the observations used in estimation. Residuals and count-model deviance should be reported overall and by measurement type, line, direction, stop, period, and vehicle journey where those labels exist. Systematic grouped residuals indicate misspecification, data problems, or an inadequate observation model; a small aggregate error can hide compensating local errors.

Identification is a different question. It concerns sensitivity of the likelihood to parameter changes and is assessed through response derivatives, curvature eigenvalues, condition measures, profiles, prior sensitivity, and replicated fits. A model can fit calibration counts closely while leaving some parameters weakly identified. A strongly identified but misspecified model can also fit poorly. The two diagnoses must be reported separately.

For parameter vector $\boldsymbol{\eta}$, local curvature of the negative log posterior or negative log likelihood summarizes sensitivity near the fitted point. Small eigenvalues identify locally weak directions; a large condition number warns that reported parameter precision may be unstable. These local quantities should be complemented by prior sensitivity, profiles, and repeated fits because bounded, asymmetric, or multimodal objectives are not characterized by one Hessian.

The reduced-OD fit result operationalizes these checks without reconstructing the complete OD vector. It reports the symmetric Hessian eigensystem, numerical rank, condition estimate, and weak eigenvector loadings using production-basis labels where available. Negative curvature and numerical singularity produce warnings rather than being hidden by a covariance calculation. For MAP fits, the report separates likelihood and prior contributions, gives the penalty of each raw parameter, standardizes displacement from the prior mean, and warns when the prior dominates the fitted objective.

Production diagnostics are calculated from the origin–period production vector, not by materializing all OD cells. They include fitted-to-baseline ratios, total production, dispersion proximity to its numerical floor, and extreme production multipliers. These summaries test numerical and scientific plausibility but do not convert operational boarding or alighting events into journey productions. Active bounds, extreme multipliers, poor conditioning, or strong prior dominance must therefore be reported alongside residual adequacy.

When the objective is to plan additional data collection, these diagnostics must be evaluated for explicit candidate measurements. Expected information gain, improvement in conditioning, or reduction in uncertainty can support a counter-placement decision. Existing grouped residuals alone cannot: they identify where the current model fits poorly, not necessarily where an additional count would resolve a weak parameter direction.

21 Grouped Holdout Validation

Holdout validation asks whether a model fitted without selected observations predicts those untouched observations. Measurement rows must not be split independently when they share a vehicle journey, stop time series, or another operational unit: such a split leaks closely related information between calibration and validation. The grouping unit must be selected before examining results and must correspond to the intended generalization question.

For a partition $\mathcal{M} = \mathcal{M}_{\text{cal}} \cup \mathcal{M}_{\text{hold}}$, estimation uses likelihood contributions only from $\mathcal{M}_{\text{cal}}$. Predictions are computed for both sets, but held-out counts must have no effect on fitted

parameters. This invariance is a required validation check. Calibration and holdout results should report the same metrics and grouping definitions.

Holding out complete vehicle journeys tests transfer to other journeys in the represented timetable; holding out stops tests spatial transfer; holding out time blocks tests temporal transfer. None establishes validity under a changed network, timetable, fare, or crowding regime. Here grouped holdout is a model- validity check, not a claim of forecasting performance.

22 Detailed-Assignment Validation

22.1 Reconstruction and Comparison

After estimation, the compact free demand is inserted into the canonical full OD layout, positive frozen values are copied unchanged, and frozen-zero cells remain zero. Reconstruction is a reporting and validation operation, not part of an objective evaluation.

Let $\widehat{\mathbf{y}}_{\text{red}}$ be the reduced prediction and $\widehat{\mathbf{y}}_{\text{det}}$ the detailed-assignment prediction obtained from the reconstructed OD table under matching routing assumptions. Validation reports the difference vector and, at minimum,

$$\max_{\mathrm{m}} |\widehat{\mathrm{y}}_{\text{det},\mathrm{m}} - \widehat{\mathrm{y}}_{\text{red},\mathrm{m}}|, \qquad \sqrt{\frac{1}{M}\sum_{\mathrm{m}} (\widehat{\mathrm{y}}_{\text{det},\mathrm{m}} - \widehat{\mathrm{y}}_{\text{red},\mathrm{m}})^2}, \qquad \frac{\|\widehat{\mathbf{y}}_{\text{det}} - \widehat{\mathbf{y}}_{\text{red}}\|_2}{\max(\|\widehat{\mathbf{y}}_{\text{det}}\|_2, \epsilon)}.$$

The detailed calculation is executed only for post-fit validation. Agreement tests the reduced response relative to the detailed model; it does not validate behavioral assumptions shared by both.

23 Advisory Model Relaxations

Residual and sensitivity diagnostics may suggest a broad-period cost effect, a destination-zone attraction effect, an origin-zone production effect, or a transfer-place correction. Such a suggestion is a candidate scientific hypothesis, not an automatic model-selection decision. A residual pattern can also be caused by erroneous counts, an incorrect stop mapping, timetable misalignment, or an inadequate observation distribution.

Every proposed relaxation must state its added parameters, normalization, required metadata, prior, expected identifying variation, and minimum observation support. It begins at a zero-effect value that reproduces the parent model. Only after this nesting check should the child be fitted and compared using adequacy, grouped holdout, curvature, and sensitivity to initialization and prior scale.

Retain the smallest model that passes the declared checks. A relaxation that improves calibration fit but produces unstable parameters, worsens held-out performance, or merely absorbs a data defect should not be adopted. Automated scores can rank candidates for review; they cannot replace the scientific decision.

Relaxations can now be expressed as complete serialized specifications rather than Python edits. In addition to the three original atomic helpers, the declarative interface supports origin, destination, time-period, origin–time, destination–time, zone-pair, and custom-group effects, plus an optional smooth time basis. The validation summary lists the generated parameter blocks, required feature mappings, calibration and excluded rows, regularization, feature-cache and structural-zero fingerprints, and all identifiability warnings before optimization begins.

Completed model results record parameter count, convergence state, gradient norm, regularization contribution, supported count mass, and, when supplied, held-out likelihood and predictive error. Comparison utilities refuse to rank models that lack held-out metrics; ranking is based on held-out evidence, predictive error, and then parsimony, never on in-sample objective alone.

24 Recommended Validation Workflow

The following sequence keeps structural, statistical, numerical, and predictive questions separate.

1. **Validate inputs.** Check timetable and stop identities, physical-stop mapping, observations, time alignment, units, structural zeros, frozen demand, and fingerprints.
2. **Validate the response on a small case.** Compare reduced or fixed-routing predictions and adjoints with an explicit construction; verify frozen offsets and finite-difference derivatives.
3. **Fit the minimal gravity MAP model.** Record initialization, objective components, bounds, stopping criteria, iterations, gradient norm, elapsed time, and completion status.
4. **Establish numerical reproducibility.** Repeat from dispersed starts, verify prediction and objective agreement, inspect active bounds, verify the actual arithmetic precision and tolerance resolvability, and check sensitivity to tighter tolerances.
5. **Assess adequacy and identification separately.** Examine grouped residuals and deviances, then curvature, profiles, prior sensitivity, and weak directions.
6. **Compare likelihoods and priors.** Fit Poisson and negative-binomial models on the same prepared artifacts, then compare ML with named MAP prior scenarios. Explain material differences through dispersion, regularization, weak information, bounds, or distinct local optima rather than requiring equality.
7. **Run grouped holdout.** Refit without complete operational groups and report calibration and holdout metrics under the same model.
8. **Consider one justified relaxation at a time.** Verify nesting, refit, and retain it only if the combined evidence improves.
9. **Perform detailed-assignment validation.** Reconstruct the full OD table only for the final candidate and compare event and link predictions under matching routing assumptions.
10. **Report scope and uncertainty.** State which quantities are informed by counts, which depend mainly on assumptions, and which behavioral effects are outside the model.

Passing one step does not waive a later one. Optimizer convergence is not evidence of identification, calibration fit is not holdout validity, and agreement with fixed routing is not validation of the fixed-routing behavioral assumption.

Part VI
Limitations and Conclusions

25 Current Limitations

25.1 Large OD Layouts

An explicit dense measurement-by-OD matrix is infeasible for a full network, and repeated detailed graph traversals can dominate runtime even when only a small demand block changes. Matrix-free and sharded representations bound storage but do not remove the cost of an unnecessarily high-dimensional demand model. This motivates the reduced gravity parameterization and the separation between response construction, estimation, and detailed post-fit validation.

25.2 Identification and Model Dependence

Stop counts generally identify combinations of OD flows rather than every cell. Priors, gravity structure, production constraints, entropy, or seeds can select stable estimates, but they do not create observational information. Reported uncertainty and sensitivity must distinguish data-driven precision from restrictions imposed by the demand model.

25.3 Fixed Routing and Capacity

Routing probabilities are conditional on a fixed timetable, generalized costs, destination rules, and dispersion. Demand does not change travel time, capacity, denied boarding, or route utility. This is internally consistent with the current assignment but excludes crowding feedback and capacity-constrained equilibria. In heavily loaded conditions, a capacity-aware outer assignment and re-estimation loop would be required.

25.4 Observation and Data Limitations

The count likelihood assumes a correct mapping from operational records to timetable events and a suitable conditional count distribution. Missing records are not zeros, transfer events are not journey origins or final destinations, and systematic detector failures cannot be repaired by the OD model. Temporal aggregation and physical-stop mappings can remove or create apparent distinctions, so their uncertainty remains a source of model risk.

25.5 Reduced Journey Choices and External Inputs

The reduced model is conditional on feasible alternatives, fixed shares, travel-time and transfer summaries, attractiveness values, and journey productions or a production basis. Transfer limits and choice-set pruning may exclude behavior that occurs in practice. Gravity parameters can diagnose weak information within this specification, but optimal placement of new measurements is not yet implemented.

25.6 Inference and Computational Limits

MAP provides a mode and local curvature, not a complete uncertainty distribution. Variational inference provides an approximate posterior whose quality depends on the guide and optimization. Full cell-level estimation remains expensive even with a linear response because solving a large, ill-conditioned inverse problem is fundamentally different from merely executing a forward neural-network pass. The low-dimensional gravity model addresses this boundary

through modeling as well as software efficiency. Section 6.6 explains why the laptop-scale reduced workflow depends jointly on bounded timetable search, reuse of fixed responses, and parameter reduction. RAPTOR by itself does not remove the cost of exhaustive preprocessing or an unnecessarily large response representation.

26 Discussion

This software framework combines timetable information and operational counts to infer OD demand and quantify uncertainty, while using a differentiable assignment model compatible with ML, MAP, and gradient-based Bayesian inference. ML and MAP provide computationally convenient point estimates and local curvature diagnostics, whereas Bayesian VI additionally provides an approximate distribution that represents parameter uncertainty.

Planned extensions of the software include incorporating capacity effects in a differentiable manner, estimating additional measurement parameters (e.g., ρ and $\mathtt{r}$), supporting weighted measurement aggregations, and adding opt-out alternatives.

27 Conclusions and Recommended Use

The central difficulty is not only computational scale but intrinsic underdetermination: boarding and alighting counts observe events along journeys rather than complete passenger trajectories. A credible solution must combine a transparent forward model, explicit structural zeros, a scientifically defensible demand restriction, regularization, and validation at observable counts.

For the full network, the recommended primary workflow is fixed-routing, conditional-gravity MAP estimation. Fixed routing makes the response additive; gravity replaces one parameter per OD-time cell with a small set of shared behavioral and production coefficients; MAP stabilizes directions weakly informed by counts. ML remains the principal prior-sensitivity reference, and variational Bayesian inference is appropriate when posterior uncertainty is required and its approximation can be validated.

This recommendation is conditional, not universal. Route-level IPF and entropic transport remain useful audits and initializers, while cell-level MAP is valuable on small examples and for justified local refinement. Final acceptance requires reproducible optimization, adequate grouped residuals, reasonable curvature and prior sensitivity, grouped holdout, and agreement with detailed assignment under matching assumptions.

The next scientific extensions are evidence-led model relaxations, explicit measurement-design analysis, and capacity- or crowding-aware routing where the data require it. Until then, results should be reported as estimates under the stated timetable, routing, production, attractiveness, likelihood, and prior assumptions, with weakly informed quantities identified rather than hidden.

Part VII
Implementation and Validation Appendices

A Software Architecture

A.1 Separation of Responsibilities

Separation of responsibilities. The assignment procedure is treated as a deterministic mapping

$$(\mathbf{f}, \theta) \longmapsto \mathbf{x}(\mathbf{f}; \theta)$$

and remains agnostic to priors and baseline information. The construction of $\mathbf{f}$ from $\mathbf{f}^{(0)}$ and $\widetilde{\boldsymbol{z}}$ is performed in the *Bayesian layer* (implemented using NumPyro and stochastic variational inference), which: (i) reads the baseline $\mathbf{f}^{(0)}$ provided by the user, (ii) draws the raw deviations $\widetilde{\boldsymbol{z}}$ from their prior distribution, (iii) applies the smooth bound to obtain $\boldsymbol{z}$, and (iv) computes $\mathbf{f}$ using the expression above before invoking the assignment. This design avoids introducing additional indexing conventions and keeps the assignment module independent of the inference engine.

Implementation note (departure interval as OD/time-bin data). Although centroid-in nodes are not time-tagged in the graph, the desired departure *interval* $[\mathfrak{a}_t, \mathfrak{b}_t]$ remains an attribute of the OD/time-bin pair. In computations, the access-link penalty is evaluated by combining the interval bounds $(\mathfrak{a}_t, \mathfrak{b}_t)$ with the clock time τ of the *departure event node* reached by that access link. This yields a zero penalty for departures within $[\mathfrak{a}_t, \mathfrak{b}_t]$ and an early/late penalty based on the nearest bound otherwise, without introducing backward-in-time links in the acyclic time-expanded network.

A.1.1 Separation of routing preparation and demand loading

For a destination group $\mathfrak{g}$, let $\mathfrak{d}_\mathfrak{g}$ denote its destination, let $\mathcal{A}_\mathfrak{g}^+$ denote its enabled links after destination gating, and let $\mathbf{f}_\mathfrak{g}$ collect the OD/time-bin demand injected for that group. The backward value recursion and the resulting transition probabilities depend on $(\mathcal{G}, \mathbf{c}, \mathfrak{d}_\mathfrak{g}, \mathcal{A}_\mathfrak{g}^+, \theta)$, but not on $\mathbf{f}_\mathfrak{g}$. The assignment can therefore be written as two distinct operations:

$$\mathcal{R}_\mathfrak{g}(\mathcal{G}, \mathbf{c}, \mathfrak{d}_\mathfrak{g}, \mathcal{A}_\mathfrak{g}^+, \theta) = (\mathbf{V}_\mathfrak{g}, \mathbf{P}_\mathfrak{g}),$$

followed by

$$\mathbf{x}_\mathfrak{g} = \mathcal{L}_\mathfrak{g}(\mathcal{G}, \mathbf{P}_\mathfrak{g}, \mathcal{A}_\mathfrak{g}^+, \mathbf{f}_\mathfrak{g}), \qquad \mathbf{x} = \sum_\mathfrak{g} \mathbf{x}_\mathfrak{g},$$

where $\mathcal{R}_\mathfrak{g}$ is routing preparation and $\mathcal{L}_\mathfrak{g}$ is the forward flow-loading operation. For fixed probabilities, $\mathcal{L}_\mathfrak{g}$ is linear in the injected demand. This separation is exact: it does not approximate the route-choice model or alter its probabilities.

Activation policy. The modes are `off`, `auto`, `dense`, and `bcoo`. Explicit modes are preserved. In automatic mode, a valid BCOO cache is reused immediately. Without a valid cache, construction is selected only when the expected number of evaluations multiplied by the measured per-evaluation saving exceeds the expected construction time. Missing construction evidence or a nonpositive saving keeps the ordinary fixed-routing loader active. Estimated dispersion always disables the operator irrespective of this policy.

B Indexing Contracts and Provenance

B.1 Immutable Identities

B.1.1 Immutable reduced-OD configuration and identity contracts

The first implementation phase provides only the typed, immutable contracts needed by later preprocessing and inference; it does not yet construct routes, journeys, response operators, or estimates. The namespace `public_transportation.preprocessing.reduced_od` contains a strict version-1 TOML loader and versioned artifact identities. The namespace `public_transportation.inference.reduced_od` contains the canonical journey OD/time key, the exact free/frozen problem partition, and a small model identity.

The TOML loader rejects missing and unknown keys. It fixes the observation unit to timetable events, accepts only boarding and alighting in this backend, uses the missing-as-unobserved and duplicate-as-error policies, and requires external APC cleaning. Journey origins and destinations are fixed to first boarding and final alighting, time-bin membership is half-open, and route shares are fixed within one fit. The production mode is either `provided`, with an explicit input path, or `estimated_basis`, initially with the `origin_period` basis. Stop mapping is declared `authoritative` or `reviewed_generated`; output geography is scenario-stop or physical-stop level; detailed assignment is `explicit_only`. Relative input paths are resolved against the TOML file before the effective configuration is fingerprinted.

Artifact manifests carry independent schema and implementation versions, an artifact kind, the complete configuration fingerprint, sorted named source fingerprints, and sorted dimensions. Numerical arrays are copied into owned, C-contiguous, read-only NumPy storage. Their canonical descriptors include name, shape, dtype, and a SHA-256 content digest. Array order and all named metadata are canonical; duplicate or unsorted names are rejected rather than silently normalized.

The inference problem contract requires unique, lexicographically sorted journey OD/time keys. Strictly increasing free and frozen index arrays must be disjoint and must partition every key exactly once. Fixed values are finite and nonnegative. These arrays are also owned and read-only. Frozen cells therefore remain absent from the parameter dimension from the first public contract onward. Problem and model fingerprints use deterministic compact JSON plus SHA-256 and change when configuration, timetable artifact, response artifact, OD partition, fixed values, likelihood, production mode, or estimated parameter names change.

Canonical identity serialization is setup work, not an optimizer operation. A Phase-1 CPU microbenchmark with 100,000 OD keys produced a 2.30 MB JSON payload and a median combined serialization-and-hashing time of approximately 0.072 seconds. The measurement is machine-dependent and is not a test threshold; it confirms that contract identity is negligible relative to later preprocessing and occurs outside the repeated objective and gradient.

Cache consistency. A prepared cache is associated with the exact graph instance, base link costs, destination-group ordering, and original destination masks from which it was constructed. These quantities are checked before cached loading is used. A change to any routing-sensitive input invalidates the cache. Empty destination sets and compact OD layouts are handled explicitly. The cached scan accumulates only total link flow and does not materialize per-destination link-flow arrays when they have not been requested.

Persistent cache and provenance. Operators can be saved as compressed NumPy archives and reused by a fresh process. The cache key includes the assignment identity; content digests of the graph and all loading arrays; the complete measurement mapping; OD-layout and compact-layout fingerprints, including frozen indices and values; fixed dispersion; representation; numeric type; package version; and cache-schema version. Loaded dimensions and

array shapes are also validated. Cache writes use a temporary file followed by an atomic replacement. A missing, corrupt, outdated, or provenance-mismatched artifact is rejected and rebuilt safely. The cache directory may be supplied on the estimation request or through `PUBLIC_TRANSPORTATION_OPERATOR_CACHE_DIR`. Dense and JAX BCOO representations remain available as explicit overrides.

B.1.2 Deterministic indexing requirement

Reproducibility of inference results requires that the indexing of nodes, links, and OD cells remain strictly deterministic across runs. Graph construction, link ordering, and OD indexing must therefore follow fixed ordering conventions (e.g., sorted iteration and deterministic array construction). Any change in indexing invalidates stored inference outputs, as the Bayesian layer relies on a fixed mapping between parameters and link indices.

C Persistence, Restart, and Reporting

C.1 Persistent Artifacts and Recovery

Persistent assignment artifacts. Assignment preparation is also demand-independent. Its immutable reusable state consists of the time-expanded graph arrays and labels, destination/OD grouping arrays, and generalized-cost components. It does not contain demand magnitudes, observations, priors, optimizer settings, or compiled executables. The package can store this state as compressed NPZ plus canonical JSON through optional `off`, `auto`, `refresh`, and `readonly` policies; ordinary uncached calls remain the default.

The assignment-cache provenance includes the complete graph-relevant scenario contents, OD keys, assignment and cost configuration, numeric type, package and schema versions, sentinel conventions, and indexing rules. Loaded archives are checked for exact provenance, recognized fields, counts, shapes, dtypes, CSR and padded-mask consistency, and integer/Boolean conventions before JAX arrays are reconstructed. Cache writes use a unique temporary file, synchronization to disk, and atomic replacement. Thus concurrent writers are safe, corrupt or incompatible entries rebuild under `auto`, and `readonly` never writes. The measurement-operator cache retains its own compact-input content hash, so a valid assignment archive cannot validate an incompatible operator.

Internal TPG profiling initially assigned 3.142 s of a 3.313 s preparation to destination grouping. The cause was repeated Python scalar extraction from JAX node arrays, which introduced thousands of small device synchronizations. One NumPy view is now reused for immutable node indexing and validation, and all destination link masks are constructed in one broadcasted operation. Uncached assignment construction consequently fell to 0.097 s: graph construction took about 0.049 s, OD indexing and grouping 0.022 s, and cost construction plus synchronization 0.026 s in the compilation-cache-enabled run.

The TPG artifact contains 36,478 nodes, 11,259 links, 22,022 OD records, and 113 destination groups. Its explicit arrays occupy 6,909,418 bytes and compress to 576,227 bytes. A representative strict cache hit spent 0.00044 s opening the archive, 0.00460 s decompressing arrays, 0.01473 s in combined validation and decompression, 0.000006 s reconstructing host objects, 0.00500 s transferring and synchronizing JAX arrays, and 0.08787 s generating the canonical provenance. The complete assignment-cache call took 0.1084 s.

After eliminating scalar synchronization, uncached reconstruction is already slightly faster than strict cache validation in the assignment stage itself; canonical fingerprinting accounts for most of the hit time. The persistent artifact is therefore useful primarily for strict reproducibility and validation, while the focused grouping simplification provides the main speedup. Relative to the original profile, complete one-iteration time falls from 5.389 s to roughly 2.0–2.2

Policy/case	Assignment	Preparation	Warm V&G	Optimizer	Process
Off	0.0971	1.1756	0.000992	0.2908	2.1939
Populate	0.3412	1.3532	0.000993	0.2975	2.4106
Valid hit	0.1084	1.0775	0.000937	0.2721	2.0211
Corrupt/rebuild	0.3201	1.3339	0.000965	0.3073	2.4081
Mismatch/rebuild	0.3321	1.3463	0.001048	0.3166	2.3935
Read-only hit	0.1128	1.1242	0.000972	0.2922	2.1872

Table 4: Fresh-process TPG assignment-cache measurements in seconds with the measurement-operator and JAX compilation caches enabled.

s. Cache-hit runs at 1, 5, and 20 iterations reproduce objectives 55,243.4375, 49,635.4922, and 49,140.4258 exactly.

Anytime state and recovery. Initialization already constitutes a complete feasible approximate solution. After every accepted block or conflict-free parallel batch, the estimator has a complete flow vector, complete prediction, current and best objective, and convergence diagnostics. The durable journal records the replayable flow and prediction deltas using temporary-file creation, synchronization, atomic publication, and a separate commit marker. Periodic compact checkpoints store the complete state and exact next schedule position. Resume is rejected when any authoritative fingerprint changes, including scenario, assignment, OD layout, fixed demand, measurements, prior, routing, partition, solver semantics, or schema. Returned statuses distinguish convergence, time, sweep, and update budgets, graceful interruption with an approximate solution, resource guards, and numerical failure.

Progress events do not claim a generic percentage precision. They distinguish exact, sampled, stale, deferred, and unavailable diagnostics and report objective improvements, projected-gradient measures, block and variable coverage, completed sweeps, elapsed time, checkpoint commitment, and estimated remaining sweep time.

For a bounded large-network pilot, complete-operator products are governed by an explicit fingerprinted policy. The initial prediction may be computed or supplied with validated shape, finiteness, bounds, fixed offset, and provenance. Initial, periodic, and final exact projected gradients can be enabled independently; a disabled diagnostic is recorded as deferred, never relabeled as an exact global quantity. Resume prediction validation may likewise be exact or deferred. Deferred operation avoids an otherwise mandatory global forward or transpose product, but postpones the corresponding global consistency or optimality claim. A separate validation entry point can later recompute the prediction, objective, and projected gradient, including in a fresh process.

A restricted pilot schedule is separately authorized from a complete sweep. Every named block must belong to a completed support group, have compatible persisted exact support, and satisfy support-row, nonzero, operator-storage, and temporary-memory ceilings. The checkpoint records that deterministic schedule and its next position. Numerical semantics and the schedule remain fingerprinted, whereas invocation limits may extend safely, so resuming a five-update run with a six-update budget executes only the sixth update.

The monotonic elapsed-time deadline covers initialization, global products, selected-block construction, solving, and checkpoint persistence. JAX compilation or dispatched device work cannot generally be interrupted inside a process; an indivisible phase can therefore overshoot its allowance. Such an overshoot is reported explicitly, and no subsequent expensive phase is started. Structured diagnostics expose global forward and transpose counts and times, block construction and solve time, checkpoint time, prediction source, and validation status.

The selected-block numerical kernels pass graph and routing arrays as explicit dynamic JAX arguments rather than capturing complete assignment arrays in Python closures. Tracing, lowering, compilation, synchronized device execution, and host transfer are timed separately. A

bounded lock-protected compiled-executable cache is keyed by assignment provenance, backend, numeric type, effective OD width, mapped-edge width, graph dimensions, and kernel schema; deadline and callback state are excluded. Deadline checks remain in Python before and after each indivisible JAX stage. On the public example, fresh-process one-variable construction took 0.093–0.194 seconds, with zero captured array constants and identical sparse results; compilation dominated the roughly 0.001-second device execution.

For scheduler diagnostics, the builder can emit structured start and completion events immediately around each phase. An optional append-only JSON-lines sink flushes every complete record and can call `fsync`; it is separate from operator-cache validity. Consequently, cancellation during XLA compilation leaves a durable `jax_compilation_start` record without publishing an incomplete numerical operator. The public benchmark can stop intentionally after tracing, lowering, compilation, or execution, allowing these indivisible stages to be assigned separate external wall-time limits.

C.2 Phase-Specific Reduced-OD Reuse

Reduced-OD preprocessing artifacts are invalidated according to their actual dependencies rather than one monolithic configuration fingerprint. Physical stops, service-day selection, and timetable construction depend only on their corresponding service inputs. Journey artifacts additionally depend on the journey-search policy. Measurement responses and response operators add the measurement policy and spatial output convention. Likelihood and production mode belong to the model identity and therefore do not invalidate timetable, journey, or response artifacts.

Every reusable artifact still validates its schema, phase fingerprint, parent fingerprints, and numerical payload before acceptance. A changed likelihood can consequently reuse the complete prepared response chain, whereas a changed journey-search limit rebuilds journey choices and their descendants while retaining compatible physical-stop, service, and timetable artifacts. A changed production input republishes that input and the gravity/model stages without rebuilding the measurement response operator. Corrupt or incompatible artifacts fail closed.

Observed-data support reports are diagnostic artifacts, not scientific operator identities. Before a potentially long direct scheduled build, the complete positive-boarding admission report is atomically written as `positive_boarding_support_preflight.json` in the identity-specific checkpoint directory. The report is refreshed at canonical-origin, routing-support, and realized-cache stages. Changing observations therefore does not invalidate compatible routing or operator shards, but every new activation rechecks the supplied positive boarding rows before reusing or constructing an operator.

Potentially long journey construction accepts a progress callback that is not part of any numerical fingerprint. Messages contain the phase and status, completed and total query counts, current origin and departure, elapsed time, recent mean query time, predicted remaining time, and peak resident memory. Reporting is throttled by both a query interval and a wall-clock interval, with the final query always emitted. The callback therefore provides useful live feedback without changing reproducibility or flooding logs on fast cases.

Gravity run manifests use a separate model-level identity. They now contain the complete serialized gravity specification and fingerprint, named parameter blocks and transformations, feature-cache and direct-operator artifact fingerprints, calibration, holdout, and unsupported-row summaries, regularization settings, time-bin definitions, and destination-attractiveness provenance. Checkpoint fingerprints include the specification and parameter layout, so a changed scope, mapping, constraint, or ridge setting cannot resume an incompatible optimizer state. Conversely, such a model change does not invalidate a scientifically compatible fixed-routing assignment artifact.

D Computational Backends

D.1 Compilation, Batching, and Parallelism

Batched evaluation. For computational efficiency, value functions and flow propagation are computed in batches for OD groups sharing the same destination (and, depending on the implementation, the same departure interval), enabling vectorized evaluation with JAX.

Precomputation for fixed dispersion. When θ is fixed during inference, and generalized costs do not depend on the current demand, the routing state $(\mathbf{V}_g, \mathbf{P}_g)$ is invariant across all likelihood evaluations. The software consequently prepares the effective destination masks and link probabilities once, stores them in an immutable JAX-compatible cache, and evaluates only $\mathcal{L}_g$ as the OD parameters change. Reverse-mode differentiation then traverses the demand-loading and measurement computations, but not the invariant value-function and route-probability calculations.

This optimization is valid under the current uncongested assignment model, because link costs are fixed. It must not be applied when θ is estimated, or if a future model makes generalized costs depend on passenger flows. In those cases, routing is recomputed inside every assignment evaluation. The ML, MAP, and variational-inference pipelines select the cached path automatically for fixed θ and retain the dynamic path for estimated θ.

Computational consequences. Let G be the number of active destination groups, N the number of graph nodes, and A the number of links. Routing preparation requires one backward pass and probability construction per group, while each subsequent cached evaluation requires only the forward loading passes. The principal incremental cache arrays scale as $O(GA)$ for probabilities and masks. Compact OD grouping can reduce both G and this storage when frozen-zero demand removes complete destination groups.

Table 5 reports illustrative CPU measurements from the package examples. Timings are warm medians using 32-bit arithmetic and are machine-dependent; numerical equivalence is tested separately and these values are not used as test thresholds.

Example and layout	G	Cache (MiB)	Forward	Value–gradient	Speedup
Simple example, compact	7	0.20	9.44 → 2.75 ms	12.78 → 6.21 ms	2.06×
Geneva, compact	18	3.43	0.449 → 0.109 s	0.537 → 0.180 s	2.98×
Geneva, full	62	11.83	1.552 → 0.383 s	1.788 → 0.591 s	3.03×

Table 5: Dynamic-to-cached fixed-routing measurements. The final column reports the likelihood value-and-gradient speedup; forward speedups range from 3.43× to 4.12×.

Stable objective compilation and reuse. The fixed-operator MAP objective is defined at package scope and receives all numerical operator arrays, observations, baselines, and offsets as dynamic JAX arguments. Thus parameter values, observations, and optimizer iteration limits do not alter the compiled signature; shapes, dtypes, backend, JAX/JAXLIB versions, and compilation-relevant configuration still must agree. The public `prepare_ml_objective` and `compile_ml_objective` interface separates tracing, lowering, compilation, executable loading, and first execution, and `run_ml` accepts the resulting executable without creating another jitted closure.

An isolated investigation showed that a previously observed 29.484 s first-call delay was not XLA compilation. Fixed-routing preparation had launched unused asynchronous device work before the valid operator cache was loaded, and the first objective result synchronized that work. The pipeline now validates and loads the cached operator before preparing routing. On

the TPG case, isolated tracing takes about 0.026 s, lowering 0.011 s, cold XLA compilation 0.120–0.129 s, executable loading below 0.00001 s, and first execution about 0.0014 s.

The compact persistent artifact remains canonical row-major BCOO storage, with sorted unique indices. Three objective kernels were compared after all scenario preparation had completed. BCOO matvec, direct indexed scatter-add, and sorted segment reduction gave identical float32 objectives and gradients. Their warm value-and-gradient medians were respectively 0.001073 s, 0.000944 s, and 0.000973 s; direct indexed aggregation is therefore used by the stable objective.

Iter.	Cache	Compile/load	Warm V&G	Optimizer	Process	Objective
1	cold	0.1204	0.000975	0.303	5.800	55243.4375
1	populate	0.1285	0.000996	0.292	5.845	55243.4375
1	hit	0.0070	0.000983	0.297	5.167	55243.4375
5	cold	0.1231	0.001001	2.878	8.377	49635.4922
5	populate	0.1194	0.001021	2.604	8.167	49635.4922
5	hit	0.0075	0.000958	2.539	7.622	49635.4922
20	cold	0.1260	0.000991	5.805	11.623	49140.4258
20	populate	0.1260	0.000972	5.777	11.401	49140.4258
20	hit	0.0070	0.000996	5.820	11.047	49140.4258

Table 6: Fresh-process TPG compilation-cache and optimizer measurements. The compile column is cold compilation or persistent executable loading; process time also includes scenario and assignment preparation.

The JAX persistent compilation cache is configured before computation through `PUBLIC_TRANSPORTATION_JAX_COMPILATION_CACHE_DIR` or the public package configuration function. Entry-size and compile-time thresholds default to zero so the objective is eligible. Fresh hit processes create no new cache entry and reproduce cold objectives and gradients. JAX 0.8.1 on the tested Apple CPU emits an AOT target-feature warning while loading entries even though execution succeeds and results agree exactly; this runtime warning should be rechecked when JAX is upgraded.

Safe parallelism. Independent block operators can be constructed and cached concurrently. For block solves, the software builds a conflict graph whose nodes are blocks and whose edges represent shared measurement support or declared prior/constraint coupling. A deterministic coloring gives batches with disjoint exact update support. Such blocks may be solved concurrently and their deltas are merged atomically in deterministic order. Overlapping proposals are never treated as independent Gauss–Seidel updates. Construction workers, solver workers, and threads per worker are explicit, and their product is checked against the available CPUs; native numerical thread pools are bounded as well.

D.2 Implementation notes for fast JAX evaluation

The time-expanded network is fixed across evaluations. All graph connectivity data are precomputed once and stored as immutable arrays: a topological ordering of nodes; compressed representations of outgoing adjacency; and, for each link, tail/head indices and link type.

The backward recursion for the value function and forward propagation for flows are implemented using JAX primitives such as `logsumexp` and scan-like passes over the topological order. Variable out-degrees are handled through CSR-like adjacency and/or padded adjacency with masks, enabling compilation with `jit`. The implementation exposes routing preparation and demand loading as separate JAX-compatible operations. A composed operation preserves the original dynamic assignment behavior, while the fixed-routing inference path supplies prepared probabilities directly to the loading scan.

Complexity boundary. The software exposes a structural complexity profile so that dimensionality can be audited without relying on machine-dependent timings. For reduced dimension d and effective low rank r_q^{eff}, the principal statistical state sizes are

state	number of scalar entries
optimizer vector or gradient	d
diagonal Gaussian guide	$2d$
low-rank Gaussian guide	$2d + dr_q^{\text{eff}}$
full Gaussian guide	$d + d(d+1)/2$
dense Hessian or covariance	d^2.

These statistical quantities depend only on the free cells and optional θ, not on the number of frozen cells.

There is a separate deterministic cost boundary. Each likelihood evaluation passes a compact demand vector containing the n_{free} free cells and the $n_{\text{fixed},+}$ strictly positive frozen cells. Frozen-zero cells are absent from that vector, from the padded OD group arrays, and from assignment demand injection. A destination group is also removed when it contains no remaining cell. Thus the assignment demand-vector size is $n_{\text{free}} + n_{\text{fixed},+}$ rather than n_{OD}. The time-expanded graph and other scenario-wide static arrays remain unchanged, so forward runtime still depends on the graph and on the surviving destination groups. Positive frozen flows must participate in assignment predictions; when θ is estimated, even their route-choice probabilities change with the estimated parameter. The full n_{OD} vector is reconstructed only after estimation for persistence and reporting. For every VI, ML, or MAP problem, the implementation reports the total, free, frozen-zero, and frozen-positive OD counts; the compact assignment-vector size; and the original, surviving, and removed destination-group counts. These structural metrics accompany elapsed time so that runtime differences can be interpreted in terms of the work actually performed by the assignment model.

E Scalable Linear MAP

E.1 Representations and Resource Admission

Operator representations. All solvers use a common forward–transpose protocol. Small problems may use a dense reference operator. Production alternatives include a native sparse operator with versioned persistence, a matrix-free operator that performs fixed routing during each product, and a sharded sparse operator whose construction and resident storage are bounded. The fixed offset c is assembled once. Cache identities cover the assignment and OD layouts, fixed demand, measurement mapping, routing parameter, numeric representation, construction configuration, schema, and package version. Cache writes are atomic and an incomplete, corrupt, or incompatible artifact is never accepted as complete.

The monolithic solver interface currently provides a dense reference backend and a bound-constrained TRF/LSMR backend. Results are returned in physical demand units with objective components, residuals, gradients, bound KKT diagnostics, elapsed time, and available forward/transpose product counts. Exact rank and resolution analysis is reserved for small problems. The scalable path instead uses reproducible spectral and randomized probes and reports their convergence and Monte Carlo uncertainty explicitly.

Resource preflight and adaptive refinement. The scalable workflow first creates a structural partition and then performs bounded exact-support discovery. The old sharded planner materializes global origin support and retains per-column patterns and future construction tasks; it is therefore a compatibility path for reviewed small cases, not a full-network preflight. The streaming preflight instead prepares and analyzes one destination group at a time. It reduces exact reachable measurement rows immediately to group and block summaries, writes an

atomic checkpoint, and releases group-local routing, reachability, pattern, and mapping state before continuing. Retained state is proportional to destination and block summaries, not to OD–measurement support entries.

Four explicit modes distinguish structural inspection, deterministic sampled exact support, complete streaming exact support, and an explicitly authorized materialized compatibility plan. The materialized mode is rejected unless a prior conservative logical-support estimate fits its retained-state budget. Sampled selection includes small, median, 95th-percentile, and maximum structural groups. Its extrapolations report observed ranges and assumptions; sampled completion alone cannot authorize a complete-network pilot.

Elapsed time, process RSS, temporary group state, retained summaries, support rows, nonzeros, and operator bytes are effective limits. Budget exhaustion or interruption produces a typed partial result and a valid atomic checkpoint. Resume validates scenario, assignment, OD layout, fixed demand, measurement mapping, routing, partition, configuration, and schema fingerprints. It neither repeats completed groups nor skips pending groups.

Checkpoint identity separates numerical semantics from invocation policy. The semantic fingerprint covers mode, deterministic sampling and selected groups, support chunking and tolerance, persisted representation, and schema; the authoritative data and routing fingerprints remain mandatory. Elapsed and memory allowances, checkpoint cadence, and deterministic worker settings have a separate recorded policy fingerprint and may change compatibly on resume. Tightened limits are rejected before work if retained summaries or accepted blocks already violate them.

Cumulative support time is never reset and includes discarded work from an incomplete group. In contrast, the elapsed-time allowance applies only to the current invocation and its timer begins at zero after each resume. Reports give previous, current, and cumulative time, invocation count, allowance overshoot, and whether stopping occurred before a group or inside a bounded chunk. Thus exhausting a 120-second allowance does not cause the next 120-second invocation to stop immediately. An unfinished group is not committed and may be recomputed, while every completed group is skipped.

Only after support discovery does the resource preflight select the smallest, median, 95th-percentile, largest, or explicitly named blocks. Before invoking any builder it estimates CSR values and indices, transpose storage, construction temporaries, retained cache, and solver work arrays. An unsafe block is rejected before allocation; no unselected operator is constructed. For accepted blocks it records machine memory, logical and physical CPUs, cache storage, operator and local-solver memory, construction and product times, and cache/checkpoint sizes. The `auto`, `laptop`, `workstation`, and `server` profiles apply different conservative fractions of available memory. Worker count is bounded by both CPUs and safety-adjusted measured peak memory:

$$n_{\text{workers}} \leqslant \min\left(n_{\text{CPU}}, \left\lfloor \frac{M - M_0}{M_B} \right\rfloor\right),$$

where M is the selected job budget, M_0 is coordinator and assignment memory, and M_B is the conservative measured worker peak. Estimated cache storage is checked separately. The structured recommendation contains hard variable and nonzero ceilings, worker and thread allocation, peak memory, cache size, first-sweep and cache-hit sweep times, and uncertainty. Automatic sizing is never activated until the exact recommendation fingerprint is explicitly accepted.

Selected fixed-routing blocks are constructed directly from their authoritative assignment inputs, compact free-to-active OD layout, destination-group routing, measurement mapping, and persisted exact support. The production builder does not require a complete measurement operator. It combines one or more base OD chunks into a separately configured, memory-guarded OD batch. For each batch, one JIT-compiled forward dynamic program computes node reach probabilities once; fixed-size mapped-edge gathers then aggregate only bounded supported-measurement chunks on the host. Numerical zeros are filtered before canonical

COO-to-CSR assembly. Both CSR and CSC storage retain only the union of supported rows, while forward products scatter into the complete measurement coordinate system and transpose products gather from it. Consequently, the forbidden dense array of shape $\mathfrak{n}_{\mathcal{M}} \times |\mathsf{B}|$ is never allocated: the largest dense numerical buffers are bounded by the effective OD batch times the node count, mapped-edge chunk, or supported-measurement chunk.

Measurement-row lookup, enabled-link filtering, global-to-local row translation, and padded mapped-edge plans are invariant for the block and are prepared once, outside the OD-batch loop. The requested batch factor is reduced automatically until conservative reach, mapping, sparse-assembly, routing, solver, and retained storage estimates satisfy both temporary and worker ceilings. Failure of even one base batch is reported before numerical allocation. Because batching does not change the canonical per-column accumulation order, safe batch sizes share one logical cache identity.

Exact selected support and each numerical block are persisted independently by atomic replacement. Their identities include package and schema versions, all scenario, assignment, OD-layout, fixed-demand, measurement, routing, and partition fingerprints, the block and support fingerprints, fixed routing parameter, tolerance, dtype, and canonical accumulation contract. Pure performance chunk sizes are excluded from the logical cache identity. Content hashes, sparse dimensions, row coordinates, and dtype are validated before reuse; corrupt numerical caches are discarded and rebuilt, while corrupt or incompatible support artifacts are rejected. A fresh process can therefore load a block without repeating routing or numerical construction. In-memory retention is explicitly bounded and releasable.

The accepted measurements also define a conservative memory/runtime cost model. Before estimation, any unsafe candidate block is bisected deterministically until every child satisfies the accepted limits. Coverage remains exact, children retain a conservative measurement support, and blocks are never silently enlarged. If a minimum-size block is still unsafe, a resource guard stops the workflow before its operator is constructed. Refinement creates a new partition fingerprint, and the revised schedule is written in the initial checkpoint. A checkpoint for the obsolete parent partition therefore cannot resume the refined run.

This block-coordinate implementation is presently restricted to the fixed-routing linear MAP objective. It must not be used when routing changes with the OD iterate or when the required conditional prior cannot be evaluated exactly.

E.2 Fail-Fast Positive-Boarding Support Admission

Observed support is now an admission condition for direct scheduled construction, rather than a diagnostic deferred until after the linear map has been generated. The activation and construction interfaces require the observation vector. Let $\mathcal{O}_+$ be the active assignment-origin nodes (free OD cells together with positive fixed cells), let $\mathcal{L}_{\mathfrak{m}}$ be the assignment links mapped to boarding row $\mathfrak{m}$, and let $\mathfrak{y}_{\mathfrak{m}} > 0$. Before routing preparation, the implementation checks the cheap necessary condition

$$\{\mathrm{tail}(\ell) : \ell \in \mathcal{L}_{\mathfrak{m}}\} \cap \mathcal{O}_+ \neq \varnothing.$$

Because the strict boarding mapper uses access links entering the scheduled departure event, failure of this condition proves that no modeled demand can produce the observed boarding. This graph-level check distinguishes departure-interval centroid nodes even when they share a physical stop label. The canonical OD table is used to explain whether the location–interval is absent altogether, all of its cells are fixed at zero, or active cells exist but none starts at a mapped access-link tail. Callers may supply the structural-zero reason for every fixed-zero full OD index; these reasons are then counted in the row report instead of being guessed. The report also states explicitly that transfer boardings are outside an access-link-only first-boarding measurement contract.

The cheap condition is necessary but not sufficient. Immediately after origin-specific routing support has been discovered—and before planning, JAX kernel lowering, compilation, or numerical shard construction—the builder performs a second exact row-support audit. A positive boarding row must occur in the union of free-column support and positive-fixed support. If it does not, the reported cause is that no retained route under the current timing and feasibility rules reaches the mapped boarding event. A valid cached operator is audited in the same way from its realized nonzero rows and fixed offset before it is returned.

Any failure raises `UnsupportedPositiveBoardingError` and stops the expensive path. The exception carries the complete structured report; an atomic JSON copy named `positive_boarding_support_preflight.json` is written under the identity-specific checkpoint directory. Every affected row records its canonical row and measurement identifiers, observed value, location and interval, available OD-role counts, optional fixed-zero cause counts, mapping method, time, trip and line when supplied, a cause code, explanation, and remediation. The exception text contains aggregate row and mass counts plus a short preview, so tens of thousands of offending rows remain available in the JSON artifact without flooding the terminal.

This check addresses the failure mode observed in the completed full-network case: positive boarding rows were found only after construction even though their corresponding origin/period cells contained no free or positive-fixed demand. Under the current contract those rows are rejected before routing or linear-map generation. Regression tests cover absent origins, all-zero origins with upstream reason provenance, fixed-positive support, exact routing-support rejection before kernel compilation, cache auditing, and atomic row-report persistence.

E.3 Split Reachability and Measurement-Edge Gathering

The production sharded fixed-routing builder separates the expensive graph dynamic program from the measurement mapping. For each padded OD chunk it now evaluates one fixed-shape forward-reachability kernel, retains the OD–node state on the device, and applies a fixed-shape edge-gather kernel to each mapped-edge block. Consequently, the graph traversal is independent of the number of mapped-edge blocks. Graph topology arrays are dynamic JAX arguments and the two compatible executables are lowered and compiled once; the worker pool shares those executables while each worker retains its own routing-shard reader.

The serialized construction plan records separate estimates for `estimated_reachability_evaluations` and `estimated_edge_gather_evaluations`; the construction result also reports both realized counts and their dispatch times, in addition to total JAX, synchronization, host-transfer, sparse-filtering, and persistence times. Publication remains ordered and atomic, so worker completion order cannot change manifests or numerical artifacts.

Numerical shards carry the algorithm identifier `reachability-edge-gather-v1` in operator provenance. A mismatch invalidates old numerical shards while the support checkpoint continues to use the scientific/support provenance hash, allowing the expensive support discovery to be reused safely. The split implementation is validated against the existing dense reference and serial construction: structural zeros, fixed offsets, forward products, and transpose products are unchanged.

As a bounded smoke benchmark, the public `simple_example_02` fixture (eight-origin chunks and three reach/gather pairs) produced the following scale check on the development laptop. The old column is the pre-refactor checkout; the split column is the current checkout. Both stored 60 nonzeros in 9 shards, and the split path used two compiled executables.

	Old combined kernel	Split kernel
Construction (one worker, s)	0.278	0.306
Peak RSS (MiB)	445	444
Stored shard bytes	5,558	5,558
Reachability evaluations	3 (fused)	3
Edge-gather evaluations	3 (fused)	3

The fixture is intentionally too small to predict full-network wall time; the benchmark is a regression guard for artifact identity, execution accounting, and worker scaling. A completed full-network artifact was subsequently validated and timed separately.

E.4 Completed Full-Network Linear-Map Benchmark

The full-network direct-scheduled artifact has fingerprint

```
2b4285c26c5721c2dad749af459767ab
51c47734878bfd51d94d1c0916ed6b7a
```

It contains 5,674 persisted temporal blocks and exposes the fixed-routing affine response

$$\widehat{\mathbf{y}} = \mathsf{A}\mathbf{f}_{\mathcal{U}} + \mathbf{c}, \qquad \mathsf{A} \in \mathbb{R}^{426{,}436 \times 627{,}123},$$

with 100,257,824 stored nonzero coefficients. The corresponding dense matrix would contain 267,427,823,628 logical entries; the persisted operator has density approximately 0.0375%. The artifact occupies approximately 1.6 GB on disk.

The cache was loaded on a development laptop using the production CSR/CSC execution backend. The benchmark used an all-ones float32 free-OD vector; the particular OD values were deliberately irrelevant because the experiment measured the operator path rather than a substantive demand specification. The reported assignment response included the fixed positive-flow offset $\mathbf{c}$. The measurements were:

Phase	Time (s)
Contract preparation	45.352
Artifact validation and CSR/CSC activation	421.190
Warm-up assignment	0.211
Timed assignment	0.180
Complete benchmark process	467.099

Thus the expensive operation is one-time artifact loading and materialization of the in-memory forward-CSR and adjoint-CSC representations. Once loaded, one full assignment is a sparse matrix–vector product over approximately 100 million nonzeros and took 0.18 s in this CPU run, corresponding to about 5.6 forward products per second under the measured conditions. No routing construction or timetable traversal occurred during the benchmark. These results establish that repeated fixed-routing operator products are computationally practical on the laptop; they do not, by themselves, certify the wall time or convergence of a complete objective-gradient optimization. The next validation is one full objective-gradient smoke evaluation using the actual feature cache and observations.

F Stochastic and Progressive-Fidelity Evaluation

F.1 Experimental Partial Evaluation

F.1.1 Progressive-fidelity objective and gradient

For exploratory optimization steps, the gravity layer provides a deliberately simple stochastic calculation at an effort level between 1 and 100. Its sampling units are the routing shards that

already exist in the persistent fixed-routing cache; it does not repartition destination groups into transient microshards. A seeded random permutation of the persistent shard identities is generated once conceptually, and an effort level selects a prefix of that permutation. Selections are consequently deterministic, nested, and reproducible for a fixed seed. If k of N shards are selected, every shard contribution receives the Horvitz–Thompson expansion weight N/k. The result reports both the requested effort and the realized fraction k/N.

Since the likelihood is nonlinear in the complete predicted-count vector, the implementation does not scale separate shard likelihoods. It first estimates the complete additive routed-count vector, includes fixed measurement offsets exactly, evaluates the likelihood at that approximate vector, and applies the same sampled and weighted transpose operator to its cotangent. The resulting gradient therefore corresponds to the reported approximate objective. Direct parameter transformations, regularization, and dispersion terms remain exact. Only effort 100 is called exact; it uses every shard once with unit weight and delegates to the established complete adjoint calculation.

The calculation is a streaming two-pass adjoint. During the forward pass it loads one selected persistent shard, adds its weighted measurement contribution to one accumulator, synchronizes outstanding device work, and evicts the shard before loading the next one. It then evaluates the nonlinear likelihood once on the assembled approximate count vector. During the reverse pass it reloads the same selected shards in the same order, immediately pulls each weighted measurement cotangent back to one demand-space accumulator, and again releases all shard-local arrays. One JAX vector–Jacobian pullback maps the completed accumulator to the small gravity-parameter space after the last shard. Prepared forward batches, futures, and per-shard measurement vectors are not retained. Concurrency one is the safe default and currently the only admitted mode; larger concurrency must not be enabled until measured memory, including allocator reserves and temporary buffers, can be bounded conservatively.

A reusable exact anchor replaces the unanchored estimate of Ax by the control-variate estimate

$$Ax_0 + \widehat{A(x - x_0)}.$$

It uses the same sampled shards and weights in both passes and reproduces the anchor exactly when $x = x_0$. Operator, routing-cache, compact-layout, measurement, parameter, and likelihood identities prevent incompatible anchor reuse. An effort request of exactly 100 never follows this sampled path: it delegates to the established complete adjoint implementation.

The implementation records RSS before dispatch and at every shard boundary, the current phase and shard, completed work, a conservative estimate for the next shard, wall time, coverage, maximum shard influence, and scalar sampling- dispersion indicators for measurements and gradients. A configured RSS ceiling and safety margin can prevent the next shard from being loaded and return a structured, resumable interruption instead of relying on an out-of-memory kill. The quality labels derived from the dispersion indicators are internal sampling diagnostics, not exact error bounds. In particular, low dispersion among selected shards does not rule out bias from omitted shards. Convergence must therefore be confirmed with effort 100.

On the small public persisted-shard benchmark, separating compilation from warm execution reveals the intended runtime ordering. The 10%, 25%, and 50% evaluations take approximately 0.032, 0.033, and 0.041 seconds, respectively, compared with 0.043 seconds for the warm exact evaluation. Their peak RSS remains essentially constant near 614 MB, while the exact evaluation reaches approximately 663 MB. The first sampled call nevertheless takes about 2.6 seconds because it compiles the common forward and reverse shard kernels. These small-case timings demonstrate removal of per-shard dispatch and garbage- collection overhead; they do not predict the speedup or approximation error on the full network.

That limitation was resolved by the full-network validation dated 2026-08-05, recorded in `docs/reports/full_network_stochastic_gravity_validation_2026-08-05.md`. The experiment reused 234 persisted routing shards, a compatible exact control-variate anchor,

and an existing displaced-point exact reference at public revision `55ed81601d0d786f80b3 33f4e2b0474505c6a472`. Both sampled jobs used seed 20260722, nested uniform selection, sequential two-pass streaming, concurrency and resident-shard limit one, no prepared-batch retention, no exact recomputation, and no optimization.

The exact reference objective was 13,361,880, requiring 3,176.25 seconds and 98.67 GiB peak RSS. The 10-percent request selected 24 of 234 shards (10.256 percent realized), required 1,427.48 seconds, and attained a 2.225 speedup. Its internal peak RSS was about 8.53 GiB, but its gradient relative norm error was 19.20 percent despite a gradient cosine of 0.99870. The 25-percent request selected 59 shards (25.214 percent realized), required 3,475.87 seconds, and was therefore about 9.4 percent slower than exact. Its internal peak RSS remained about 8.58 GiB, while gradient relative error improved only to 17.21 percent. The 24-shard selection was a strict prefix of the 59-shard selection.

The validation therefore has two distinct outcomes. First, sequential shard residency and RSS admission successfully bound memory: sampled internal peak RSS remained nearly constant with effort and far below the exact peak. Second, uniform persisted-shard reduction failed to provide an optimization-quality runtime–accuracy tradeoff. Increasing selected work by a factor of 2.46 increased runtime by about 2.44 while reducing gradient error by only two percentage points; at 25 percent it eliminated the runtime benefit.

Objective agreement is not sufficient evidence of a useful gradient. The sampled objective relative errors were only 2.25×10^{-6} and 1.52×10^{-5}, and both gradient cosines were high, but gradient magnitude errors remained material. The 25-percent objective was also less accurate than the 10-percent objective. Both internal quality labels were `poor`. Their dispersion indicators were pessimistic relative to the measured reference errors, but correctly did not certify either evaluation. These indicators remain diagnostics, not confidence intervals or exact error bounds.

Consequently sub-100-percent uniform persisted-shard evaluation must be treated as an experimental diagnostic capability, not as a transparent faster backend for a precision-oriented deterministic optimizer. Merely increasing or decreasing the uniform effort percentage is not recommended. Effort 100 still delegates to the exact backend, and callers must validate gradient accuracy for their own application. This finding is specific to deterministic nested uniform sampling with sequential forward and reverse passes; it does not rule out materially different stratified or influence-aware sampling, variance reduction across iterations, gradient calibration, or optimizers designed for noisy gradients.

The first validation mode is deliberately a full-calibration adequacy analysis, not predictive validation. It compares reassigned counts with every observation used for fitting and reports observed and modeled totals, negative-binomial and Poisson deviances, MAE, RMSE, inverse-variance weighted RMSE, standardized negative-binomial residuals, configurable exceedance rates, and observed–modeled quantiles. User-provided measurement-aligned labels produce residual summaries by measurement type, line, direction, stop, time period, origin zone, and destination zone. Ordered vehicle-journey labels enable adjacent standardized-residual correlations. Resulting flags distinguish possible demand restrictions, routing or timing patterns, isolated suspect observations, and systematic overdispersion cautiously: they do not establish causality and never modify the model automatically.

Three atomic relaxations are available when the minimal model is inadequate. For K user-defined groups, each introduces $\mathsf{K}-1$ free deviations and derives the final component as the negative sum, so that the full effect vector is exactly centred. The alternatives are destination-zone log-attractiveness deviations, broad time-period deviations multiplying β_{time}, and multiplicative origin-zone corrections to $\mathsf{Q}_{\mathtt{ot}}$. A time-period intercept is intentionally excluded because it would cancel within an origin–time softmax. The zone and period mappings are application inputs and must be contiguous; origin-zone and period labels must also be constant within each origin–time group.

Each child model is warm-started by copying its parent coordinates and setting the new de-

viations to zero, which reproduces the parent predictions exactly. A configurable ridge penalty on the complete centred effect vector is therefore zero at the parent. Adding or removing one relaxation is explicit and reports its exact parameter count and execution impact. These effects require only indexed cell or group operations and retain the same sparse demand, fixed-routing operator, gradient, checkpoint, and validation machinery.

Candidate relaxations can be screened without fitting them. At the fitted parent, each applicable child is initialized at its exact zero-deviation warm start. With objective gradient g_c and regularized Hessian block H_c in the new child coordinates, the local estimated improvement is

$$\widehat{\mathsf{G}}_c = \tfrac{1}{2}\mathsf{g}_c^\top \mathsf{H}_c^{-1}\mathsf{g}_c.$$

The implementation uses an eigendecomposition and floors non-positive or nearly singular curvature only for this approximation, while reporting the resulting identification warning. The candidate record contains the score statistic, estimated deviance improvement, observation and group support, exact parameter increment, computational cost, recommendation strength, and a plain-language interpretation. It also warns when the parent gradient is too large for an optimum-based diagnostic.

The score is interpreted together with grouped standardized residuals by destination zone, period, and origin zone. Correlations with journey-time and transfer sensitivities can motivate future impedance or transfer-penalty designs, but these are explicitly marked unavailable until a centred child model and its mapping are selected. Coherent within-journey residuals instead warn about possible routing or timing errors, and isolated extremes warn about possible data problems. The catalog is strictly advisory: it neither changes the specification nor estimates or applies a child model.

Completed models are retained in an immutable lineage. Every node has a deterministic identifier and records its parent and applied relaxation, complete specification and parameter layout, raw and physical estimates, feature and compact-layout fingerprints, assignment, graph, and measurement-mapping provenance, optimizer state, assigned-count adequacy diagnostics, runtime and stored-memory diagnostics, checkpoint path, and explicit calibration and validation measurement identities. A child's declared atomic relaxation must remove cleanly to its parent specification. The append-only lineage returns a new value when a child is added, preserving both completed estimation results.

The progression API first loads a completed parent, lists applicable relaxations, and returns the advisory recommendation catalog. It proceeds only after the user passes one explicit relaxation scope. The resulting child layout is initialized from the parent and its predictions are checked against the parent before any optimization. The child is then estimated and validated with the existing checkpointed machinery. The comparison reports changes in objective, likelihood, negative-binomial and Poisson deviance, RMSE, parameter count, and runtime. Calibration and validation measurement identities must remain unchanged during this step. No recommendation is selected automatically, and grouped holdout validation remains a separate subsequent workflow.

After selecting a model structure, predictive validation uses deterministic grouped holdout re-estimation. Complete vehicle journeys, stop-time series, lines, directions, broad time blocks, or explicit application groups are kept indivisible; independent random measurement-row holdout is not supported. A seeded hash orders groups reproducibly. Optional strata may combine measurement type, line, direction, stop, period, origin zone, and destination zone. Groups that cross several stratum values remain intact and use their complete label signature. The resulting complementary calibration and holdout masks, group labels, measurement identity, metadata identity, and policy are serialized under a content fingerprint and can be reused across model structures.

For each selected structure, all parameters are re-estimated using calibration groups only. The resulting complete OD demand is then assigned to the complete measurement operator, and the excluded groups are evaluated as predictions. Calibration and holdout results separately

report totals, negative-binomial and reference Poisson deviance, likelihood, MAE, RMSE, and inverse-variance weighted RMSE. The immutable report retains the split and model fingerprints, complete measurement predictions, free and full OD demand, and the fitted result. Tests verify that changing only excluded observations changes holdout metrics but cannot change the estimated parameters.

The recommended workflow is therefore to develop structure with full-data adequacy, select it explicitly, re-estimate under one reusable grouped split, compare predictive evidence, and revise only when that evidence warrants it. The accepted structure may subsequently be refitted on all observations, while the grouped holdout result is retained as the honest predictive assessment.

A public synthetic performance benchmark exercises several gravity parameter counts with dense and BCOO measurement operators. It records tracing, lowering, compilation, first and warm execution, changed-parameter reuse, forward and transpose routing, batched-forward and adjoint value-and-gradient execution, operator storage, process peak memory where measurable, and problem dimensions. The two gradients are checked numerically before timings are accepted. Small-case CPU results show mixed strategy winners with differences often near timer noise; they do not justify raising the conservative eight-parameter guard above which automatic selection avoids a wide batched demand Jacobian. Compiled-handle reuse under changed parameter values is counted directly. Persistent JAX cache hits are reported only by a separate fresh-process protocol because the public in-process API provides no authoritative hit counter.

The complete methodology has also been exercised on the committed public Geneva snapshot, whose OD truth and prior are synthetic. Scheduled path metrics prepare journey times and transfers, while positive prior free demand prepares production and attractiveness offsets. The case contains 96 free cells in a 15,128-cell full layout and 8,967 boarding/alighting measurements. Because a node-by-measurement reverse state is unsuitable at that measurement dimension on the laptop, an exact scalar-column builder compiles the fixed-routing forward map once and constructs BCOO data and indices directly, without a dense operator. The stored operator has 0.556% density and occupies 95,760 bytes.

A bounded two-iteration run completed minimal negative-binomial estimation, full-data adequacy, advisory diagnostics, an exact zero-difference warm start for a selected broad-period child, immutable two-node lineage, and grouped vehicle-journey holdout re-estimation. The holdout contains 864 measurements and the calibration set 8,103. These iteration-limited figures establish public end-to-end software integration; they are not converged estimates and provide no scientific model-selection conclusion.

Two smaller synthetic examples provide worked applications of the same methodology. Simple Example 01 contains four free cells and two frozen-positive cells. Its run reconstructs both fixed cells exactly, exercising the fixed measurement offset; since every free origin–time group contains one destination, this is not evidence for behavioral identification. Simple Example 02 contains 70 free cells, two frozen-positive cells, and 270 measurements. Its sequence estimates the minimal model, constructs an explicitly selected broad-period child with an exact zero-difference warm start, records two-node lineage, and re-estimates the child on 208 calibration measurements before scoring 62 measurements held out by complete vehicle journey. These summaries are instructional checks, not scientific model-selection conclusions.

The gravity objective now consumes a documented operator protocol rather than an explicit matrix. The protocol includes dimensions, fixed offsets, fingerprints, representation, JAX forward, transpose and optional multiple-right-hand-side products, plus declared progress, deadline, cancellation and cache-diagnostic capabilities. Dense and BCOO operators retain their stored matrix for compatibility, while matrix-free implementations satisfy the same contract without materialization. Thus a large logical measurement-by-OD matrix need never be constructed or transferred to the host. The adjoint value-and-gradient path performs one forward and one transpose product and reuses the routed mean when assembling diagnostics. For a

matrix-free operator, automatic derivative selection chooses adjoint directly, avoiding two cold compilations of the complete network; batched-forward remains an explicit small-parameter option. A linear custom-JVP rule also supports forward-mode transformations without materializing the Jacobian. Explicit deadline-governed preparation separates tracing, lowering, compilation, and first and warm execution for forward and transpose products and records the deadline phase, indivisible-operation overshoot, shapes, dtype, backend, and devices. A bounded packaged-example benchmark records these diagnostics, process memory, adjoint objective timings, and checkpoint size while asserting that no global operator was constructed.

Complete fixed-routing preparation itself can be too large when both the effective mask and probability arrays have destination-group-by-link shape. The bounded implementation partitions destination groups canonically, applies both a group-count and a byte ceiling, pads the final shard, and reuses one fixed-shape compiled routing kernel. Each completed shard is flushed and atomically published with strict fingerprints covering the assignment, graph, costs, source masks, destinations, theta, dtype, numerical implementation, schema, package version, and shard plan. An atomic manifest makes deadline or memory stops resumable without recomputing valid shards.

The sharded matrix-free operator loads routing state on demand with bounded LRU residency. Its former Python traversal of every graph node for every group is retained only as a small-case numerical reference. Production forward and reverse products use fixed-shape compiled JAX kernels over all padded groups in a shard, consuming the persisted effective masks and routing probabilities. They neither recompute paths nor routing probabilities. An optional bounded execution batch processes several independent routing shards in one compiled call; a partial final batch is padded without changing canonical OD order. Measurement scatter-add is fused into the compiled forward product, and the reverse kernel is its exact VJP. No complete routing array, link-flow-by-shard stack, Jacobian, or measurement-by-OD matrix is assembled.

The compiled assignment core expresses its destination-group dimension as a sequential `lax.scan`. Consequently, merely enlarging an aggregate batch does not expose independent shard work to additional CPU cores. The multicore implementation instead dispatches already-compiled single-shard scan kernels through a bounded worker pool. Its concurrency is limited by an explicit request, the detected CPU and shard counts, and a ceiling on live in-flight routing bytes. Results are accumulated in canonical shard order, preserving deterministic forward, transpose, and multiple-right-hand-side products. A vectorized-group alternative exposed more cores but was slower on the larger public forward benchmark because it materialized group-by-link intermediates, so it remains experimental.

Structured progress distinguishes product start, batch load, execution, accumulation, completion and deadline-prevented dispatch. Before an indivisible batch, the operator compares remaining absolute-deadline allowance with its recent batch prediction plus a safety margin. Cancellation or an insufficient allowance raises a dedicated interruption exception and discards the partial sum. The estimator propagates its deadline into these products and checkpoints only complete optimizer iterates. On Simple Example 02, four shards covering seven destination groups reused one compiled shape and agreed exactly with complete masks and probabilities; a fresh reload reused all four persisted shards.

After the first measured batch or concurrent wave, deadline admission projects the duration of every remaining wave plus its safety margin. If the complete product is infeasible, a `product_deadline_infeasible` event records completed and total shards, the measured wave, predicted remainder, deadline allowance, discarded partial-work duration, batch size, and concurrency before any further dispatch. Since a partial operator product is not a valid objective evaluation, its accumulated vector is never returned or checkpointed.

A public bounded preflight validates every cache shard and its fingerprints, then, at individually selectable safe boundaries, executes one forward product, one reverse product, both three-parameter objective-gradient strategies, runtime projection, and an execution recommendation. It reports the measured warm strategy, current shard batch and residency settings,

expected evaluation time and memory, suggested wall time and checkpoint interval, and a laptop-versus-server recommendation. The scalable synthetic benchmark varies graph, link, group, shard, OD, measurement, residency and operator-batch dimensions and reports synchronized forward, reverse and multiple-right-hand-side timing and resource decompositions.

On the 8,192-node public synthetic case, the best aggregate scan required 28.1 ms for a warm forward product at approximately one effective core. Four concurrent scan workers reduced this to 14.7 ms at 2.28 effective cores, and eight workers to 13.9 ms at 2.74 effective cores, a 2.01-fold throughput gain. Reverse and three-column products improved similarly. The benchmark emits a regression warning unless concurrent forward throughput exceeds the aggregate reference by at least ten percent. These results establish the mechanism but do not substitute for a bounded target-cache preflight.

Before the first measured batch is available, the caller may supply a conservative initial batch-duration prediction obtained from preflight. It is used by the same pre-dispatch deadline inequality and is replaced by the first completed empirical duration. This operational estimate does not participate in routing or model fingerprints. For unattended execution, an atomic run manifest records repository and package revisions, numerical fingerprints, dimensions, dtype, theta, operator batching and residency, estimator policy, JAX devices and relevant thread environment. A shared append-only JSONL sink accepts both operator and optimizer progress, timestamps and flushes each record, and optionally synchronizes it to durable storage. Consequently a terminated process leaves both its last complete optimizer checkpoint and a diagnostic record of the active product boundary.

Shard construction may use a bounded thread pool sharing the graph and one XLA executable. Before every dispatch, the parent accounts for current RSS, active-shard estimates, and a safety margin; progress and manifests are committed in canonical order even when execution finishes out of order. A rolling warm- shard estimate prevents launching work that clearly cannot finish before the absolute deadline. Synchronized diagnostics separate input preparation, transfer, kernel dispatch, device synchronization, host transfer, validation, persistence, and cleanup.

Controlled DAG experiments varied groups, nodes, links, outgoing degree, and enabled density. Runtime was essentially unchanged between 10% and 100% enabled density but increased with graph size and group count, confirming full-graph traversal per group. The packaged example is approximately 91% dense, so a compact-support schema is deferred until target-network diagnostics demonstrate sufficient sparsity. Serial, two-worker, and four-worker synthetic preparations produced identical probabilities and compiled one shared executable.

G Benchmarks and Validation Records

G.1 Assignment and TPG Measurements

Construction profiling. Each chunk reports its ordinal number, fixed shape, completed columns, elapsed time, host peak memory, kernel dispatch, device synchronization, and NumPy transfer. Final metrics record the explicit compilation count and time, number of chunks, construction and assembly time, logical dense bytes, stored bytes, nonzero count, density, host peak memory, and device peak memory when the backend exposes it. Measurement aggregation has no separately dispatched phase because it is fused into the reverse loading kernel; it is therefore not double-counted as a separate device operation.

TPG structural measurement. For the TPG case, the operator has shape $3{,}690 \times 15{,}772$, or 58,198,680 logical entries. Only 158,219 entries are nonzero, giving a density of 0.271860%. The logical dense float32 size is 222.01 MiB, whereas the direct BCOO representation occupies 1.811 MiB. This observed sparsity, rather than an a-priori assumption, motivates BCOO as the automatic representation.

Chunk	Calls	Compile (s)	Build (s)	Process peak (GiB)	Warm value–gradient (s)
128	184	0.237	116.587	4.212	0.001115
256	124	0.238	79.264	4.220	0.001133
512	113	0.241	71.999	4.222	0.001075
1024	113	0.238	72.781	4.224	0.001164
2048	113	0.244	72.544	4.210	0.001104

Table 7: TPG direct-BCOO construction by fixed OD chunk size. The 2048 case was the largest tested safe value. At 512 and above every destination group fits in one call, so larger chunks do not reduce the 113 calls.

These measurements were rerun on the EPFL Scitas Jed server with Python 3.14.5, JAX 0.11.0, and NumPyro 0.21.0. At chunk size 1024, device synchronization accounted for 71.316 s of the 72.781 s construction; NumPy transfer took only 0.0065 s and compilation occurred exactly once. Relative to the 3.3866 s Jed reference evaluation, the measured construction break-even is approximately 21.5 evaluations. Fresh processes validated and loaded the persistent cache in 0.0203–0.0210 s. The chunk-1024 construction process peaked at 4.224 GiB RSS; the complete sequential benchmark batch peaked at 6.16 GiB.

Iterations	Evaluations	Loader (s)	Cached BCOO (s)	Speedup	Objective difference
1	4	14.814	0.950	15.6×	0.0000
5	8	34.514	7.127	4.8×	0.0039
20	24	104.566	17.413	6.0×	0.0078

Table 8: Fresh-process TPG optimizer timings from the unchanged profiling harness on Jed. Compilation is reported separately by that harness. Warm cached BCOO value-and-gradient time was 0.001007–0.001276 s, versus 3.378727–3.657539 s for the fixed-routing loader.

Across three distinct demand vectors, the maximum absolute differences were 0.00390625 for the objective, 0.001953125 for the likelihood, and 2.861×10^{-6} for any gradient coordinate, consistent with float32 rounding. Tests also cover raw measurement predictions, frozen-zero cells, positive frozen offsets, cache round trips, corrupt-cache rebuilds, and provenance mismatch rejection.

H Future Extensions

H.1 Reserved Modeling Alternatives

H.1.1 Opt-out alternative (future extension)

In a general formulation, an opt-out alternative can be introduced for each OD pair and departure interval. In the first implementation, opt-out alternatives are not included: all demand is assumed to be assigned to the public transportation network.

Alternative observation models (future extension). Other likelihood choices (e.g., Gaussian noise on counts or Poisson models) may be implemented in the future, but the current baseline implementation uses the negative binomial likelihood described above.

Alternative inference engines. Markov chain Monte Carlo methods could be used in principle, but they are computationally expensive in high-dimensional OD settings. Support for alternative probabilistic programming frameworks (e.g., PyMC) may be considered in the future; the present baseline is NumPyro+SVI.

I Relocated Implementation and Benchmark Record

This appendix preserves material that originally combined mathematical definitions with software paths, implementation phases, cache contracts, or machine-dependent benchmarks. The scientific content is restated in Parts I–VI; the original wording is retained here for implementation traceability.

I.1 Passenger journeys, vehicle legs, and observed events

The OD demand f_q^t is interpreted as *passenger journey demand*. A journey begins when a passenger first boards the public transport system at s_o and ends when that passenger finally alights at s_d. It may contain one or more *vehicle legs*. A vehicle leg is the movement on one vehicle trip between a boarding event and its corresponding alighting event. Two consecutive legs are connected by a transfer; a journey with L legs has $L - 1$ transfers. Boarding the first vehicle is not a transfer.

This distinction is essential when boarding and alighting counts are used for OD estimation. One unit of demand assigned to an L-leg journey contributes one boarding and one alighting on every leg. It therefore generates one initial boarding, one final alighting, $L - 1$ transfer alightings, and $L - 1$ transfer boardings. A transfer boarding is not a new journey origin, and a transfer alighting is not a final journey destination. Consequently, total boardings and alightings are generally not journey-origin and journey-destination marginals in a network with transfers.

The atomic boarding or alighting observation is associated with one timetable event identified by measurement method, event type, stop, clock time, and a trip or an unambiguous line reference. Platform, physical-stop, route-period, or other aggregates are derived views and require an explicit, versioned mapping. The initial reduced-dimensional backend accepts boarding and alighting events; the existing `load` schema value is outside that backend until a corresponding response contract is implemented and tested.

An absent record denotes an unobserved event and creates no likelihood term. A present record whose value is zero is an observed zero and remains in the likelihood. Duplicate, negative, non-finite, unmatched, and ambiguous records are errors; APC cleaning, outage exclusion, and imputation occur before the immutable observation table is constructed and are recorded as input provenance.

The external OD key remains the triplet comprising `origin_stop_id`, `dest_stop_id`, and `time_bin_id`. A platform-to-physical-stop mapping may be used for transfer construction, but it is an explicit, fingerprinted input. An authoritative operator or GTFS parent-station mapping is preferred; a generated mapping requires review and cannot be accepted silently. Published output remains at scenario-stop level unless an explicit aggregation policy requests physical stops.

For a reduced conditional destination model, let Q_{ot} denote the number of journeys whose first boarding occurs at origin o in desired-departure bin t. It must not be set equal to all boarding counts at (o, t) when transfer boardings may be present. The first reduced implementation therefore admits either externally provided journey-entry totals or productions estimated through a declared low-dimensional nonnegative basis with regularization. It does not provide a raw-boarding production mode. Which productions, attractiveness terms, route shares, detection parameters, and dispersion parameters are fixed, normalized, estimated, or regularized must be declared by every model specification.

These definitions constitute the observation contract for the planned reduced-dimensional backend. They do not redirect or replace the existing detailed assignment and estimation APIs. The time-expanded assignment remains the explicit validation backend and is not evaluated inside the reduced objective or gradient.

I.1.1 Physical stops, route patterns, and compact timetable index

The second implementation phase compiles the domain timetable into deterministic host arrays without constructing a time-expanded graph. Scenario stop identifiers remain the external OD and measurement geography. A *physical stop place* is an explicit aggregation of one or more scenario stops, typically platforms belonging to the same passenger interchange. When no mapping is supplied, the implementation uses an identity mapping. A nontrivial mapping must cover every scenario stop exactly, contain no unknown stop, and be labeled either `authoritative` or `reviewed_generated`. Place identifiers and member stop identifiers are canonical and sorted; the representative coordinates are the arithmetic mean of member coordinates. The mapping policy, membership, and coordinates enter its fingerprint.

A route pattern is defined by the tuple

(line identifier, direction identifier, complete ordered physical-stop sequence).

Trips are grouped only when this tuple is identical. Consequently, opposite directions, express and local services, different lines, and repeated-stop patterns remain distinct. Platform variants may become equivalent only through the explicit physical-stop mapping. Every trip belongs to exactly one route pattern, and both pattern and trip ordering are deterministic.

At this stage, a service-period class means a declared timetable `service_id`; the software does not infer calendar semantics or create periods from OD departure bins. Trips without a service identifier belong to one reserved default class. Exact scheduled arrival and departure times remain in the timetable arrays, so after-midnight values above 24 hours are preserved. Later range-routing phases may select trips by clock time without changing these service classes.

The compact index contains sorted string dictionaries for scenario stops, physical stops, trips, and lines. Its numerical data consists of a trip stop-time CSR pointer; aligned trip, scenario-stop, physical-stop, sequence, arrival, and departure arrays for every stop-time record; and one line, direction, route-pattern, and service-period index per trip. Integer seconds use 64-bit storage for timetable values, while bounded dictionary indices use 32-bit storage. All arrays are owned, C-contiguous, and read-only. Input validation rejects duplicate or unknown identifiers, incomplete trips, duplicate sequences, and nonmonotone times before publishing an artifact. Artifact provenance independently fingerprints the source timetable, physical stops, route patterns, service periods, and reduced-OD configuration.

The public Phase-2 CPU benchmark uses 5,000 trips with 15 stops per trip, or 75,000 stop-time records. The complete normalization and array build takes approximately 0.148 seconds, retains 2,520,008 array bytes, and increases the observed process peak by approximately 25.5 MiB. These machine-dependent figures are not test thresholds, but confirm linear timetable storage and the absence of time-expanded assignment state.

Event-aligned measurement resolution. Boarding and alighting mappings have additional structure that avoids generic repeated graph searches. The strict resolver formerly scanned all links for every measurement to find access links entering a matched departure event or egress links leaving a matched arrival event. It now builds node-keyed CSR indices for those relations once and retains all existing stop, time, trip/line, event-kind, uniqueness, and nonempty-link validations. On the full network, synchronized strict mapping decreased from approximately 121.2 s to 2.31 s: 0.24 s for event/link indices, 1.64 s for 426,436 record resolutions, and 0.42 s for result construction.

All 213,218 alighting measurements map to exactly one egress link. Of 213,218 boarding measurements, 207,805 map to one access link and 5,413 map to two access links because two time-bin access alternatives may enter the same departure event. The compact event-aligned representation therefore stores one primary link per measurement plus only these sparse secondary boarding pairs. It is exact; forcing every row to one link would not be. On the full-network arrays, index storage decreased from 3,454,792 to 1,749,048 bytes and warm aggregation decreased from 0.482 to 0.108 ms, with bit-identical predictions and gradients. This

optimization removes mapping startup cost but does not materially change the six-second destination-loading bottleneck.

J Free and Frozen OD Demand Cells

An estimation run may hold a selected subset of OD/time-bin cells at predefined nonnegative values. Let $\mathcal{I} = \{1, \dots, n_{\mathrm{OD}}\}$ be the full assignment indexing, and partition it into the free set $\mathcal{U}$ and the frozen set $\mathcal{F}$, with $\mathcal{U} \cap \mathcal{F} = \varnothing$ and $\mathcal{U} \cup \mathcal{F} = \mathcal{I}$. For each $i \in \mathcal{F}$, let $c_i \geqslant 0$ denote its predefined flow. The demand reconstruction is

$$f_i(z_{\mathcal{U}}) = \begin{cases} f_i^{(0)} \exp(z_i), & i \in \mathcal{U}, \\ c_i, & i \in \mathcal{F}. \end{cases}$$

The smooth bounded transformation described previously is applied to the raw coordinate associated with each z_i, $i \in \mathcal{U}$.

The frozen cells are not represented by dummy parameters. They occur neither in the optimizer vector nor in the Bayesian guide, prior, gradient, Hessian, or covariance matrix. The full vector $\mathbf{f}$ is reconstructed only at the boundary of the assignment model, because assignment necessarily consumes all OD cells. Thus the statistical dimension is exactly

$$d = |\mathcal{U}| + \mathbf{1}\{\theta \text{ is estimated}\},$$

independently of $|\mathcal{F}|$. A frozen value may be zero or positive. In contrast, a free cell requires a strictly positive baseline $f_i^{(0)}$, since the multiplicative parameterization cannot move away from a zero baseline.

Frozen values are supplied by a sparse CSV file with columns `origin_stop_id`, `dest_stop_id`, `time_bin_id`, and optionally `fixed_flow`. A missing or blank `fixed_flow` denotes zero. Keys are validated against the scenario, duplicates are rejected, and the free/frozen partition is constructed in the canonical assignment order. The same immutable partition is passed to ML, MAP, and Bayesian VI. Saved results include the free indices, frozen indices, and frozen values so that downstream processing can verify the invariant. The layout also has its own canonical SHA-256 fingerprint, computed from the OD keys, partition indices, free baselines, and frozen values. This fingerprint is stored separately from the network/assignment fingerprint: changing only the frozen-demand file must therefore change the layout identity even though the network identity remains unchanged. The canonical JSON payload used to compute the digest is persisted with the result. On loading, the digest is recomputed from that payload to detect corruption or manual modification. During post-processing, the layout is rebuilt from the current scenario and `fixed_demand.csv`; its fingerprint must match the result fingerprint before assignment or report generation proceeds. The network fingerprint and layout fingerprint are checked independently.

J.1 Topology-driven structural-zero preprocessing

The fixed-demand file may be generated by a deterministic preprocessing service before ML, MAP, or Bayesian estimation. Its single substantive input is a versioned TOML configuration. The configuration identifies the scenario and output folders, selects rules in a dedicated `rules.enabled` table, and provides a separate table for the parameters of each enabled rule. Unknown keys, missing required keys, invalid types, invalid ranges, and missing input paths are errors. Relative paths are resolved relative to the TOML file.

The service builds the same production time-expanded topology used for assignment, with the configured access-deviation, transfer-wait, and minimum dwell assumptions. A backward dynamic program is performed once per destination and evaluates only OD/time keys already

present in the demand candidate table. It records path feasibility, minimum transfers, minimum initial wait, minimum journey time, number of feasible departures, and earliest arrival. Boarding the first vehicle is not a transfer, and arrival at the destination is absorbing.

Two core rules classify a cell as structural zero when no feasible path exists, or when the minimum feasible transfer count exceeds the accepted maximum. With `max_transfers = 2`, paths requiring three transfers or more are therefore excluded. Optional rules cover identical origin and destination stops, excessive initial wait, excessive journey time, and too few feasible departures. Equality at a configured maximum or minimum is retained. All triggered reasons are saved, while a fixed precedence supplies one primary reason for mutually exclusive summary counts.

An existing fixed-demand file can be reconciled with the detected zeros. The only implemented conflict policy is *error*: a nonzero existing value on a newly detected structural zero stops preprocessing. Compatible zero and positive fixed values are preserved exactly, and newly detected cells are added with `fixed_flow = 0`. Duplicate or unknown keys, negative or non-finite values, and all conflicts are reported without producing a partial merged result.

The output transaction writes a canonically sorted `fixed_demand.csv`, a per-cell audit CSV, a JSON summary, the fully resolved TOML configuration, and scenario, graph, and configuration fingerprints with content hashes. Every file is rendered and validated before atomic replacement. Passing the generated fixed-demand file to estimation removes those structural-zero cells from the parameter vector altogether; they are not zero-valued dummy parameters.

Potentially long phases expose an optional dependency-free progress callback. Immutable events report a stable phase name, completed and total work, elapsed time, and an optional artifact message. The destination-profile, cell-classification, and audit-rendering loops report every item for small problems and use count- or ten-second throttling for large problems, always including zero and the exact final total. A context-managed optional `tqdm` adapter displays one phase at a time on standard error, leaving standard output suitable for machine-readable JSON. Callback exceptions propagate normally, and rendering failures occur before atomic file replacement, so progress reporting does not permit a partial file to replace a valid artifact.

If $|\mathcal{U}| = 0$ and θ is fixed, then $\mathrm{d} = 0$. This is treated as a deterministic evaluation rather than as an optimization or variational-inference problem. The likelihood is evaluated once for validation, while NumPyro SVI and the numerical optimizer are bypassed entirely. Returned parameter, gradient, Hessian, covariance, and posterior-coordinate arrays are correspondingly empty.

J.1.1 Bounded timetable feasibility and feature summaries

RAPTOR denotes *Round-Based Public Transit Routing*. It is a timetable-routing method for finding feasible public-transport journeys and their earliest arrival times without first converting the timetable into a time-expanded graph. Its state is organized by stop and round: round zero contains the origin and its walking reachability, and round `r` contains labels reachable with at most `r` vehicle boardings. Routes reachable from labels improved in the preceding round are scanned in timetable order; newly improved stop labels are then propagated through transfer footpaths. Keeping the best arrival label for each transfer count yields an arrival-versus-transfers Pareto frontier. Unlike graph-based shortest-path algorithms, the standard method operates directly on route and timetable arrays and does not require a global priority queue over timetable events.

In this project RAPTOR is a preprocessing algorithm, not an OD estimator. Its role is to decide which destinations are reachable from an origin and departure time and to summarize the scheduled journeys used later to construct choice sets and measurement responses. Passenger counts and demand parameters do not enter the routing calculation.

The network-level preprocessing therefore begins with a deterministic RAPTOR-style search implemented in `public_transportation.preprocessing.reduced_od.raptor`. It consumes

the compact timetable arrays described above, explicit directed footpaths, a physical origin stop, a departure time, and a maximum number of transfers. It does not construct a global time-expanded graph and does not call either assignment backend.

A search round represents exactly one vehicle boarding. With maximum transfer count K, the algorithm executes $\mathsf{K}+1$ route rounds. Each scheduled trip is scanned once per round in stop order. The scan retains its best currently boardable label and propagates it to downstream stops, so its work is linear in the stop-time records scanned rather than quadratic in possible board/alight pairs. Positive directed footpaths are relaxed without consuming a vehicle round. Transfer depth is consequently bounded by construction.

Within a round, the retained label is the deterministic earliest-arrival label, with waiting time, walking time, and path identity used as tie breakers. Across rounds, the result retains the Pareto frontier in arrival time and number of boardings. Every label reports total elapsed time, waiting, walking, in-vehicle time, transfers, and its scheduled transit legs. The feature identity

$$\mathsf{T}_{\text{elapsed}} = \mathsf{T}_{\text{wait}} + \mathsf{T}_{\text{walk}} + \mathsf{T}_{\text{vehicle}}$$

therefore remains directly auditable. Phase 4 does not enumerate every path with a distinct waiting/walking tradeoff inside one transfer round; bounded choice-set enumeration and pruning belong to Phase 5. The current code is described as RAPTOR-style because it scans every scheduled trip in every bounded round and does not yet apply the standard marked-stop/marked-route pruning or a route-pattern search for the earliest boardable trip. These omissions affect preprocessing speed, not the transfer-round semantics or the returned earliest-arrival labels.

For every absent destination, a topology-only zero/one shortest-path pass over route reachability and footpaths records one fail-closed reason: `no_topological_path`, `exceeds_transfer_limit`, or `no_timetable_feasible_journey`. The origin itself is labeled separately and is not presented as a destination. This distinction prevents a missed scheduled connection from being misreported as a disconnected network. Independent range queries sort and deduplicate their departure instants and return one complete result per instant.

The Phase-4 CPU benchmark used overlapping patterns of at most 20 stops, four departures per pattern, and a three-transfer bound. For 25, 50, 100, and 200 physical stops, mean query times were approximately 0.00065, 0.00091, 0.00124, and 0.00187 seconds, corresponding to about 1,535, 1,101, 803, and 534 queries per second. The compact timetable arrays occupied 3,656, 7,208, 14,664, and 29,576 bytes. These machine-dependent measurements are diagnostic rather than test thresholds; full-network performance must still be measured on its actual pattern, trip, period, and query counts.

J.1.2 Compact journey choices and transfer-event accounting

Phase 5 converts the timetable labels into complete, bounded passenger-journey alternatives in `public_transportation.preprocessing.reduced_od.journey_choices`. This step remains preprocessing: it neither changes demand nor estimates route shares. One choice cell is identified by physical origin, physical destination, and the half-open time period containing the journey's first boarding. A public-transport alternative must contain at least one scheduled vehicle leg; walking-only reachability cannot silently become a transit-demand cell.

For a journey with L vehicle legs, the event sequence contains exactly $2\mathsf{L}$ records. The first vehicle entry is labeled `first_boarding`, the final exit is labeled `final_alighting`, and each of the $\mathsf{L}-1$ internal transfers contributes the consecutive pair

$$\big(\texttt{transfer\_alighting}, \texttt{transfer\_boarding}\big).$$

The two transfer events may use different physical stops when an explicit footpath connects the arrival and departure locations, but the subsequent boarding time cannot precede the preceding alighting. Thus every journey has the same number of boarding and alighting events, and

transfer boardings remain distinguishable from exogenous journey origins. Every event, not merely the journey origin, is assigned to exactly one declared half-open period; this allows one journey to cross time-period boundaries without relabeling its origin.

Candidate labels are deduplicated by a SHA-256 identity containing the query, OD pair, first-boarding period, and complete scheduled leg sequence. Within a cell they are ranked deterministically by the generalized time

$$c_j = T_{\mathrm{elapsed},j} + \gamma_K K_j + (w_{\mathrm{walk}} - 1)T_{\mathrm{walk},j},$$

followed by arrival time, transfers, and identifier. The subtraction in the walking term is important because elapsed time already contains walking: a walking weight of one therefore leaves ordinary elapsed time unchanged. Only the configured maximum number of alternatives is retained, and diagnostics report candidate, retained, and pruned counts as well as the largest pre-pruning cell. The result fingerprint additionally covers every derived feature, route-pattern identifier, event kind, event period, event stop and time, and fixed share; a change in downstream measurement semantics therefore changes the artifact identity even when the scheduled leg sequence itself is unchanged.

Initial alternative shares are fixed metadata for the later estimator. With temperature τ and optional positive route-pattern initialization weight r_j for the first boarded pattern, they are

$$s_j = \frac{r_j \exp[-(c_j - c_{\min})/\tau]}{\sum_k r_k \exp[-(c_k - c_{\min})/\tau]}.$$

Missing route-pattern weights default to one; supplied unknown, nonfinite, or nonpositive weights are rejected. This is the explicit interface through which the route-level initialization may influence network choices. It does not reinterpret route-level IPF output as journey demand, and the shares remain constant within a subsequent statistical fit.

The Phase-5 benchmark measures choice construction after RAPTOR has already finished, so routing and choice-building time are not conflated. Across the 25-, 50-, 100-, and 200-stop synthetic cases it processed approximately 70,000–77,000 choice cells per second. The fixture retained one Pareto alternative per cell and used approximately 607–655 bytes of full canonical feature, leg, event, and share payload per retained alternative. Multi-route unit tests separately exercise two-alternative cells, transfer-event pairing, route-weighted shares, deterministic pruning, and journeys crossing period boundaries. The benchmark values are operational diagnostics, not performance thresholds.

J.1.3 Direct measurement responses and fail-closed cache

Phase 6 constructs the observation operator directly from the complete journey events in Section J.1.2. The implementation is split between `public_transportation.preprocessing.reduced_od.response_atoms`, `equivalence`, and `persistence`. It never materializes or applies the detailed OD-to-link assignment matrix. Consequently this preprocessing step does not import the assignment, sharding, microsharding, or block-coordinate implementations.

Each supplied atomic boarding or alighting record is resolved against the compact timetable arrays using its scenario stop, event time, trip or line, and measurement type. It must identify exactly one scheduled trip event. Unknown, unmatched, ambiguous, negative, or nonfinite records are rejected, and load measurements remain unsupported by this first reduced backend. A missing sensor creates no measurement row, whereas a supplied numerical zero remains an observed row with target zero. Two different collection methods may map to the same scheduled event, but they remain two distinct measurement rows and are never silently summed.

Let c index one journey OD/first-boarding-period cell, let $j \in J_c$ index its fixed alternatives, and let $s_{j|c}$ be the Phase-5 initial share. If n_{mj} is the number of boarding or alighting events of alternative j that match atomic measurement m, the direct response coefficient is

$$B_{mc} = \sum_{j \in J_c} s_{j|c} n_{mj}.$$

Only positive coefficients are stored, in canonical measurement-major COO order. For free journey demand x and fixed positive demand x^{fix}, prediction is

$$\widehat{y} = b^{\mathrm{fix}} + Bx, \qquad b^{\mathrm{fix}}_m = \sum_{c \in C_{\mathrm{fix}}} B_{mc} x^{\mathrm{fix}}_c.$$

A frozen cell has no column in B. A frozen positive value contributes only to the fixed offset; a frozen zero contributes neither a free coordinate nor an offset. In particular, an unreachable structural-zero key is accepted without a journey choice when its fixed value is zero; a positive fixed value without a feasible journey is rejected. Thus estimator dimension depends on free cells, not on the size of the full OD table.

Free cells are placed in the same exact response-equivalence class only when their ordered measurement indices and the bytes of every float64 coefficient are identical. Empty columns are therefore equivalent to one another, but an approximately equal column is not compressed. The artifact retains the class of every free cell, its representative, and a CSR-style member list. Phase 7 uses this partition to construct bounded operators without changing predictions.

The compressed cache is written atomically as an NPZ archive without pickle payloads. Its metadata includes configuration, timetable, journey-choice, and measurement fingerprints, canonical free/fixed keys, resolved atomic records, and a fingerprint of every numerical array. Loading reconstructs all immutable contracts, recomputes the complete artifact fingerprint, and may require exact expected upstream fingerprints. Missing members, unsupported schema versions, corrupt archives, modified contents, or stale identities fail closed.

The Phase-6 benchmark used a boarding and alighting record for every synthetic stop-time event. For 25, 50, 100, and 200 stops, respectively, 216, 432, 880, and 1,776 measurement rows were processed in approximately 0.0037, 0.0071, 0.0137, and 0.0282 seconds. The responses contained 58, 110, 110, and 110 nonzeros, while compressed cache files occupied approximately 5.3, 7.5, 11.4, and 19.3 KB. Every full-sensor response column in this fixture was distinct, so its equivalence ratio was one. A separate partial-sensor test verifies compression of genuinely identical columns. These measurements isolate preprocessing and cache construction; they are not full-network thresholds.

J.1.4 Exact reduced operator and basis response

Phase 7 exposes the Phase-6 response through `public_transportation.inference.reduced_od.response_operator`. The operator provides NumPy and JAX forward products, transpose products, and fixed offset prediction without reconstructing a full OD table or an OD-to-link matrix. Before accepting the Phase-6 equivalence partition, it recomputes the column signatures and requires exact agreement.

Let $e(c)$ be the exact response class of free cell c, and let $\bar{B}$ retain one representative column per class. Since all columns in a class are identical,

$$Bx = \bar{B}\bar{x}, \qquad \bar{x}_e = \sum_{c : e(c) = e} x_c.$$

The transpose product first computes $\bar{B}^{\mathsf{T}}\nu$ and then assigns the resulting class component to every member cell. Therefore

$$\nu^{\mathsf{T}}(Bx) = x^{\mathsf{T}}(B^{\mathsf{T}}\nu)$$

holds without storing duplicate response columns. The external input and gradient dimensions still contain every free cell; equivalence compression is an exact internal product optimization and does not by itself merge cells with different destination-choice features or denominator membership.

For a declared linear basis $x = \Phi z$, the implementation constructs

$$H = B\Phi = \bar{B}\bar{\Phi}, \qquad \bar{\Phi}_{e\cdot} = \sum_{c : e(c) = e} \Phi_{c\cdot},$$

directly from response atoms and class-aggregated basis rows. It never forms the detailed assignment operator A. Both dense and SciPy sparse bases are accepted. The result may be retained as dense or CSR storage, or selected automatically from its measured density. Although the physical response B is nonnegative, a centered or contrast-coded basis may give signed entries in H; the generic reduced operator therefore accepts finite nonzero signed coefficients.

JAX products use scatter-add operations over the same compressed arrays, so automatic differentiation returns the implemented transpose product. Tests verify dense-reference equality, the adjoint identity, dense and sparse basis responses, storage selection, JIT products, reverse-mode gradients, fixed offsets, and the empty-free-cell case.

The Phase-7 benchmark used 2,000 measurements, six nonzeros per original cell, a response-class ratio of 0.25, and a sparse 16-parameter basis. For 1,000, 5,000, and 20,000 free cells, compressed products averaged approximately 0.000009, 0.000031, and 0.000125 seconds. Operator construction took about 0.006, 0.028, and 0.145 seconds and retained 60, 236, and 896 KB. The direct basis response was built in approximately 0.011–0.026 seconds; its selected dense $2{,}000 \times 16$ representation occupied 256 KB. These synthetic figures demonstrate scaling with compact response and basis sizes, not with a time-expanded network; full-network response density must still be measured.

J.1.5 Differentiable dynamic programming formulation

Fix an OD pair $\mathsf{q} = (\mathsf{s_o}, \mathsf{s_d})$ and a departure interval t. The *terminal* node is the destination centroid-out node $\mathsf{s}_\mathsf{d}^{\text{out}}$, and the value function is initialized as

$$\mathsf{V}(\mathsf{s}_\mathsf{d}^{\text{out}}) = 0.$$

For any other node i, let $\mathcal{A}_\mathsf{q}^+(\mathsf{i})$ denote the set of outgoing links that are enabled for the destination group. We define

$$\mathsf{V}(\mathsf{i}) = -\theta \log \sum_{\ell=(\mathsf{i}\to\mathsf{j})\in\mathcal{A}_\mathsf{q}^+(\mathsf{i})} \exp\left(-\frac{\mathsf{c}_\ell + \mathsf{V}(\mathsf{j})}{\theta}\right).$$

This recursion is evaluated in reverse topological order using a numerically stable `logsumexp` formulation.

The probability that a passenger located at node i chooses link $\ell = (\mathsf{i} \to \mathsf{j})$ is

$$\mathsf{P}_\ell(\mathsf{i}) = \frac{\exp\left(-\frac{\mathsf{c}_\ell + \mathsf{V}(\mathsf{j})}{\theta}\right)}{\sum_{(\mathsf{i}\to\mathsf{k})\in\mathcal{A}_\mathsf{q}^+(\mathsf{i})} \exp\left(-\frac{\mathsf{c}_{\mathsf{ik}} + \mathsf{V}(\mathsf{k})}{\theta}\right)}.$$

J.1.6 Direct fixed-routing measurement operator

Fixed routing permits a second, more aggressive optimization when inference needs measurement predictions but not link-level outputs. For fixed transition probabilities, every forward DAG update consists only of additions and multiplication by constants. It is therefore linear in the injected OD demand. Measurement aggregation is also linear. Their composition can be represented by a matrix $\boldsymbol{A}$:

$$\boldsymbol{\lambda} = \boldsymbol{A}\mathbf{f}, \qquad \boldsymbol{\mu} = \rho\boldsymbol{A}\mathbf{f}.$$

This identity is exact, rather than a local linearization.

For a compact layout containing free cells and positive frozen cells, the implemented representation separates variable columns from constants:

$$\boldsymbol{\mu} = \rho\left(\boldsymbol{A}_{\text{free}}\mathbf{f}_{\text{free}} + \mathbf{b}_{\text{fixed}}\right), \qquad \mathbf{b}_{\text{fixed}} = \boldsymbol{A}_{\text{fixed}}\mathbf{f}_{\text{fixed}}.$$

Frozen-zero cells have neither columns nor an offset contribution. Operator construction proceeds destination group by destination group, but does not construct unit-demand link-flow columns. Instead it combines loading and measurement aggregation in one reverse dynamic program. For destination group g, let $H_g(i, m)$ be the expected accumulated contribution to measurement m of one passenger starting at node i. In reverse topological order,

$$H_g(i, m) = \sum_{\ell \in \delta^+(i)} P_{g\ell} \left[\mathbf{1}\{\ell \in \mathcal{A}_m\} + H_g(h(\ell), m) \right],$$

where $h(\ell)$ is the head of link ℓ. Operator columns are obtained by gathering H_g at the OD origin nodes. Consequently, no `chunk_size`-by-$|\mathcal{A}|$ link-flow array is constructed or transferred to the host.

Only measurements mapped to links enabled for a destination group are carried by that group's recursion. The local measurement dimension is padded to a single fixed maximum, and every OD output chunk is padded to the configured chunk size. The builder explicitly lowers and compiles this fixed-shape kernel once, then reuses the executable for every group and chunk, including the final partial chunk. For BCOO output, nonzero data and row/column indices are appended directly from returned local contributions; a dense operator is never allocated and subsequently converted.

Explicit fixed-routing adjoint. Generic reverse-mode differentiation through the node-loading scan retains more state than is necessary when link probabilities are fixed. Let c_ℓ be the cotangent of link flow and let a_i be the node-flow adjoint. A reverse topological pass evaluates

$$a_i = \sum_{\ell \in \delta^+(i)} P_\ell \left(c_\ell + a_{h(\ell)} \right).$$

The derivative with respect to initial flow injected at node i is a_i; the existing OD injection operation then gathers this adjoint at origin nodes. The custom vector–Jacobian product therefore retains a node vector and the supplied link cotangent rather than the complete forward scan tape. It deliberately defines no derivative with respect to routing probabilities and is exposed only for the fixed-routing path.

On the representative 465-cell full-network destination, ordinary reverse mode and the explicit adjoint required respectively 12.09 and 6.69 s for a warm value–gradient evaluation. Their resident memory immediately after that evaluation was 16.21 and 13.91 GiB. Objective, prediction, and gradient diagnostics agreed within float32 tolerances. The explicit adjoint is retained as the fixed-routing differentiation path and as a reference for validating future sparse-operator construction.

J.1.7 Route-level iterative proportional fitting baseline

The first numerical reduced-OD baseline estimates vehicle-leg OD flows for one route pattern and service class. It is intentionally simpler than the network journey model and is implemented in `public_transportation.inference.reduced_od.route_level`. Consider an ordered route with stops $1, \ldots, n$. Let x_{ij} be the passenger flow that boards at stop i and alights at stop j. The structural support is strictly upper triangular:

$$x_{ij} \geqslant 0 \quad \text{for } i < j, \qquad x_{ij} = 0 \quad \text{for } i \geqslant j.$$

For observed boarding marginals b_i and alighting marginals a_j, iterative proportional fitting alternately scales constrained rows and columns toward

$$\sum_{j > i} x_{ij} = b_i, \qquad \sum_{i < j} x_{ij} = a_j.$$

A supplied seed must be finite and strictly positive on every structurally allowed cell and zero elsewhere. The default seed is uniform on the allowed support. Iteration stops only when the maximum observed-marginal residual, scaled by $\max(1, \text{target})$, meets the declared tolerance.

Missing observations are represented by explicit Boolean masks. An unobserved row or column is not scaled and is not interpreted as zero; its fitted value remains dependent on the seed and other observed constraints. The result is therefore marked underdetermined whenever either set of marginals is incomplete. An explicit observed zero, by contrast, removes all flow from its constrained row or column.

When all boardings and alightings are observed, their totals must agree. The default policy reports unequal totals as infeasible. The only corrective policy implemented at this stage is explicit: `average_observed_totals` rescales each complete marginal vector proportionally to the arithmetic mean of the two totals. The original imbalance, reconciled totals, largest marginal adjustment, and a warning are retained in the data-quality report; reconciliation is never silent. A positive total cannot be reconciled with a zero total.

Complete marginals are checked against the triangular support before iteration. In particular, boarding at the final stop and alighting at the first stop must be zero. For every destination prefix through stop k, its cumulative alightings cannot exceed boardings at strictly upstream stops:

$$\sum_{j=1}^{k} a_j \leqslant \sum_{i=1}^{k-1} b_i.$$

Violations, exhausted positive support, and nonconvergence raise a dedicated exception carrying the data-quality report.

For a positive fitted boarding marginal, the route-level alighting probability is

$$p_{ij} = \frac{x_{ij}}{\sum_{k>i} x_{ik}}.$$

Zero-flow rows receive zero probabilities. Results expose immutable matrices, fitted and reconciled marginals, iterations, absolute and relative residuals, support size, coverage, and reconciliation diagnostics.

These outputs are explicitly labeled `leg_level`, with boarding semantics `leg_boarding_unclassified`, and are marked incompatible with journey OD. A boarding observed on a route may be an initial boarding or a transfer boarding; route-level counts alone do not identify that distinction. The IPF result is therefore an audit baseline and initializer, not a procedure for stitching complete multi-line passenger journeys.

The Phase-3 CPU microbenchmark generated feasible dense triangular marginals. Routes of 10, 25, 50, and 100 stops converged respectively in 14, 12, 12, and 11 iterations. Their measured wall times were approximately 0.00059, 0.00078, 0.00131, and 0.00226 seconds, and their dense result matrices occupied 800, 5,000, 20,000, and 80,000 bytes. These machine-dependent values are not test thresholds; they show that this diagnostic baseline is negligible relative to later network journey preprocessing.

J.1.8 Balanced and unbalanced entropy baselines

Phase 8 provides an optional sparse entropic-transport baseline in `public_transportation.inference.reduced_od.entropy`. Its marginal contract is deliberately strict: both vectors must represent passenger-journey origins and passenger-journey destinations. Raw APC boarding and alighting totals are rejected because transfer events make them different quantities. Consequently entropy is useful when journey marginals are externally justified or in controlled synthetic validation; it is not automatically the primary TPG estimator.

For sparse feasible support C, generalized cost c_{od}, and regularization $\varepsilon > 0$, balanced

entropy estimates

$$\min_{x \geqslant 0} \sum_{(o,d) \in C} c_{od} x_{od} + \varepsilon \sum_{(o,d) \in C} x_{od}(\log x_{od} - 1)$$

subject to the supplied origin and destination journey marginals. A log-domain Sinkhorn iteration updates origin and destination scalings on the sparse cell list. Unequal balanced totals, positive unsupported marginals, and nonconvergence fail diagnostically. Warm scaling vectors may be reused only when their dimensions and values are valid.

The unbalanced variant replaces exact marginal constraints by Kullback–Leibler penalties with separately declared positive origin and destination strengths. Its scaling powers are

$$\alpha_o = \frac{\tau_o}{\tau_o + \varepsilon}, \qquad \alpha_d = \frac{\tau_d}{\tau_d + \varepsilon}.$$

It may therefore reconcile inconsistent totals, but the fitted deviations and penalties remain part of the diagnostics; unbalanced transport does not turn APC leg-event totals into journey marginals. Segmented log-sum-exp uses one scatter pass over the support, giving work proportional to support cells plus marginal groups per iteration.

The Phase-8 benchmark used balanced dense-block supports of approximately 1,000, 5,000, and 20,000 cells. All cases converged in ten iterations, taking about 0.0009, 0.0025, and 0.0080 seconds, or approximately 1.1–2.5 million support cells per completed solve-second as reported by the benchmark convention. Residuals were below 4.1×10^{-8}. These small regular supports do not replace full-network tests of sparse connectivity and penalty sensitivity.

J.1.9 Minimal conditional gravity model

Phase 9 implements the primary reduced journey-demand model in the remaining modules of `public_transportation.inference.reduced_od`. There is one feature row per free Phase-6 response cell. Fixed alternative shares supply expected scheduled travel time T_c and expected transfers K_c; externally declared positive attractiveness is a_c. Within origin/first-boarding-period group g, utility and conditional probability are

$$V_c = \log a_c - \beta_T \frac{T_c}{T_{\text{scale}}} - \beta_K K_c, \qquad p_c = \frac{\exp(V_c)}{\sum_{k \in g} \exp(V_k)}.$$

The segmented softmax subtracts the group maximum before exponentiation. Structural-zero and frozen cells are absent from the free feature table; they do not receive zero-probability parameter slots.

With externally supplied journey production q_g, demand is $x_c = q_g p_c$. The alternative `estimated_basis` mode uses a declared $G \times P$ production basis Z and

$$q_g = q_g^{(0)} \exp\big((Z\gamma)_g\big).$$

It therefore adds P production coefficients rather than one parameter per origin-time group. The minimal Poisson model has only β_T and β_K; negative binomial adds one positive dispersion parameter; estimated production adds only the explicitly declared basis dimension. Positive behavioral and dispersion parameters use smooth softplus transformations.

Measurement means are evaluated entirely through the Phase-7 operator,

$$\mu = \rho\big(B x + b^{\text{fix}}\big),$$

with a positive numerical mean floor. Both Poisson and mean/dispersion negative-binomial log likelihoods use the existing stable JAX kernels. Exact gradients differentiate grouped demand, the compressed response product, and the fixed-offset likelihood without reconstructing full OD demand or invoking detailed assignment.

An entropy plan can initialize gravity without changing either model's semantics. Conditional entropy flows are projected by least squares onto log-attractiveness, scaled time, transfers, and origin-time intercepts. When production is basis-estimated, entropy group totals additionally initialize γ through the declared basis. Tests verify normalization, extreme utilities, fixed offsets, Poisson finite differences, negative-binomial gradients, basis productions, entropy initialization, and recovery of a synthetic two-parameter truth.

With 2,000 measurements and three response nonzeros per cell, the Phase-9 JAX benchmark compiled a two-parameter Poisson value/gradient in approximately 0.12–0.14 seconds. Warm evaluations took about 0.000067, 0.000171, and 0.000581 seconds for 1,000, 5,000, and 20,000 free cells, respectively. Thus warm cost scales with compact cells and response nonzeros while parameter dimension stays fixed. Cold compilation, warm evaluation, and preprocessing are reported separately.

Modeling choice (smoothly bounded log-deviation). For each OD/time-bin cell, introduce an unconstrained latent variable $\widetilde{z}_q^t \in \mathbb{R}$. To protect the exponentiation and the downstream assignment from extreme values while retaining a smooth gradient, define the effective log-deviation as

$$z_q^t = z_{\max} \tanh\left(\frac{\widetilde{z}_q^t}{z_{\max}}\right), \qquad z_{\max} > 0.$$

The demand used by the assignment is then

$$f_q^t = f_q^{t,(0)} \exp(z_q^t).$$

This transformation guarantees $f_q^t \geqslant 0$ whenever $f_q^{t,(0)} \geqslant 0$ and preserves the baseline structure up to multiplicative adjustments. It also implies

$$|z_q^t| \leqslant z_{\max}, \qquad \exp(-z_{\max}) \leqslant \frac{f_q^t}{f_q^{t,(0)}} \leqslant \exp(z_{\max})$$

for positive baseline cells. The default $z_{\max} = 6$ therefore permits demand multipliers ranging approximately from 1/403 to 403. In the software configuration, this smooth bound is exposed under the historical name `z_clip`; despite that name, the implementation uses a smooth hyperbolic-tangent transformation rather than hard clipping.

J.2 Reduced-dimensional gravity demand estimation

The public inference layer now provides a minimal dynamic gravity model on the canonical sparse list of feasible OD–departure-time cells. For origin–time production Q_{ot} and fixed positive destination attractiveness A_{dt},

$$q_{odt} = Q_{ot} \frac{I_{odt} A_{dt} \exp\{-\beta_{\text{time}} \widetilde{T}_{odt} - \beta_{\text{transfer}} N_{odt}\}}{\sum_{d'} I_{od't} A_{d't} \exp\{-\beta_{\text{time}} \widetilde{T}_{od't} - \beta_{\text{transfer}} N_{od't}\}}.$$

Masked segmented JAX reductions preserve exact structural zeros and the production totals without materializing a dense origin–destination–time tensor. The time and transfer coefficients, and the negative-binomial dispersion, are mapped from unconstrained optimizer coordinates through stable positive transformations.

Gravity demand is passed directly through the existing dense or BCOO fixed-routing measurement operator, including its frozen-positive measurement offset. The observation layer supports Poisson likelihood and the negative-binomial parameterization $\mathrm{Var}(Y_i) = \mu_i + \mu_i^2/r$. For small parameter counts, a batched-forward derivative applies the routing operator to all demand directions together. The adjoint alternative applies one routing transpose product and a JAX vector–Jacobian product. Neither constructs a dense OD-by-parameter routing Jacobian.

The minimal estimator uses an explicitly traced, lowered, and compiled JAX value-and-gradient kernel with L-BFGS. Its automatic policy benchmarks first and warm executions of the two derivative strategies under a bounded preflight. Atomic checkpoints are written at the initial state and after every completed iteration; their fingerprint covers gravity inputs, model and parameter layouts, routing and measurement provenance, observations, and likelihood semantics. Resume reconstructs the full OD layout, including frozen-positive cells and structural zeros, while restarting non-portable L-BFGS history from the saved iterate. Compilation, first execution, warm execution, strategy selection, persistent-cache configuration, and elapsed optimization time are exposed as structured diagnostics.

A detailed OD table produced this way is not equivalent to estimating every OD cell independently: its detail is induced by the low-dimensional gravity structure and the fixed production and attractiveness inputs.

J.2.1 Reduced-model ML/MAP estimation and restart operations

Phase 10 fits the minimal gravity model through `public_transportation.inference.reduced_od.estimator`. Maximum likelihood minimizes the negative log likelihood from Section J.1.9. MAP uses the same compiled objective and adds independent Gaussian penalties on the *raw*, unconstrained parameter vector $\boldsymbol{\eta}$:

$$Q_{\mathrm{MAP}}(\boldsymbol{\eta}) = -\ell(\boldsymbol{\eta}) + \frac{1}{2}\sum_{j}\left(\frac{\eta_j - m_j}{s_j}\right)^2.$$

The softplus transformation described above is applied after this raw-vector prior. Thus a prior intended for a transformed behavioral coefficient must be converted deliberately rather than being interpreted as a raw Gaussian. Setting every $s_j = +\infty$ gives an exactly flat penalty and makes MAP numerically identical to ML. A finite informative prior generally changes the estimate, especially in weakly identified directions; neither estimate should be expected to equal a Bayesian posterior mean.

SciPy L-BFGS-B drives a JIT-compiled JAX value-and-gradient calculation. The optimizer dimension is the small number of gravity and production-basis parameters, not the number of feasible OD cells. Objective evaluations retain only grouped demand and compact response arrays. They cannot call detailed assignment, reconstruct a full OD vector, or write per-cell iteration output. Compilation and optimization times, evaluation and iteration counts, final gradient norm, likelihood, prior contribution, method, and model fingerprint are recorded separately.

The operational layer in `operations.py` defines immutable fit manifests and atomic JSON checkpoints. A checkpoint contains the model fingerprint, method, parameter dimension, latest raw parameters, objective, iteration, and elapsed time. Resume first requires exact manifest compatibility; a cache from a different problem, statistical model, method, or dimension fails closed. Progress callbacks and checkpoint writes occur only at declared iteration boundaries. A deadline produces status `deadline`, saves a resumable checkpoint, and never claims successful completion. Result files likewise carry an explicit `complete`, `deadline`, or `failed` status.

On the public 20,000-cell, 2,000-measurement synthetic benchmark, the two-parameter ML fit compiled in about 0.13 seconds and completed 12 L-BFGS iterations (30 value/gradient evaluations) in about 0.021 seconds after compilation. Writing a small checkpoint at every iteration did not dominate this run; the final checkpoint was 237 bytes. These figures demonstrate that the operational wrapper has no visible cell-wise Python loop, but they remain machine-dependent and do not replace the full-network admission benchmark.

Prior on θ. The dispersion parameter θ is strictly positive. When it is estimated, the implementation introduces an unconstrained raw variable $\widetilde{u} \in \mathbb{R}$ and defines

$$u = u_{\max} \tanh\left(\frac{\widetilde{u}}{u_{\max}}\right), \qquad \theta = \exp(u), \qquad u_{\max} > 0.$$

Thus θ is positive and smoothly bounded between $\exp(-u_{\max})$ and $\exp(u_{\max})$. The configuration currently exposes $u_{\max}$ under the historical name `u_clip`, although no hard clipping is performed.

A Gaussian prior is placed on the raw variable,

$$\widetilde{u} \sim \mathcal{N}(\mu_{\widetilde{u}}, \sigma_u^2).$$

If the requested center in effective log-dispersion space is μ_u—for example, $\mu_u = \log(\theta^{(0)})$ for a baseline value $\theta^{(0)}$—the raw prior center is chosen by the inverse transformation,

$$\mu_{\widetilde{u}} = u_{\max} \operatorname{arctanh}\left(\frac{\mu_u}{u_{\max}}\right).$$

This guarantees that transforming the raw prior center gives exactly the requested effective center. It requires $|\mu_u| < u_{\max}$.

J.2.2 Adequacy, grouped holdout, and advisory relaxation

Phase 12 separates three questions that must not be conflated. Full-data *adequacy* asks how well a fitted model reproduces the observations used to estimate it. Grouped *holdout validation* asks how well a model refitted without selected observations predicts those untouched observations. An *advisory relaxation score* identifies a possible next model but neither constructs nor fits that model automatically. The implementation is divided between `validation.py`, `diagnostics.py`, and `lineage.py` in the reduced-OD inference package.

The minimal objective now accepts an optional immutable calibration mask. Its default remains all measurements, so Phases 9–11 are unchanged. During a holdout fit, likelihood contributions outside the calibration mask are replaced by zero before summation. Predicted means are nevertheless returned for every measurement row. Consequently changing holdout counts cannot change the calibration objective, gradient, or estimate; a regression test changes held-out counts by 10,000 and obtains exactly the same fitted raw parameters.

Holdout construction is deterministic from a measurement identity, seed, and group labels. Supported grouping units include complete vehicle journeys, stop time series, lines, directions, time blocks, and explicit groups. Every group lies wholly in calibration or holdout, both subsets must be nonempty, and their masks are complementary. In particular, boarding and alighting rows from one held-out vehicle journey cannot leak individually into calibration. Calibration and holdout summaries report their own counts, observed and modeled totals, likelihood, Poisson and negative-binomial deviances, MAE, RMSE, and variance-weighted RMSE. For a Poisson model the negative-binomial deviance is reported at its Poisson limit; a fitted negative-binomial model uses its fitted dispersion.

Full-data adequacy reports raw and standardized residuals, threshold exceedance rates, and grouped summaries for available measurement type, line, direction, stop, period, vehicle journey, origin zone, destination zone, and transfer place labels. These are calibration diagnostics and the report refuses a masked problem rather than presenting calibration fit as predictive evidence. The Hessian of the small raw-parameter objective supplies minimum and maximum curvature eigenvalues and a stabilized condition number. Near-singular curvature, a large condition number, and too few observations per fitted parameter generate explicit weak-identification warnings.

Residual patterns may recommend exactly one of four conventional children: J1, broad-period cost effects; J2, coarse destination-zone attraction; J3, coarse origin-zone production;

or J4, selected transfer-place correction. A candidate is applicable only when its declared grouping metadata contains at least two groups. Its score is based on the largest grouped mean standardized residual and is reduced when observation support per added parameter is weak. The recommendation object is permanently labeled `advisory_only`; it does not alter J0, add parameters to the Phase-9 objective, select a model, or claim predictive improvement. Implementing and estimating these children is a later model-extension decision.

Lineage nevertheless prepares that decision safely. A fitted J0 root has a fingerprinted identity, parameter names, and estimates. Each proposed J1–J4 child records its parent, appends only explicitly named zero-effect parameters, and is accepted as a warm start only after the caller demonstrates that the child prediction at those zeros reproduces the parent prediction within a declared tolerance. Such nodes remain marked unfitted. Thus every proposed child is auditable without silently turning a diagnostic recommendation into an estimation run.

On the public 20,000-cell, 2,000-measurement benchmark, cold adequacy including the first Hessian compilation took about 2.18 seconds and the warm repeated adequacy calculation about 0.046 seconds. Four advisory candidates were scored in about 0.00018 seconds, and a vehicle-journey-grouped 20-percent holdout refit took about 0.153 seconds after the relevant JAX shapes had been compiled. These diagnostic costs remain separate from both preprocessing and detailed-assignment validation.

J.2.3 Explicit OD reconstruction and detailed validation

Phase 11 keeps reconstruction strictly outside estimation. The fitted compact demand vector is aligned with the canonical free-cell keys and inserted into the full problem contract only when `public_transportation.inference.reduced_od.reconstruction` is called explicitly. Frozen positive values are copied unchanged, structural-zero cells remain zero, and every output row records whether its value was estimated or fixed. A mismatch in free-cell order, dimension, or canonical key identity is an error rather than an implicit reorder.

The same module exposes a post-fit detailed-assignment adapter. The caller supplies a function that accepts the reconstructed OD object and returns measurement predictions plus named transfer-audit quantities. The adapter invokes that function exactly once and compares its output with the compact prediction using the difference vector, maximum absolute error, root mean square error, and relative ℓ_2 error. The detailed assignment configuration is deliberately captured by the caller: it is not part of the optimizer and cannot be reached from an objective evaluation. Detailed-assignment time and memory must consequently be reported as validation cost, never as fitting cost.

The public benchmark reconstructed 20,000 canonical all-free cells in about 0.010 seconds and allocated 160,000 bytes for the demand vector. Tests also cover mixed free, frozen-positive, and frozen-zero layouts, rejection of a noncanonical free order, a single detailed-assignment invocation, and the separation between the estimator and all post-fit operations.

J.2.4 Scalability limits for very large OD layouts

The direct operator is not suitable when its dimensions or its construction state exceed the available memory. In the TPG full-network case there are 628,164 free OD cells and 426,436 measurements. A dense float32 operator would require approximately 998 GiB. More importantly, the reverse operator construction described above would require a destination-local state proportional to the number of graph nodes times the number of reachable measurements. Measurements from a sampled destination cover approximately 213,000 observations, implying about 344 GiB of value state for one destination. The explicit builder must therefore not be used for this layout.

Six isolated full-network destinations spanning 9, 137, 276, 465, 770, and 1,181 free cells were measured under a 24 GiB process ceiling. Warm forward loading was 5.98–6.08 s and ordinary value–gradient evaluation was 12.07–12.38 s; peak resident memory was 16.02–18.16 GiB.

The near-constant runtime over a 131-fold change in OD count shows that graph traversal, not demand injection, dominates. These measurements establish that destination streaming alone is not a viable large-scale solver: it repeats expensive graph traversals and does not exploit the linear structure available when routing is fixed. The large-scale mode should instead expose the fixed-routing assignment as a sparse linear operator and solve the resulting explicitly regularized, bound-constrained least-squares problem.

References

Cameron, A. C. and Trivedi, P. K. (2013). *Regression Analysis of Count Data*, second edn, Cambridge University Press.

Chen, X., Cheng, Z. and Sun, L. (2025). Bayesian inference of time-varying origin–destination matrices from boarding and alighting counts for transit services, *Transportation Research Part B: Methodological* **199**: 103278.

Dial, R. B. (1971). A probabilistic multipath traffic assignment model which obviates path enumeration, *Transportation research* **5**(2): 83–111.

Lambert, D. (1992). Zero-inflated poisson regression, with an application to defects in manufacturing, *Technometrics* **34**(1): 1–14.

Menon, A. K., Cai, C., Wang, W., Wen, T. and Chen, F. (2015). Fine-grained OD estimation with automated zoning and sparsity regularisation, *Transportation Research Part B: Methodological* **80**: 150–172.

Santos Silva, J. M. C. and Tenreyro, S. (2006). The log of gravity, *The Review of Economics and Statistics* **88**(4): 641–658.

Yuan, S., Tan, P.-N., Cheruvelil, K. S., Collins, S. M. and Soranno, P. A. (2019). Spatially constrained spectral clustering algorithms for region delineation, *arXiv preprint arXiv:1905.08451* .